

Neovim User Guide

**Leader key: Space. Keys, commands and workflows of
this config.**

Contents

Chapter 1: Day-to-Day Cheat Sheet (section 2)	14
Navigation	14
Copy, delete and move	14
Indented blocks	15
Text objects (what to delete, change or copy)	15
Macros	15
Bulk rename and replace	15
Undo, redo and repeat	16
Visual block (many lines at once)	16
Comments and surrounding pairs	16
Search in a file	16
Buffers, splits, tabs	17
Save and quit	17
Sessions, dashboard and zoxide	18
Code intelligence (LSP)	18
Git day to day	18
Mouse	19
Language guides (tools that exist only for one language)	19
Neovide (Neovim in its own window)	19
Lists with a search bar (pickers)	19
Quickfix list (search results, errors)	20
Spell checking	20
Run code and terminal	20
Python debugger keys	20
Keyboards without function keys, Insert or Page keys	21
Folding	21
Text tricks	21
Claude and fuzzy finders	22
Other most-used keys (from the former Quick Reference)	22
Chapter 2: Basics: modes, saving, recovering, getting help	23
1. Understanding Modes	23
Entering Insert Mode	23
Leaving Insert Mode	23
Visual Mode Variants	23
69. Tips for Vim Beginners	24
The Most Important Habits	24
Common Mistakes and How to Fix Them	24
Keyboard Layouts With Dead Keys (for example US International)	24
Learning Path	25
70. The Verb + Noun System (How Vim Commands Work)	25
Operators (Verbs)	26
Motions (Nouns)	26
Text Objects (Structured Nouns)	27
Combining Verbs and Nouns	27
Using Counts	28

Why This Matters	28
72. Saving, Quitting, and File State	29
Saving	29
Quitting	29
Closing Buffers (Without Quitting Neovim)	29
73. Recovering from Mistakes	29
Undo and Redo	29
Undo Tree (builtin)	30
Time-Based Undo	30
If You Accidentally Deleted a File	30
74. Discovering Keymaps and Getting Help	30
Which-Key: See Available Keybindings	30
Browse All Keymaps	31
Getting Help	31
Checking System Health	31
Chapter 3: Navigation	32
3. Core Navigation (Moving Without the Mouse)	32
Basic Cursor Movement	32
Moving Within a Line	32
Moving by Word	33
Moving by Line/Screen	33
Jumping to Matching Brackets/Parentheses	33
Jumping to Specific Characters	34
Jump Navigation with hop.nvim (Plugin)	34
Jump History	34
Word References (vim-illuminate)	34
Marks (Bookmarks)	35
22. Jump Navigation (hop.nvim)	35
23. Search Lens (nvim-hls-lens)	35
37. Symbol Outline (aerial.nvim)	35
50. Code Navigation Strategies	35
Finding Files	36
Finding Code	36
Understanding Code	36
Refactoring Code	37
Chapter 4: Editing, text objects, macros and tricks	38
4. Editing	38
Entering Insert Mode for Editing	38
Deleting Text	38
Copying (Yanking) Text	39
Pasting Text	39
Changing (Delete + Enter Insert)	39
Replacing Text	40
Undo / Redo	40
Repeating Actions	40
Line Operations	40
Indentation	41

Insert Mode Shortcuts	41
Miscellaneous Editing	42
5. Selection (Visual Mode)	42
Selecting Characters	42
Selecting Lines	42
Selecting Blocks (Columns)	43
Selecting Inside/Around Delimiters (Text Objects)	43
Parentheses, Brackets, Braces	43
Quotes	43
Words, Lines, Paragraphs	44
Tags (HTML/XML)	44
Using with Operators (d, c, y)	44
Treesitter Node Selection (builtin)	44
More Text Objects	45
Markdown Code Block Text Objects	45
Precision Selection: From the Cursor to an Exact Spot	45
Quick lookup	46
End of line: <code>L</code>	46
<code>t</code> and <code>T</code> : stop just before a character	46
Counting: <code>v2tX</code> is not <code>2vtX</code>	47
<code>)</code> and <code>(</code> are sentence motions, not parentheses	47
Search as a selection motion (crosses lines)	48
Hop: select to something you can see (no counting)	49
Target on another line, count unknown: use the relative numbers	49
Typing before or after: <code>i a I A</code>	50
Non-contiguous lines (for example line 3 and line 10 together)	50
Line Range Yanking (Command Mode)	51
6. Working with Parentheses, Quotes, and Brackets	51
Jumping to Matching Pair	51
Selecting Inside/Around Pairs	51
Changing Text Inside Pairs	52
Deleting Text Inside Pairs	52
Adding Surrounding Pairs (vim-sandwich Plugin)	52
Removing Surrounding Pairs (vim-sandwich Plugin)	52
Replacing Surrounding Pairs (vim-sandwich Plugin)	53
Auto-Pairing (nvim-autopairs Plugin)	53
16. Code Commenting	53
vim-commentary (Plugin)	53
Smart Commenting (Custom)	53
17. Surrounding Pairs (vim-sandwich + nvim-autopairs)	54
24. Yank History (yanky.nvim)	55
25. Undo History	55
29. Registers & Macros	55
Registers	55
Macros	55
60. Macros In Depth	56
Recording a Macro	56

Playing a Macro	56
Scenario: Add Semicolons to the End of 20 Lines	56
Scenario: Wrap Each Line in Double Quotes	56
Scenario: Append the Same Text to a Block of Lines (Tested)	57
Scenario: Convert a List of Variables to Assignments	57
Scenario: Turn CSV into SQL VALUES	57
Tips for Writing Macros	58
Visual Mode Macros	58
61. The Dot Command (<code>.</code>) – Repeating Actions	58
What Counts as a "Change"	58
Scenario: Change a Variable Name One-by-One	58
Scenario: Add a Prefix to Multiple Lines	58
Scenario: Delete the First Word on Several Lines	59
Scenario: Indent Multiple Blocks	59
Combining <code>.</code> with Counts	59
62. Visual Block Editing (Multi-Cursor-Like)	59
Scenario: Add a Prefix to Multiple Lines at Once	59
Scenario: Append Text to Multiple Lines	59
Scenario: Delete a Column	60
Scenario: Replace a Column	60
64. Everyday Editing Scenarios	60
Swap Two Lines	60
Swap Two Words	60
Duplicate a Line	60
Duplicate a Block of Code	60
Fix Indentation of Entire File	61
Remove All Blank Lines	61
Sort Lines	61
Convert Tabs to Spaces (or Vice Versa)	61
Wrap a Selection in a Tag/Function	61
65. Swapping Function Arguments (<code>vim-swap</code>)	61
68. Useful Vim Tricks	61
Run a Normal-Mode Command on Every Line	61
Execute a Command on a Range	61
Increment/Decrement Numbers	62
Letter Sequences (a, b, c...) Instead of Numbers	63
Open the File Under Cursor	64
Change Case	64
Align Text	64
Command Abbreviations	65
71. The Global Command (<code>:g</code>)	65
Common Uses	65
Everyday Scenarios	66
Delete All Print/Debug Statements	66
Keep Only Lines Matching a Pattern	66
Extract All Function Signatures	66
Add Prefix/Suffix to Matching Lines	66

Sort Lines Matching a Pattern to Top	66
76. Common Editing Power Combos	66
Changing Text	67
Deleting Text	67
Copying Text	67
Selecting Text	68
Quick Transformations	68
The Most Powerful Patterns	68
Chapter 5: Completion and snippets	69
14. Autocompletion (<code>nvim-cmp</code>)	69
15. Snippets (<code>UltiSnips</code>)	69
Java Snippets	70
Other Snippets	70
45. Autocompletion In Depth	70
How It Works	70
The Smart Tab Behavior	71
Completion Keymaps	71
Visual Indicators	71
52. Snippets for Developers	71
What Snippets Are	71
How to Use Snippets	71
Custom Snippets	71
Creating Your Own Snippets	72
Chapter 6: Searching, replacing, files and the file tree	73
10. Searching, Replacing, and Refactoring Text	73
Searching in the Current File	73
Search Modifiers	73
Search Examples	74
Substitution (Find & Replace in Current File)	74
Understanding the Range (Where to Replace)	74
Understanding the Flags (How to Replace)	74
Flag Combinations	75
Confirmation Mode (<code>c</code> Flag) Controls	75
Changing the Delimiter	75
Special Replacement Patterns	76
Substitution Examples	76
Searching in the Current Buffer with <code>*</code> and <code>#</code>	76
Using <code>ciw</code> + <code>.</code> for Selective Single-File Replacement	76
67. Multi-File Search and Replace (Complete Guide)	77
Quick Decision Guide: Which Method to Use	77
Method 1: LSP Rename (Best for Code Symbols)	77
Method 2: <code>:grep</code> + <code>:cfd</code> (Best for Plain Text)	77
Step-by-Step: Replace All Without Confirmation	78
Step-by-Step: Replace with Confirmation for Each Occurrence	78
Step-by-Step: Review Results Before Replacing	78
Using Regex with <code>:grep</code>	78
Limiting to Specific File Types	78

Method 3: <code>:vimgrep</code> + <code>:cfd</code> (Built-in, Slower but Portable)	79
Method 4: <code><Space>fg</code> for Finding (No Replace)	79
Complete Examples	79
Example 1: Rename an API Endpoint Across the Project	79
Example 2: Replace a Deprecated Function Name (with Confirmation) ..	80
Example 3: Delete All Console.log Statements in JavaScript Files	80
Example 4: Add a Comment Before Every TODO	80
Example 5: Replace Only in Python Files in the src/ Directory	80
Example 6: Case-Insensitive Project-Wide Replace	80
Understanding <code>:cdo</code> VS <code>:cfd</code> VS <code>:bufdo</code>	80
Undoing a Multi-File Replace	80
12. Fuzzy Finding & Project-Wide Search (<code>fzf-lua</code>)	81
Moving Inside Any Picker (Lists With a Search Bar)	81
Keymaps	81
Inside the FZF Popup	82
<code><Space>fg</code> - Live Grep (Project-Wide Text Search) In Depth	82
What It Does	82
Plain Text Search	82
Regex Search	82
Use Cases for Live Grep	83
<code><Space>ff</code> - Find Files (File Name Search)	83
The Difference Between Search Methods	83
26. Quickfix & Location List	84
Commands	84
Inside the Quickfix Window (nvim-bqf, quicker.nvim)	84
Trouble (Plugin)	84
51. Quickfix Workflows for Developers	84
What Populates the Quickfix List	85
Navigating the Quickfix List	85
Batch Operations on Quickfix Items	85
Trouble (Better Quickfix UI)	85
11. File Explorer (<code>nvim-tree</code>)	85
30. Working with Directories	87
Path Modifiers (for use in commands)	87
57. File Management for Developers	88
File Operations	88
Project Structure Navigation	88
63. Working with Multiple Files	88
Opening Several Files	88
Comparing Two Files Side by Side	88
Copying Between Files	89
Running the Same Edit Across Multiple Files	89
Closing Files You're Done With	89
Chapter 7: Windows, buffers, terminal, sessions and the interface	90
7. Windows, Splits, and Buffers	90
Key Concepts	90
Creating Splits	90

Navigating Between Windows	90
Resizing Windows	91
Moving Windows Around	91
Buffer Tabs (the Top Line)	91
Closing Windows	92
Buffer Management	92
Tabs	93
Closing Floating Windows	93
8. Terminal Integration	93
Opening a Terminal	93
Navigating In and Out of Terminal	93
Closing a Terminal	94
58. Session and Productivity	94
Auto-Save	94
Session Management	94
Collaborative Editing	94
32. Statusline (<code>luaLine.nvim</code>)	95
33. UI Features	96
Dashboard (Start Screen)	96
Help keys: the user guide and Claude (any buffer, and the dashboard) ..	97
Chapter 8: Code: LSP, running, debugging, folding, treesitter	98
13. LSP: Language Server Protocol	98
Configured Language Servers	98
LSP Keymaps	99
Built-in Neovim LSP and Diagnostic Keys	99
LSP Commands	100
Glance: Peek Without Jumping	100
Diagnostics (Errors, Warnings)	100
44. Language Server Protocol (LSP) In Depth	100
What LSP Is	100
How LSP Is Managed	101
Configured Servers and What They Provide	102
LSP Keymaps (All Languages)	104
Peeking Without Jumping (Glance)	104
Diagnostics In Depth	104
The Lightbulb	105
18. Code Folding (<code>nvim-ufo</code>)	105
47. Code Folding In Depth	106
What It Is	106
How It Works	106
Folding Keymaps	106
21. Treesitter & Text Objects	107
Treesitter (Plugin)	107
targets.vim (Plugin)	107
vim-matchup (Plugin)	107
46. Treesitter In Depth	107
What Treesitter Is	107

Installed Parsers	107
19. Code Running	108
Filetype-Specific	108
55. Code Running In Depth	108
The Universal Runner	108
Language-Specific Details	109
Filetype-Specific Runners	109
36. Debugging	110
56. Debugging In Depth	110
Debug Adapter Protocol (DAP)	110
GDB Integration	110
43. How the Development Toolchain Fits Together	110
53. Documentation Lookup	111
DevDocs (Plugin)	111
Hover Documentation (LSP)	111
59. Useful Developer Commands	112
Part III: Everyday Scenarios & Recipes	112
75. Real-World Developer Workflows	112
Workflow: Investigating a Bug	112
Workflow: Code Review (Reviewing Your Own Changes)	113
Workflow: Refactoring a Function Name Across the Project	113
Workflow: Adding a Feature in a New Branch	113
Workflow: Quickly Editing a Config File	113
Workflow: Working with JSON	113
Workflow: Writing Documentation (Markdown)	114
Workflow: Pair Programming with Split Views	114
35. Java Development (<code>nvim-java</code>)	114
Build & Run	114
Testing	114
Refactoring	115
54. Java Development In Depth	115
How It Works	115
Build & Run	115
Testing	116
Debugging	116
Refactoring	116
Chapter 9: Git	117
20. Git Integration	117
vim-fugitive (Plugin)	117
gitsigns.nvim (Plugin)	117
gitlinker.nvim (Plugin)	117
Other Git Tools	118
Resolving Merge Conflicts (diffview.nvim)	118
48. Git Workflow In Depth	118
The Git Plugin Ecosystem	118
Daily Git Workflow	119
Understanding Gitsigns	119

Chapter 10: Claude, Markdown, LaTeX/Typst, spelling, URLs	121
9. AI Assistant Window (Claude Code)	121
Claude Code	121
49. AI-Assisted Development In Depth	121
Claude Code	121
27. Markdown Support	122
Preview	122
Footnotes	122
Text Objects & Operators (Markdown Only)	123
Other Markdown Plugins	123
28. LaTeX and Typst Support	123
LaTeX (<code>vimtex</code>)	123
Typst	123
31. Spell Checking	123
38. URL & Unicode	124
Chapter 11: Various: custom commands, other plugins, configuration, Neovide	126
34. Custom Commands	126
Plugin Manager Shortcuts	126
39. Other Plugins	127
SQL Databases (nvim-dbee, vim-dadbod-ui)	128
40. Configuration Management	129
41. Filetype-Specific Settings	130
42. Automatic Behaviors	130
Part II: Developer Guide	132
66. Shell Commands from Inside Neovim	132
Running a Shell Command	132
Inserting Command Output into the Buffer	132
Filtering a Selection Through a Command	132
The AsyncRun Plugin	132
77. Neovide (Graphical Neovim)	132
Starting it	133
What this configuration sets for Neovide	133
Keys that only exist in a GUI	133
Chapter 12: Java	134
78. Java (nvim-java, jdtls, tests, debugging)	134
1. What You Get And Why	135
How nvim-java is loaded	135
The Java devShell (<code>use_dev_env java</code>)	136
Outside the devShell	136
2. Quick Start	137
3. Project Layout Rules	137
The root folder	137
First start and where files appear	137
4. Keys And Commands	138
Build (<code><Space>jb</code>)	138
Runner (<code><Space>jr</code>)	138

Test (<code><Space>jt</code>)	138
Refactor / extract (<code><Space>je</code>)	139
Settings and debugger setup	139
The <code>:Java*</code> commands	139
The runner window	139
Profiles window (<code><Space>jrp</code>)	140
Clean workspace (<code><Space>jbc</code>)	140
Change runtime (<code><Space>jj</code>)	140
5. Language Server Features In A Java Buffer	140
6. Tests (JUnit)	141
Why and how	141
Using it	141
Needs	142
7. Debugging	142
Why and how	142
8. Refactoring (Extract)	144
9. Snippets	145
10. Windows While Running, Testing And Debugging	146
11. Troubleshooting	147
Related Sections	149
Chapter 13: Python	150
79. Python (pyright, ruff, black, uv, running and debugging)	150
What You Get	151
Quick Start	153
Why Each Tool Exists	154
Devshell and Tools	155
Running Code	156
When <code><Space>rf</code> / <code><F9></code> uses uv	157
Stopping a running program	158
Formatting and Linting	158
Code actions from ruff	159
Linting from the command line (<code>:compiler ruff</code>)	159
LSP Keys	160
Debugging with pdb	161
Virtual Environments	163
Testing	164
Navigation and Text Objects	164
Snippets	165
Troubleshooting	166
Related Sections	168
Chapter 14: LaTeX	169
80. LaTeX (vimtex, texlab, ltex, PDF viewer)	169
The big picture	169
What it needs (the devShell)	169
Quick start	170
Compiling	170
Errors	171

Several files (a project with chapters)	172
The viewer	172
Table of contents	173
Moving and editing (vimtex)	173
Motions	174
Text objects (use after <code>d</code> , <code>y</code> , <code>c</code> , <code>v</code>)	174
Change, delete, toggle	175
Language server (texlab)	176
Citations (bibliography)	176
Grammar and spelling (ltex_plus)	176
Snippets	177
Troubleshooting	178
Related sections	178
Chapter 15: Markdown	179
81. Markdown (writing, preview, footnotes, PDF)	179
The tools and why they exist	180
Quick start	181
Requirements	181
Try the keys: an example file	181
Keys	182
Text objects and operators	184
Footnotes	184
Reference links and tables	185
Preview (markdown-preview.nvim)	185
Rendering inside the buffer (render-markdown.nvim)	186
Formatting with prettier	187
PDF export (<code>:ToPDF</code>)	187
Writing quality	188
ltex_plus (grammar, LanguageTool)	188
Vim spell checking	189
typos_lsp	189
Troubleshooting	190
Related sections	190
Chapter 16: Typst	191
82. Typst (typst.vim, tinymist, watch and preview)	191
What you get	191
Requirements: the Typst devShell	191
Outside the devShell	192
Quick start	192
Keys and commands	193
The language server: tinymist	193
Watch, compile and the PDF	194
Prose checking in Typst	196
Filetype settings	196
Example document	196
What to try	197
Troubleshooting	198

Related sections	198
------------------------	-----

Chapter 1: Day-to-Day Cheat Sheet (section 2)

The things you do all day, in one place. Details are in the sections named at the end of each table.

Navigation

Keys	What it does
<code>h j k l , w b , H L , gg G</code>	Move by character, word, line start / end, file start / end
<code>f</code> + two characters	Hop to any matching spot on screen (then press the label letter)
<code>%</code>	Jump to the matching bracket
<code><Ctrl-o></code> / <code><Ctrl-i></code>	Back / forward through your jump history (after <code>gd</code> , searches, <code>gg</code> ...)
<code>ma</code> , then <code>`a</code>	Set mark <code>a</code> , jump back to it later
<code>gd</code>	Go to definition (use <code><Ctrl-o></code> to come back)
<code><Space>ff</code> / <code><Space>fg</code> / <code><Space>fr</code>	Find a file / search text in the project / recent files
<code><Space>s</code>	File tree (follows <code>:cd</code> , <code>:Z</code> , <code>:z</code>)
<code>gb</code> / <code>gB</code> , <code><Space>bp</code>	Next / previous buffer, pick a buffer by letter
<code><Space>t</code>	Symbol outline of the file
<code>:Z <word></code>	Change the working folder with zoxide

More: sections 3, 7, 11, 12, 22.

Copy, delete and move

Keys	What it does
<code>yy</code> then <code>p</code>	Copy a line, paste it below
<code>dd</code> then <code>p</code>	Cut a line, paste it below
<code>diw</code> / <code>ciw</code> / <code>yiw</code>	Delete / change / copy the word under the cursor (<code>c</code> does not overwrite your paste register)
<code>5dd</code> / <code>5yy</code>	Delete / copy 5 lines
<code><Alt-j></code> / <code><Alt-k></code>	Move the line (or selection) down / up
<code><Space>y</code>	Copy the whole buffer
<code><Space>p</code> / <code><Space>P</code>	Paste on a new line below / above
<code>p</code> in Visual mode	Replace the selection with what you copied
<code>[y</code> / <code>]y</code>	After a paste, step back / forward through the yank history
<code>:Rename <name></code> / <code>:Move <path></code> / <code>:Duplicate <name></code> / <code>:Delete</code>	Rename, move, copy or delete the current FILE
tree: <code>a d r c x p</code>	Create, delete, rename, copy, cut, paste files in the file tree

More: sections 4, 11, 24, 57.

Indented blocks

Keys	What it does
<code>dii</code> / <code>yii</code> / <code>cii</code> / <code>vii</code>	Delete / copy / change / select the code block at the cursor's indent level
<code>dai</code> / <code>yai</code> / <code>cai</code> / <code>vai</code>	Same, including the lines that open and close the block
<code>[i</code> / <code>]i</code>	Jump to the top / bottom of the block

More: section 5 ("`ii` / `ai`").

Text objects (what to delete, change or copy)

Keys	What it does
<code>diw</code> / <code>daw</code>	The word / the word with its space
<code>das</code> / <code>dis</code> / <code>cis</code>	The sentence with / without the space after it; change the sentence
<code>dip</code> / <code>dap</code>	The paragraph (without / with the blank line after it)
<code>di(</code> / <code>da(</code>	The text inside the parentheses / with the parentheses
<code>diS(</code> / <code>daS(</code>	Sandwich: empty the surrounding <code>()</code> / remove them with their content
<code>ii</code> / <code>ai</code>	The indented block (see above)

The same objects work after `c`, `y` and `v`. More: sections 5, 6, 17 and 70.

Macros

Keys	What it does
<code>Qa</code>	Start recording into register <code>a</code> (<code>qa</code> works too)
<code>q</code>	Stop recording
<code>@a</code> / <code>5@a</code> / <code>@@</code>	Play it once / 5 times / repeat the last one (in Markdown buffers <code>@@</code> returns from a footnote instead: use <code>@a</code> again)
<code>:%normal@a</code>	Play it on every line of the file

More: sections 29 and 60.

Bulk rename and replace

Goal	What to do
Rename a code symbol everywhere in the project	<code><Space>rn</code> , type the new name, <code><Enter></code> (LSP)
Replace text in the current file	<code>:%s/old/new/g</code> (add <code>c</code> to confirm each one)
Replace text in every file of the project	<code>:grep "old" then :cfd %s/old/new/g update</code>
Replace one by one, deciding each time	<code>*</code> , <code>ciw</code> + new word + <code><Esc></code> , then <code>n</code> to skip or <code>.</code> to replace
Rename or move one file	<code>:Rename <name></code> / <code>:Move <path></code> , or <code>r</code> in the file tree
Undo a multi-file replace	See section 67

Files are renamed one at a time with the commands above. More: sections 10 and 67.

Undo, redo and repeat

Keys	What it does
<code>u</code> / <code><Ctrl-r></code>	Undo / redo
<code><Space>u</code>	Show the undo tree in a left panel
<code>.</code>	Repeat the last change (e.g. <code>ciw</code> + word, then <code>n</code> and <code>.</code> on the next match)
<code>[y</code> / <code>]y</code>	After a paste, replace the pasted text with an earlier / later yank (yank two lines, <code>p</code> , then <code>[y</code>)

More: sections 24, 25, 61.

Visual block (many lines at once)

Keys	What it does
<code><Ctrl-v></code> then <code>j</code> / <code>k</code>	Select a rectangle of text
<code>I</code> then text then <code><Esc></code>	Insert the text at the start of every selected line
<code>A</code> then text then <code><Esc></code>	Append the text at the end of every selected line
<code>d</code> / <code>c</code>	Delete / change the selected block
<code>g<Ctrl-a></code>	Count up 1, 2, 3 ... down the selected numbers
<code><Alt-j></code> / <code><Alt-k></code>	Move the selected lines down / up

More: section 62.

Comments and surrounding pairs

Keys	What it does
<code>gcc</code> / <code>gc</code> + motion / <code>gc</code> in Visual	Toggle a comment on a line / a motion / the selection (<code>gcip</code> = paragraph)
<code>gcSS</code> / <code>gcs</code> + motion, <code>gcr</code> + motion	Comment / uncomment exactly the rows given (<code>gcsip</code> , <code>gcr200j</code>)
<code>gcu</code>	Uncomment the adjacent commented lines
<code>:1,3Commentary</code>	Toggle comments on a line range
<code>dgc</code> / <code>ygc</code>	Delete / copy the comment block under the cursor
<code>sa!w"</code>	Surround the word with <code>"</code>
<code>sd"</code>	Delete the surrounding <code>"</code>
<code>sr"'</code>	Change the surrounding <code>"</code> into <code>'</code>

More: sections 16 and 17.

Search in a file

Keys	What it does
<code>/text</code> then <code><Enter></code> , <code>n</code> / <code>N</code>	Search forward, next / previous match
<code>*</code> / <code>#</code>	Search the word under the cursor forward / backward
<code>;noh<Enter></code>	Clear the search highlight
<code>:%s/old/new/gc</code>	Replace with a question for each match

More: sections 10 and 23.

Buffers, splits, tabs

Keys	What it does
<code>gb / gB, <Space>bp</code>	Next / previous buffer, pick one by letter
<code><Ctrl-^></code> (in Neovide also <code><Ctrl-6></code>)	Jump to the previous buffer and back again, like Alt-Tab for files
<code>\d / \D</code>	Close this buffer / close all other buffers
<code><Space>- / <Space> </code>	Split below / to the right (same file)
<code><Ctrl-w>h j k l</code>	Move between windows
<code><Left> <Down> <Up> <Right></code>	The same, with the arrow keys (normal mode)
From a terminal window (run output, <code>:term</code>)	<code><Esc></code> , then <code><Ctrl-w>h</code> (or an arrow key) to go to the code; <code>i</code> to type in the terminal again
From the Claude panel	<code><Ctrl-h></code> goes back to the code on the left (<code><Esc></code> is sent to Claude); <code><Ctrl-w>l</code> or <code><Right></code> from the code goes back in
<code><Ctrl-w>= / <Ctrl-w>o</code>	Make windows equal / keep only this window
<code>gt / gT, \t / \T</code>	Next / previous tab, close this tab / the other tabs
<code>:sp <file> / :vs <file></code>	Open a file in a new horizontal / vertical split (; works like <code>:</code> , so <code>;vs <file></code> too)

More: section 7.

Save and quit

Keys	What it does
<code><Space>w</code>	Save (only writes if changed)
<code><Space>q</code>	Save and close this window
<code><Space>Q</code>	Quit nvim, asks first (default No), discards unsaved work
<code>:wa / :q!</code>	Save all buffers / close this window and discard changes
(automatic)	Auto-save: a changed file saves itself when you switch buffer or leave the nvim window; a message "AutoSave: saved at ..." appears (not for unnamed or read-only buffers, terminals, LaTeX and Typst files)

More: section 72.

Sessions, dashboard and zoxide

Keys	What it does
<code>\h</code> / <code>\H</code>	Open the dashboard / return to the previous buffer
<code>r</code> / <code>L</code> on the dashboard	Restore the session of this folder / of the last folder
<code><Space>?</code>	Open the user guide as a PDF in zathura next to Neovim (any buffer and the dashboard; clickable table of contents; <code>q</code> closes it)
<code><Space>a</code>	Ask Claude "how do I do X in Neovim": split on the right (own tab on the dashboard), skill loaded; <code>/exit</code> closes it
<code>:Z <word></code> / <code>:z <word></code>	Jump to the best zoxide match (the file tree follows)
<code>:Obsession</code> , <code>nvim -S Session.vim</code>	Keep a <code>Session.vim</code> up to date, restore it later
<code><Space>sv</code>	Restart nvim (writes all files first)

More: sections 30 and 58.

Code intelligence (LSP)

Keys	What it does
<code>gd</code> / <code>K</code>	Go to definition / hover documentation
<code><Space>gd</code> / <code><Space>gr</code> / <code><Space>gi</code>	Peek definitions / references / implementations
<code><Space>rn</code> / <code><Space>ca</code>	Rename symbol / code actions
<code><Space>fm</code>	Format the file
<code>]d</code> / <code>[d</code>	Next / previous diagnostic
<code><Space>dd</code>	Show the diagnostic under the cursor

More: sections 13 and 44.

Git day to day

Keys	What it does
<code><Space>gs</code>	Git status
<code><Space>gw</code> / <code><Space>gc</code>	Add the current file / commit
<code><Space>gpl</code> / <code><Space>gpu</code>	Pull / push
<code><Space>gB</code>	Branch menu: pick a branch to switch to (same as clicking the branch in the statusline)
<code>]c</code> / <code>[c</code>	Next / previous changed hunk
<code><Space>hp</code> / <code><Space>hb</code>	Preview the hunk / blame the line
<code><Space>gl</code>	Copy a permalink for the line
<code>:Neogit</code> / <code>:NeogitLogCurrent</code>	Open the Neogit status window / the log of the current file (<code>q</code> closes)

More: sections 20 and 48.

Mouse

Action	What it does
Click a tab in the top line	Switch to that buffer
Click the <code>x</code> / <code>•</code> of a tab	Close the buffer (a changed file asks Save / Discard / Cancel)
Click the branch name in the statusline	Branch menu (also <code><Space>gB</code>)
Click the language-server name in the statusline	Popup with the attached language servers (also <code>:LspAttached</code>)
Right-click in the text	Neovim's menu: <code>Paste</code> pastes the clipboard, <code>Select All</code> selects the whole file, <code>Inspect</code> shows which highlight and syntax group is under the click, <code>How-to disable mouse</code> opens the help for it

More: sections 7 and 32.

Language guides (tools that exist only for one language)

Language	What the guide covers	Section
Java	nvim-java, jdtls, running, JUnit tests, debugging, refactoring, profiles	78
Python	pyright, ruff, black, uv, <code><Space>rf</code> / <code><F9></code> and <code><Space>rr</code> , pdb debugging	79
LaTeX	vimtex, texlab, ltx, compiling, the PDF viewer	80
Markdown	marksman, rendering, preview, footnotes, <code>:ToPDF</code>	81
Typst	tinymist, <code><Space>tw</code> watch, the PDF viewer	82

Neovide (Neovim in its own window)

Command	What it does
<code>neovide</code>	Open Neovim in a normal desktop window instead of the terminal, with smooth cursor animation. Same config, same keys.

Full explanation: [section 77](#).

Lists with a search bar (pickers)

Keys	What it does
<code><Ctrl-n></code> / <code><Ctrl-p></code> or <code><Down></code> / <code><Up></code>	Next / previous item (branch menu, code actions, <code><Space>ff</code> , <code><Space>fg</code> , Telescope ...)
<code><Ctrl-j></code> / <code><Ctrl-k></code>	Same, except in Telescope
<code><Tab></code>	Move down (marks the item too in fzf-lua and Telescope)
<code><Enter></code> / <code><Esc></code>	Choose / close (in snacks and Telescope the first <code><Esc></code> only leaves the search bar)

`j` and `k` type letters into the search bar. More: section 12.

Quickfix list (search results, errors)

Keys	What it does
<code>:copen</code> / <code>:cclose</code>	Open / close the list
<code>:cnext</code> / <code>:cprev</code>	Next / previous item
<code>\x</code>	Close the quickfix and location lists
<code>p</code> / <code>zf</code> in the list	Preview the item / filter the list with <code>zfz</code>
<code><Tab></code> in the list	Mark the item
<code><Ctrl-x></code> / <code><Ctrl-v></code> in the list	Open the item in a horizontal / vertical split
<code>:cfd %s/old/new/g update</code>	Replace in every file of the list

More: sections 26 and 51.

Spell checking

Keys	What it does
<code><Space>cz</code>	Spell checking on / off (<code>:set spell</code> / <code>:set nospell</code> too)
<code>]s</code> / <code>[s</code>	Next / previous misspelled word
<code>z=</code>	Show suggestions
<code>zg</code> / <code>zw</code>	Add the word to your allowed list / mark it as wrong (<code>2zg</code> Italian, <code>3zg</code> German, <code>4zg</code> French)
<code>zug</code>	Undo the last <code>zg</code>
<code>:e ~/.config/nvim/spell/en.utf-8.add</code>	Open the list of allowed English words (one per line: delete a line to forbid the word again)

`:set nospell` only turns off the built-in checker. Typo underlines that come from the language servers (`typos_lsp` in code, `ltex_plus` in prose) stay; they are diagnostics. More: section 31.

Run code and terminal

Keys	What it does
<code><Space>rr</code>	Run the current file in a terminal on the left
<code><Space>rf</code> / <code><F9></code>	Run or compile the file the editor's own way (Lua, Vim script, Python, C++, LaTeX; one warning in other file types)
<code>:term</code>	Open a terminal in this window
<code><Esc></code> / <code>i</code>	Leave terminal mode / go back into it (<code><Esc></code> works differently in the Claude panel, see sections 8 and 9)

More: sections 8 and 19. `<Space>rf` in any other file type shows one warning (no editor-run for this file type (use `<Space>rr` for a terminal run)).

Python debugger keys

Start with `<Space>dp` in a Python buffer (pdb through nvim-gdb); the keys work only during that session (otherwise one warning `pdb: no debug session here (start one with <Space>dp)`). nvim-gdb's own F-keys still work too.

Keys	What it does
<code><Space>dp</code>	Start the pdb debugger on this file
<code><Space>dB</code> / <code><F8></code>	Toggle a breakpoint on this line
<code><Space>dc</code> / <code><F5></code>	Continue to the next breakpoint
<code><Space>dn</code> / <code><F10></code>	Next line (step over)
<code><Space>ds</code> / <code><F11></code>	Step into the call
<code><Space>df</code> / <code><F12></code>	Finish (run until the function returns)
<code><Space>du</code> / <code><F4></code>	Run until this line
<code><Space>dv</code>	Evaluate the word under the cursor (Visual: the selection)
<code>:GdbDebugStop</code>	Quit the debugger

More: section 79.

Keyboards without function keys, Insert or Page keys

Nearly every key in this guide has a version without function keys. `<F9>` is `<Space>rf` (run the file the editor's own way); the Python debugger has the `<Space>d` keys above. `<S-Insert>` (paste in the GUI, ginit.vim) is replaced by `<Ctrl-r>` then `+` in Insert mode or on the command line (built-in Vim, works everywhere); `<Ctrl-d>` / `<Ctrl-u>` scroll instead of the Page keys. Unicode `<F4>` is replaced by `<Ctrl-k>` + two letters (section 38). The only keys left are three vimtex defaults in section 80 (`<F6>`, `<F7>`, `<F8>`).

Folding

Keys	What it does
<code>za</code>	Toggle the fold under the cursor
<code>zR</code> / <code>zM</code>	Open / close all folds
<code>zr</code> / <code>zm</code>	Open / close one more fold level
<code><Space>K</code>	Preview the folded lines

More: section 18.

Text tricks

Keys	What it does
<code>gUw</code> / <code>guiw</code>	Uppercase / lowercase the word
<code><Ctrl-a></code> / <code><Ctrl-x></code>	Add 1 to / subtract 1 from the number under the cursor (<code>10<Ctrl-a></code> adds 10)
<code>:%!sort</code>	Sort the whole buffer
<code>:Tabularize /=</code>	Align the <code>=</code> signs
<code>:g/pattern/d</code>	Delete every line that matches
<code>:g/pattern/normal @a</code>	Run macro <code>a</code> on every matching line
<code>!:cmd</code> / <code>:read !cmd</code>	Run a shell command / insert its output

More: sections 66, 68 and 71.

Claude and fuzzy finders

Keys	What it does
<code><Space>cc</code>	Toggle the Claude Code window
<code><Space>a</code>	Ask Claude how to do something in Neovim (vertical split, skill loaded)
<code><Space>?</code>	Open the user guide PDF
<code><Space>ff</code> / <code><Space>fg</code>	Find a file / search text in the project
<code><Space>fb</code> / <code><Space>fr</code> / <code><Space>fh</code>	Open buffers / recent files / help tags

More: sections 9 and 12.

Other most-used keys (from the former Quick Reference)

Keymap	Action
<code>]<Space></code> / <code>[<Space></code>	Insert a blank line below / above (cursor stays; <code>3]<Space></code> inserts 3)
<code>gS</code>	Split / join the list, arguments or block under the cursor (treesj)

Chapter 2: Basics: modes, saving, recovering, getting help

1. Understanding Modes

Neovim is a modal editor. You are always in one of these modes:

Mode	How to enter	What it does
Normal	<code><Esc></code> from any mode	Navigate, delete, copy, paste, run commands. This is your "home base".
Insert	<code>i</code> , <code>a</code> , <code>o</code> , <code>O</code> , <code>c</code> from Normal (<code>s</code> is disabled here: it is the vim-sandwich prefix)	Type text into the file.
Visual	<code>v</code> , <code>V</code> , <code><Ctrl-v></code> from Normal	Select text (character, line, or block).
Command	<code>;</code> or <code>:</code> from Normal	Type commands at the bottom of the screen (e.g., <code>:w</code> to save).
Terminal	When inside a terminal buffer	Interact with a shell. Press <code><Esc></code> to go to Normal mode (in the Claude Code panel <code><Esc></code> goes to Claude: use <code><Ctrl-\><Ctrl-n></code> there).

Entering Insert Mode

Keymap	Description
<code>i</code>	Insert before cursor
<code>I</code>	Insert at beginning of line
<code>a</code>	Insert after cursor
<code>A</code>	Insert at end of line
<code>o</code>	Open new line below and insert
<code>O</code>	Open new line above and insert

Leaving Insert Mode

Keymap	Description
<code><Esc></code>	Return to Normal mode
<code>jk</code> (typed quickly)	Return to Normal mode (via better-escape.vim plugin, 200ms window)

Visual Mode Variants

Keymap	Description
<code>v</code>	Character-wise visual (select individual characters)
<code>V</code>	Line-wise visual (select entire lines)
<code><Ctrl-v></code>	Block visual (select a rectangular block of text)

69. Tips for Vim Beginners

The Most Important Habits

1. **Stay in Normal mode.** Only enter Insert mode to type, then immediately `<Esc>` back. Normal mode is where all the power lives.
2. **Think in verbs + nouns.** `d` (delete) + `iw` (inner word) = delete word. `c` (change) + `i"` (inside quotes) = change quoted text. `y` (yank) + `ap` (around paragraph) = copy paragraph.
3. **Use `.` aggressively.** Make one change, then `.` to repeat it everywhere.
4. **Use `*` and `n`.** Search for a word with `*`, jump through occurrences with `n`.
5. **Use text objects.** `ciw`, `di(`, `va"` are faster than selecting character-by-character.
6. **Press `<Space>` and wait.** The which-key popup shows you all available keybindings.

Common Mistakes and How to Fix Them

Problem	Cause	Fix
Typing random commands instead of text	You're in Normal mode	Press <code>i</code> to enter Insert mode first
Text won't stop appearing	You're in Insert mode	Press <code><Esc></code> to go back to Normal
Screen looks weird / frozen	You pressed <code><Ctrl-s></code> (terminal freeze)	Press <code><Ctrl-q></code> to unfreeze. (In kitty + tmux with this config <code><Ctrl-s></code> did not freeze the screen: tested. In Insert/Select mode <code><Ctrl-s></code> is LSP signature help.)
Can't exit Neovim		Type <code><Space>q</code> and answer <code>y</code> to the confirmation, or <code>;qa!<Enter></code> (no confirmation)
Pasted text looks wrong	Paste from outside with <code><Ctrl-v></code> in terminal mode	Use <code>"p</code> in Normal mode, or the terminal paste key
Search highlight won't go away	Yellow boxes left over from a search or <code>*</code>	Type <code>;noh<Enter></code> (tested). <code><Esc></code> does not clear it in this config
Accidentally opened a macro	Pressed <code>q</code>	Press <code>q</code> to stop recording
A key like <code>"</code> does nothing until you press another key, or an accented letter appears (<code>ë</code> , <code>è</code>)	Your keyboard layout uses dead keys (see below)	Press <code><Space></code> right after the key

Keyboard Layouts With Dead Keys (for example US International)

Some layouts (for example US International, used on the main machine) treat certain keys as **dead keys**: the key does not type anything by itself, it waits for the next key to decide. `"` followed by `e` gives `ë`; `"` followed by `<Space>` gives a plain `"`.

Neovim only receives the character after that decision, so commands that need `"` look like they do nothing.

- **Symptom (tested with `"`):** `vt"l` selected nothing, because `t` kept waiting for its character. `vt"<Space>l` works. In a search, `v/"<Enter>` worked because `<Enter>` also ends the wait.
- **The rule:** after a dead key, press `<Space>` before continuing (`vt"<Space>l`).
- **Keys that may be affected** (not confirmed, depends on the layout; `"` is the only one tested): `'` and ``` (marks such as ``a` and `'a`), `"` (registers such as `"ay`, text objects such as `ci"`), `~` (toggle case) and `^` (start of line). Without the space, a dead key followed by a letter can become an accented letter ( `"a -> ä`, ``a -> à`) and the Vim command never runs.
- **This is not Neovim:** it happens in any application with that layout. A different PC may have a different layout, so if a key does nothing or types a strange character, check the layout before suspecting the config.
- **Quick test in Insert mode:** type the key, then a letter (`"e`); if you get an accented letter, it is a dead key on your layout.

Learning Path

1. First week: `h j k l , i , <Esc> , :w , :q , dd , yy , p , u`
 2. Second week: `w , b , e , 0 , $ , gg , G , /search , n , N`
 3. Third week: `ciw , di( , vi" , V , > , < , .`
 4. Fourth week: `<Space>ff , <Space>fg , gd , K , <Space>ca , gcc`
 5. After that: Macros, quickfix, text objects, splits, registers
-

70. The Verb + Noun System (How Vim Commands Work)

This is the single most important mental model for understanding Vim. Almost every command follows this pattern:

`[count] operator motion` OR `[count] operator text-object`

- **Operator** (verb): What you want to do (`d` delete, `c` change, `y` yank, `>` indent, `gU` uppercase, etc.)
- **Motion** (noun): Where to do it (`w` word, `$` end of line, `gg` top of file, `}` next paragraph, etc.)
- **Text object** (noun): A structural unit to act on (`iw` inner word, `i(` inside parentheses, `at` around an HTML tag, etc.)
- **Count:** How many times (optional)

Operators (Verbs)

Operator	What it does
<code>d</code>	Delete (and cut to register)
<code>c</code>	Change (delete and enter insert mode)
<code>y</code>	Yank (copy)
<code>></code>	Indent right
<code><</code>	Indent left
<code>=</code>	Auto-indent (fix indentation)
<code>gU</code>	Convert to UPPERCASE
<code>gu</code>	Convert to lowercase
<code>~</code>	Toggle case (in this config <code>~</code> is an operator because <code>tildeop</code> is set)
<code>gc</code>	Toggle comment (vim-commentary)
<code>gcs</code> / <code>gcr</code>	Comment / uncomment whole rows (smart commenting)
<code>gq</code>	Format/wrap text

Motions (Nouns)

Motion	What it means
<code>w</code>	To start of next word
<code>b</code>	To start of previous word
<code>e</code>	To end of current/next word
<code>\$</code>	To end of line
<code>0</code>	To start of line
<code>^</code>	To first non-whitespace
<code>gg</code>	To top of file
<code>G</code>	To bottom of file
<code>}</code>	To next blank line
<code>{</code>	To previous blank line
<code>%</code>	To matching bracket
<code>j</code>	Down one line
<code>k</code>	Up one line

Text Objects (Structured Nouns)

Text Object	What it selects
<code>iw</code> / <code>aw</code>	Inner word / a word (with whitespace)
<code>iW</code> / <code>aW</code>	Inner WORD / a WORD
<code>is</code> / <code>as</code>	Inner sentence / a sentence (with trailing whitespace)
<code>iS</code> / <code>aS</code>	vim-sandwich "query" object: type the surrounding character after it, e.g. <code>diS(</code> , <code>caS"</code> , <code>viS[</code> (inside / around that surrounding pair)
<code>ip</code> / <code>ap</code>	Inner paragraph / a paragraph
<code>i(</code> / <code>a(</code>	Inside / around parentheses
<code>i{</code> / <code>a{</code>	Inside / around braces
<code>i[</code> / <code>a[</code>	Inside / around brackets
<code>i"</code> / <code>a"</code>	Inside / around double quotes
<code>i'</code> / <code>a'</code>	Inside / around single quotes
<code>it</code> / <code>at</code>	Inside / around HTML tags

Combining Verbs and Nouns

Every operator works with every motion and every text object. This creates hundreds of commands from a small set of building blocks:

Command	Verb	Noun	What it does
<code>dw</code>	delete	word forward	Delete from cursor to next word
<code>diw</code>	delete	inner word	Delete the word under cursor
<code>daw</code>	delete	a word	Delete word + surrounding whitespace
<code>di(</code>	delete	inside parentheses	Delete everything inside <code>(...)</code>
<code>da(</code>	delete	around parentheses	Delete everything including the <code>()</code>
<code>di"</code>	delete	inside quotes	Delete everything inside <code>"..."</code>
<code>d\$</code>	delete	to end of line	Delete from cursor to line end
<code>dG</code>	delete	to end of file	Delete from here to bottom of file
<code>d}</code>	delete	to next paragraph	Delete to next blank line
<code>ciw</code>	change	inner word	Delete word, enter insert mode
<code>ci(</code>	change	inside parens	Delete contents of parens, enter insert
<code>ci"</code>	change	inside quotes	Delete quoted text, enter insert
<code>cit</code>	change	inside HTML tag	Delete tag content, enter insert
<code>yiw</code>	yank	inner word	Copy the word under cursor
<code>yi{</code>	yank	inside braces	Copy contents of <code>{...}</code>
<code>yap</code>	yank	a paragraph	Copy the paragraph
<code>>ip</code>	indent	inner paragraph	Indent the current paragraph
<code>=i{</code>	auto-indent	inside braces	Fix indentation inside <code>{...}</code>
<code>gUiw</code>	uppercase	inner word	Uppercase the entire word
<code>guap</code>	lowercase	a paragraph	Lowercase the entire paragraph
<code>gcip</code>	comment	inner paragraph	Comment out the paragraph
<code>gc3j</code>	comment	3 lines down	Comment out 3 lines

Using Counts

Counts multiply the action:

Command	What it does
<code>3dw</code>	Delete 3 words
<code>5dd</code>	Delete 5 lines
<code>2yy</code>	Yank 2 lines
<code>3>></code>	Indent 3 lines
<code>10j</code>	Move down 10 lines

Why This Matters

Once you learn a few operators and a few motions/text-objects, you can combine them freely. Learning one new operator (e.g., `gU` for uppercase) instantly gives you dozens of new commands (`gUiw`, `gUi"`, `gU$`, `gUap`, etc.) without memorizing anything extra.

72. Saving, Quitting, and File State

All the ways to save and quit, consolidated in one place.

Saving

Keymap / Command	What it does
<code><Space>w</code>	Save the current buffer (<code>:update</code> - only writes if modified)
<code>:w</code>	Save the current buffer
<code>:w filename.txt</code>	Save as a new file (original stays open)
<code>:wall</code> OR <code>:wa</code>	Save ALL open buffers
<code>:saveas filename.txt</code>	Save as new file AND switch to it

Auto-save is also active: files save on `FocusLost` (switching to another app) and `BufLeave` (switching buffers).

Quitting

Keymap / Command	What it does
<code><Space>q</code>	Save and quit the current window (<code>:x</code>)
<code><Space>Q</code>	Force quit all windows (<code>:qa!</code>) after a Yes/No confirmation
<code>:q</code>	Quit current window (fails if unsaved changes)
<code>:q!</code>	Quit current window, discard unsaved changes
<code>:qa</code>	Quit all windows (fails if any unsaved)
<code>:qa!</code>	Quit all windows, discard all unsaved changes
<code>:wq</code>	Save and quit current window
<code>:wqa</code>	Save all and quit all
<code>ZZ</code>	Save and quit (same as <code>:wq</code>)
<code>ZQ</code>	Quit without saving (same as <code>:q!</code>)

Closing Buffers (Without Quitting Neovim)

Keymap	What it does
<code>\d</code>	Close current buffer, keep the window (shows previous buffer)
<code>\D</code>	Close all other buffers; buffers with unsaved changes and running terminals (Claude panel, <code>:terminal</code>) are kept, with one warning

73. Recovering from Mistakes

Undo and Redo

Keymap	What it does
<code>u</code>	Undo the last change
<code><Ctrl-r></code>	Redo (undo the undo)
<code>U</code>	Undo all changes on the current line (rarely used)

Undo Tree (builtin)

Vim's undo history is a tree, not a linear stack. If you undo several times and then make a new edit, the old states aren't lost – they become branches. Neovim's builtin undo tree (`nvim:undotree`) shows this tree.

Keymap	What it does
<code><Space>u</code>	Toggle a 30-column undo tree panel on the far left

Inside the panel just move the cursor (`j`/`k`): the buffer switches to that undo state. Close it with `<Space>u` again or `:q`.

Time-Based Undo

Command	What it does
<code>:earlier 5m</code>	Restore the file to how it was 5 minutes ago
<code>:earlier 1h</code>	Restore to 1 hour ago
<code>:earlier 10</code>	Undo 10 changes
<code>:later 5m</code>	Go forward 5 minutes (redo)
<code>:earlier 1f</code>	Go back to the state before the last file save

This works because Neovim stores persistent undo history (the `undofile` option is enabled). Even if you close and reopen a file, you can still undo.

If You Accidentally Deleted a File

The `auto-save.nvim` plugin saves when you leave a buffer or Neovim loses focus, and Neovim creates backups in `~/.local/share/nvim/backup/`. You may be able to recover from there.

74. Discovering Keymaps and Getting Help

Which-Key: See Available Keybindings

Press `<Space>` (the leader key) and **wait about 200ms**. A popup appears showing every available `<Space>+...` keybinding organized by category.

You can also press any partial key sequence and wait:

- `g` then wait – shows all `g...` keybindings
- `z` then wait – shows all `z...` keybindings (folding, spelling, etc.)
- `<Ctrl-w>` then wait – shows all window management keybindings
- `"` then wait – shows all registers

Browse All Keymaps

Command	What it does
<code>:Telescope keymaps</code>	Searchable list of all defined keymaps
<code>m</code> on the dashboard	Search keymaps (fzf-lua)
<code>:map</code>	Show all mappings (raw output)
<code>:nmap</code>	Show normal-mode mappings
<code>:imap</code>	Show insert-mode mappings
<code>:vmap</code>	Show visual-mode mappings
<code>:verbose nmap <Space>fg</code>	Show exactly where a specific mapping was defined (file + line)

Getting Help

Command	What it does
<code>:help keyword</code>	Open Neovim's built-in help for any topic
<code>:help ciw</code>	Help on the <code>ciw</code> motion
<code>:help :substitute</code>	Help on the substitute command
<code><Space>fh</code>	Fuzzy search help tags
<code><Space>?</code>	Open this user guide as a PDF next to Neovim
<code><Space>a</code>	Ask Claude how to do something in Neovim (vertical split with the skill loaded)
<code>K</code> (on a symbol)	LSP hover documentation

Checking System Health

Command	What it does
<code>:checkhealth</code>	Diagnose installation issues (LSP servers, providers, etc.)
<code>:LspInfo</code>	LSP status: opens <code>:checkhealth vim.lsp</code> (clients and configuration)
<code>:LspAttached</code>	Popup with the LSP servers attached to this buffer
<code>:Lazy</code>	Open the plugin manager
<code>:messages</code>	Show recent notification messages

Chapter 3: Navigation

3. Core Navigation (Moving Without the Mouse)

All navigation happens in **Normal mode**. Press `<Esc>` first if you are in Insert mode.

Basic Cursor Movement

Keymap	Description
<code>h</code>	Move cursor one character left
<code>l</code>	Move cursor one character right
<code>j</code>	Move cursor one line down (follows visual/wrapped lines when no count is given)
<code>k</code>	Move cursor one line up (follows visual/wrapped lines when no count is given)
<code>5j</code>	Move 5 lines down (with a count, moves by actual lines, not wrapped lines)
<code>12k</code>	Move 12 lines up
<code>3l</code>	Move 3 characters right (same as pressing <code>l</code> three times)
<code>4h</code>	Move 4 characters left (same as pressing <code>h</code> four times)

General rule: a number typed before `h`, `j`, `k` or `l` repeats that key that many times. `Nh` moves N characters left, `Nl` N characters right, `Nj` N lines down, `Nk` N lines up.

- It works the same in Visual mode (`v3l` extends the selection by 3 characters, `v3j` selects the current line plus 3 below) and after an operator (`d3l` deletes 3 characters, `d3j` deletes the current line and the 3 below).
- With relative line numbers on, the number shown next to a line is exactly the `N` to use with `Nj` / `Nk` to reach it.

Moving Within a Line

Keymap	Description
<code>H</code>	Jump to first non-whitespace character of the line (custom, mapped to <code>^</code>)
<code>L</code>	Jump to last non-whitespace character of the line (custom, mapped to <code>g_</code>)
<code>0</code>	Jump to the very first column (column 0) of the line
<code>^</code>	Jump to first non-whitespace character (same as <code>H</code> in this config)
<code>\$</code>	Jump to the end of the line
<code>g_</code>	Jump to the last non-blank character of the line

Moving by Word

Keymap	Description
<code>w</code>	Move forward to the start of the next word
<code>b</code>	Move backward to the start of the previous word
<code>e</code>	Move forward to the end of the current/next word
<code>ge</code>	Move backward to the end of the previous word
<code>W</code>	Move forward to the next WORD (separated by whitespace only, ignores punctuation)
<code>B</code>	Move backward to the previous WORD
<code>E</code>	Move forward to the end of the current/next WORD

word vs WORD: A "word" stops at punctuation (e.g., `foo.bar` is 3 words: `foo`, `.`, `bar`). A "WORD" only stops at whitespace (e.g., `foo.bar` is 1 WORD).

Moving by Line/Screen

Keymap	Description
<code>gg</code>	Jump to the first line of the file
<code>G</code>	Jump to the last line of the file
<code>42G</code> or <code>:42</code>	Jump to line 42
<code><Ctrl-d></code>	Scroll half a screen down
<code><Ctrl-u></code>	Scroll half a screen up
<code><Ctrl-f></code>	Scroll one full screen down (forward)
<code><Ctrl-b></code>	Scroll one full screen up (backward)
<code><Ctrl-e></code>	Scroll screen down by one line (cursor stays)
<code><Ctrl-y></code>	Scroll screen up by one line (cursor stays)
<code>zz</code>	Center the current line on screen
<code>zt</code>	Move current line to the top of the screen
<code>zb</code>	Move current line to the bottom of the screen
<code>{</code>	Jump to the previous blank line (previous paragraph)
<code>}</code>	Jump to the next blank line (next paragraph)
<code>(</code>	Jump to the beginning of the previous sentence
<code>)</code>	Jump to the beginning of the next sentence

Jumping to Matching Brackets/Parentheses

Keymap	Description
<code>%</code>	Jump to the matching bracket/parenthesis/brace . If your cursor is on <code>(</code> , pressing <code>%</code> jumps to the matching <code>)</code> , and vice versa. Works with <code>()</code> , <code>[]</code> , <code>{}</code> , and also language keywords like <code>if/endif</code> (via vim-matchup plugin).

The `matchpairs` option also includes: `<>`, and several CJK bracket pairs.

Jumping to Specific Characters

Note: The built-in `f` motion has been replaced by the `hop.nvim` plugin (see Jump Navigation section). The following built-in motions still work:

Keymap	Description
<code>t<char></code>	Jump forward to just before the next occurrence of <code><char></code> on the current line
<code>T<char></code>	Jump backward to just after the previous occurrence of <code><char></code>

`;` is the command key in this config, so it does **not** repeat `t/T`. `,` still repeats the last `t/T` in the opposite direction.

Jump Navigation with `hop.nvim` (Plugin)

Keymap	Mode	Description
<code>f</code>	n, x, o	Press <code>f</code> , then type 2 characters. All matches on screen get labeled. Press the label letter to jump there instantly. Case insensitive. Press <code><Esc></code> to cancel.

Jump History

Keymap	Description
<code><Ctrl-o></code>	Jump back to the previous location in the jump list
<code><Ctrl-i></code>	Jump forward to the next location in the jump list

Every time you use a jump command (like `gg`, `G`, `/search`, `gd`, etc.), your position is saved. You can then go back and forth through your history with these keys.

Word References (`vim-illuminate`)

Other uses of the word under the cursor are highlighted when there are at least 2 (from the LSP server, else from Treesitter). It only runs in these file types: bash, c, cpp, go, java, javascript, json, lua, markdown, nix, python, rust, sh, tex (also plain TeX), toml, typescript, typst, yaml (and the React variants of javascript and typescript). In `.nix` files only the identical word is highlighted (text matching): the nix language server would mark every package of a `with pkgs; [...]` list.

Keymap	Mode	Description
<code><Alt-n></code>	n	Jump to the next reference of the word under the cursor
<code><Alt-p></code>	n	Jump to the previous reference
<code><Alt-i></code>	x, o	Text object: the reference under the cursor (e.g. <code>d<Alt-i></code>)

Commands: `:IlluminateToggle`, `:IlluminatePause`, `:IlluminateResume`.

Marks (Bookmarks)

Keymap	Description
<code>ma</code>	Set mark <code>a</code> at current cursor position
<code>`a</code>	Jump to the exact position of mark <code>a</code>
<code>'a</code>	Jump to the line of mark <code>a</code> (first non-whitespace)
<code>`u</code>	Jump to position where you last edited the file
<code>`.</code>	Jump to position of last change
<code>:marks</code>	List all marks

Marks `a-z` are local to the file. Marks `A-Z` are global (across files).

22. Jump Navigation (`hop.nvim`)

Keymap	Mode	Description
<code>f</code>	n, x, o	Type <code>f</code> then 2 characters: all matches highlight with jump labels. Press the label letter to jump. Case insensitive. <code><Esc></code> to cancel.

Note: Replaces Vim's built-in `f` motion (`F` is unchanged). `t{char}` jumps forward to just before a character on the current line, `T{char}` backward to just after it. `;` is mapped to `:`, so it does not repeat these motions; use `,` (opposite direction).

23. Search Lens (`nvim-hls`)

Keymap	Description
<code>n</code>	Next match with <code>[x/y]</code> count overlay
<code>N</code>	Previous match with count overlay
<code>*</code>	Search the word under the cursor forward as a whole word (the cursor stays on the word; with a count, e.g. <code>3*</code> , it jumps 3 matches forward from the cursor, like <code>3n</code>)
<code>#</code>	Same, backward

37. Symbol Outline (`aerial.nvim`)

The outline comes from Treesitter or the LSP server (no ctags needed).

Keymap	Description
<code><Space>t</code>	Toggle the symbol outline sidebar (functions, classes, methods; the cursor stays in your code)
<code>[t /]t</code>	Previous / next symbol (only in buffers where aerial is active; there they replace Vim's <code>:tprevious</code> / <code>:tnext</code> keys)

Commands: `:AerialToggle`, `:AerialOpen`, `:AerialNavToggle`.

50. Code Navigation Strategies

This section covers how developers typically navigate code in this setup.

Finding Files

Method	Keymap	Best for
Fuzzy file search	<code><Space>ff</code>	When you know part of the filename
Recent files	<code><Space>fr</code>	Files you've worked on recently
File explorer	<code><Space>s</code>	Browsing project structure visually
Buffer search	<code><Space>fb</code>	Switching between already-open files
Buffer pick	<code><Space>bp</code>	Quick switch when you can see the buffer tab
Buffer cycle	<code>gb</code> / <code>gB</code>	Cycling through open files linearly (<code>{N}gb</code> = buffer number N)

Finding Code

Method	Keymap	Best for
Project-wide grep	<code><Space>fg</code>	Searching for a string/pattern across all files
Word under cursor	<code>*</code> or <code>#</code>	Finding all uses of the current word in the file
Go to definition	<code>gd</code>	Jumping to where something is defined
Peek definition	<code><Space>gd</code>	Checking a definition without leaving context
Peek references	<code><Space>gr</code>	Seeing everywhere something is used
Buffer tags	<code><Space>ft</code>	Jumping to a function/class in the current file (uses ctags)
Symbol outline	<code><Space>t</code>	Sidebar (aerial) with all symbols in the file; focus stays in the code

Understanding Code

Method	Keymap	What you learn
Hover docs	<code>K</code>	Function signature, parameters, return type, docstring
Peek references	<code><Space>gr</code>	How and where this symbol is used
Git blame	<code><Space>hb</code>	Who wrote this line, when, and why (commit message)
Diagnostics	<code><Space>dd</code>	What's wrong with this line and where the error comes from
Code outline	<code><Space>t</code>	The structure of the file (classes, functions, methods)
Breadcrumb bar	(automatic)	Current location in the code shown at the top (dropbar)

Refactoring Code

Method	Keymap	What it does
Rename	<code><Space>rn</code>	Rename a symbol across the entire project
Code action	<code><Space>ca</code>	Auto-import, extract variable, fix lint issue, etc.
Format	<code><Space>fm</code>	Auto-format the file (LSP formatting; stylua in Lua buffers, prettier in Markdown buffers)
Comment/uncomment	<code>gcc / gc</code> (toggle), <code>gcs / gcr</code> (comment / uncomment explicitly; <code>gcsc / gcrr</code> for the current line)	Toggle or set comments
Surround	<code>sa / sd / sr</code>	Add/delete/replace quotes, brackets, etc.
Change inside	<code>ci(/ ci" / ci{</code>	Change text inside delimiters
Multiple replace	<code>* then ciw then n .</code>	Find-and-replace one at a time with control

Chapter 4: Editing, text objects, macros and tricks

4. Editing

Entering Insert Mode for Editing

Keymap	Description
<code>i</code>	Insert before cursor
<code>I</code>	Insert at beginning of line (first non-whitespace)
<code>a</code>	Insert after cursor
<code>A</code>	Insert at end of line
<code>o</code>	Open new line below and enter insert mode
<code>O</code>	Open new line above and enter insert mode
<code>s</code>	Note: <code>s</code> is disabled (used by vim-sandwich). Use <code>cl</code> instead (delete char + insert).
<code>S</code>	Delete entire line content and enter insert mode
<code>C</code>	Delete from cursor to end of line and enter insert mode (without polluting register)
<code>cc</code>	Delete entire line and enter insert mode (without polluting register)

Deleting Text

All delete operations also **cut** (yank) the text into a register, so you can paste it with `p`. Exception: `c`, `C`, `cc` (also `c` on a selection) in this config send to the black hole register (they do NOT save to paste register). `s` is NOT redirected: it fills the paste register.

Keymap	Mode	Description
<code>x</code>	n	Delete the character under the cursor
<code>X</code>	n	Delete the character before the cursor (like backspace)
<code>dd</code>	n	Delete (cut) the entire current line
<code>D</code>	n	Delete from cursor to end of line
<code>dw</code>	n	Delete from cursor to the start of the next word
<code>db</code>	n	Delete backward to the start of the previous word
<code>diw</code>	n	Delete the word under the cursor (inner word)
<code>daw</code>	n	Delete the word under the cursor + surrounding whitespace
<code>d\$</code>	n	Delete from cursor to end of line
<code>d0</code>	n	Delete from cursor to beginning of line
<code>dG</code>	n	Delete from current line to end of file
<code>dgg</code>	n	Delete from current line to start of file
<code>5dd</code>	n	Delete 5 lines starting from current

Copying (Yanking) Text

Keymap	Mode	Description
<code>yy</code>	n	Yank (copy) the current line
<code>yw</code>	n	Yank from cursor to start of next word
<code>yiw</code>	n	Yank the word under cursor
<code>y\$</code>	n	Yank from cursor to end of line
<code>y0</code>	n	Yank from cursor to beginning of line
<code>5yy</code>	n	Yank 5 lines
<code><Space>y</code>	n	Yank the entire buffer (custom)

Note: The clipboard is set to `unnamedplus` whenever a clipboard tool (wl-clipboard, xclip, ...) is installed, so yanking also copies to the system clipboard.

Pasting Text

Keymap	Mode	Description
<code>p</code>	n	Paste after the cursor
<code>P</code>	n	Paste before the cursor
<code>p</code>	x	Replace the selection with the register (yanky's paste: the replaced text goes into the register, so a second <code>p</code> pastes what was replaced; use <code>"0p</code> to paste the last yank again)
<code><Space>p</code>	n	Paste on a new line below (custom)
<code><Space>P</code>	n	Paste on a new line above (custom)
<code>[y</code>	n	After pasting, cycle to previous yank history entry (needs a paste first)
<code>]y</code>	n	After pasting, cycle to next yank history entry (needs a paste first)

yanky.nvim loads right after the first screen (VeryLazy), so every yank of the session is recorded in its history. `p/P` (Normal and Visual) are then yanky's paste with a 300 ms highlight of the pasted text, and `[y / ]y` exist (after a paste).

Changing (Delete + Enter Insert)

`c` (change) deletes text and puts you into insert mode. In this config, `c/c/cc` do NOT save the deleted text to the paste register (they use the black hole register).

Keymap	Description
<code>cw</code>	Change from cursor to end of word
<code>ciw</code>	Change the entire word under cursor
<code>caw</code>	Change the word + surrounding whitespace
<code>cc</code>	Change the entire line
<code>C</code>	Change from cursor to end of line
<code>c\$</code>	Same as <code>c</code>
<code>c0</code>	Change from cursor to beginning of line

Replacing Text

Keymap	Description
<code>r<char></code>	Replace the single character under cursor with <code><char></code>
<code>R</code>	Enter Replace mode (overtyping mode): every character you type replaces the existing one

Undo / Redo

Keymap	Description
<code>u</code>	Undo the last change
<code><Ctrl-r></code>	Redo (undo the undo)
<code><Space>u</code>	Toggle Neovim's builtin undo tree in a 30-column panel on the left

Undo breakpoints: Typing `,` `.` `!` `?` `;` `:` in insert mode creates undo checkpoints. This means pressing `u` after a long insert session will undo in smaller chunks instead of reverting everything at once.

Repeating Actions

Keymap	Description
<code>.</code>	Repeat the last change. Works with most editing commands. Extremely powerful: e.g., <code>ciw</code> + type new word + <code><Esc></code> , then move to another word and press <code>.</code> to repeat.

Line Operations

Keymap	Mode	Description
<code><Alt-j></code>	n	Move current line down one position (<code>3<Alt-j></code> moves it 3 lines; re-indented; nothing happens at the last line)
<code><Alt-k></code>	n	Move current line up one position (count works the same)
<code><Alt-j></code>	x	Move selected lines down (count works; re-indented; the selection stays)
<code><Alt-k></code>	x	Move selected lines up
<code>]<Space></code>	n	Insert a blank line below (cursor stays in place; <code>3]<Space></code> inserts 3)
<code>[<Space></code>	n	Insert a blank line above (cursor stays in place)
<code>J</code>	n	Join the current line with the next line (cursor stays in place; <code>3J</code> joins 3 lines)
<code>gJ</code>	n	Join lines without inserting a space (cursor stays in place)
<code>gS</code>	n	Toggle split / join of the list, argument list, table, dict or block under the cursor (treesj, Treesitter based)

Indentation

Keymap	Mode	Description
>>	n	Indent the current line to the right
<<	n	Indent the current line to the left
>	x	Indent selection right (stays in visual mode so you can press > again)
<	x	Indent selection left (stays in visual mode)
=	n, x	Auto-indent: fix indentation of the current line or selection
gg=G	n	Auto-indent the entire file

Insert Mode Shortcuts

Keymap	Description
<Ctrl-u>	Convert the current word to UPPERCASE (the word under or right before the cursor; if only spaces separate the cursor from the previous word, that word). You stay in Insert mode.
<Ctrl-t>	TOGGLE the case of the FIRST letter of that same word (foo -> Foo, press again -> foo); letters whose case does not round-trip (ß, 1) and non-letters are left alone; the cursor keeps its place
<Alt-;>	Insert a semicolon at the end of the line (without moving cursor)
<Ctrl-a>	Jump to the beginning of the line
<Ctrl-a> on the : command line	Jump to the start of the command line (the same key as in insert mode)
<Ctrl-e>	Jump to the end of the line (while the completion menu is open it closes the menu instead)
<Ctrl-d>	Delete the character to the right of the cursor
<Ctrl-w>	Delete the word before the cursor
<Ctrl-h>	Delete the character before the cursor (like back-space)
<Ctrl-s>	Show the LSP signature help of the function call you are typing

Miscellaneous Editing

Keymap	Mode	Description
<code><Space><Space></code>	n	Remove all trailing whitespace from the file (not in Markdown: there it only shows the warning "markdown: trailing spaces are hard line breaks, not stripped")
<code><Space>v</code>	n	Reselect the text that was just pasted
<code><Space>cl</code>	n	Toggle a vertical cursor column highlight
<code><Space>cb</code>	n	Blink the cursor (helps find it on screen)
<code>~</code>	n	Toggle case of character(s) (tilde is set as operator, so use <code>~w</code> for word, <code>~e</code> , etc.)

5. Selection (Visual Mode)

Press `v`, `V`, or `<Ctrl-v>` to enter visual mode, then use any motion to extend the selection. Once selected, you can act on the selection with `d` (delete), `y` (yank), `c` (change), `>` (indent), `<` (deindent), etc.

Selecting Characters

Keymap	Description
<code>v</code>	Start character-wise visual selection at cursor
<code>v + h / l</code>	Extend selection left / right by character
<code>v + w</code>	Extend selection to the next word
<code>v + b</code>	Extend selection backward to previous word
<code>v + e</code>	Extend selection to end of current word
<code>v + \$</code>	Extend selection to end of line
<code>v + 0</code>	Extend selection to beginning of line
<code>v + G</code>	Extend selection to end of file
<code>v + gg</code>	Extend selection to start of file
<code>v + }</code>	Extend selection to next blank line

Selecting Lines

Keymap	Description
<code>V</code>	Start line-wise visual (selects the entire current line)
<code>V + j / k</code>	Extend selection down / up by lines
<code>V + 5j</code>	Select current line + 5 lines below
<code>ggVG</code>	Select the entire file

Selecting Blocks (Columns)

Keymap	Description
<code><Ctrl-v></code>	Start block visual (rectangular selection)
<code><Ctrl-v> + j / k / h / l</code>	Extend the block in any direction
<code><Ctrl-v> + I</code>	Insert text at the start of every selected line (press <code><Esc></code> to apply)
<code><Ctrl-v> + A</code>	Append text at the end of every selected line
<code><Ctrl-v> + d</code>	Delete the selected block
<code><Ctrl-v> + c</code>	Change the selected block

This is extremely useful for editing columns of text, adding prefixes to multiple lines, etc.

Selecting Inside/Around Delimiters (Text Objects)

These are the most powerful selection commands. They work with `v` (select), `d` (delete), `c` (change), `y` (yank), and any other operator.

Parentheses, Brackets, Braces

Keymap	Description
<code>vi(</code>	Select everything inside <code>(...)</code>
<code>va(</code>	Select everything including <code>(...)</code> (the parentheses themselves)
<code>vib / vab</code>	Select inside / around the nearest enclosing pair of any of <code>()</code> , <code>[]</code> , <code>{}</code> (targets.vim; on <code>{ [x] }</code> with the cursor on <code>x</code> , <code>vib</code> selects <code>x</code>)
<code>vi[</code>	Select everything inside <code>[...]</code>
<code>va[</code>	Select everything including <code>[...]</code>
<code>vi{</code> or <code>viB</code>	Select everything inside <code>{...}</code>
<code>va{</code> or <code>vaB</code>	Select everything including <code>{...}</code>
<code>vi<</code>	Select everything inside <code><...></code>
<code>va<</code>	Select everything including <code><...></code>

Quotes

Keymap	Description
<code>vi"</code>	Select everything inside <code>"..."</code>
<code>va"</code>	Select everything including the <code>"</code> characters
<code>vi'</code>	Select everything inside <code>'...'</code>
<code>va'</code>	Select everything including the <code>'</code> characters
<code>vi`</code>	Select everything inside <code>`...`</code>
<code>va`</code>	Select everything including the <code>`</code> characters

Words, Lines, Paragraphs

Keymap	Description
<code>viw</code>	Select the word under cursor
<code>vaw</code>	Select the word + surrounding whitespace
<code>viW</code>	Select the WORD under cursor (delimited by whitespace only)
<code>vaW</code>	Select the WORD + surrounding whitespace
<code>vis</code>	Select the sentence (inner: without the trailing whitespace)
<code>vas</code>	Select the sentence + trailing whitespace (<code>das</code> , <code>dis</code> , <code>cis</code> work the same; the sentence motions are <code>(/)</code>)
<code>vis(/ vaS(</code>	vim-sandwich "query" objects: type the surrounding character after them (<code>vis(</code> selects inside the nearest <code>(...)</code> , <code>vaS</code> around the nearest <code>"..."</code> ; <code>vis</code> alone just waits for that character; <code>dis(</code> , <code>daS</code> , <code>cis[</code> work in the same way). <code>ib / ab</code> stay targets.vim's any-bracket objects
<code>vip</code>	Select the paragraph (block of non-empty lines)
<code>vap</code>	Select the paragraph + surrounding blank lines

Tags (HTML/XML)

Keymap	Description
<code>vit</code>	Select everything inside the nearest HTML/XML tag pair
<code>vat</code>	Select the entire tag pair including the tags

Using with Operators (d, c, y)

All the `vi` and `va` patterns above work with any operator, not just `v`:

Keymap	Description
<code>di(</code>	Delete everything inside parentheses
<code>da(</code>	Delete everything including the parentheses
<code>ci"</code>	Change text inside double quotes (deletes it and enters insert mode)
<code>ca"</code>	Change including the quotes themselves
<code>yi{</code>	Yank (copy) everything inside curly braces
<code>ya{</code>	Yank including the braces
<code>di[</code>	Delete everything inside square brackets
<code>dit</code>	Delete everything inside HTML tags
<code>ci'</code>	Change text inside single quotes
<code>dip</code>	Delete the entire paragraph

Treesitter Node Selection (builtin)

Neovim 0.12 can grow and shrink a selection along the syntax tree (needs a Treesitter parser; otherwise it uses the LSP selection range):

Keymap	Mode	Description
<code>an</code>	x, o	Select the parent (outer) node: press <code>v</code> , then <code>an</code> repeatedly to grow the selection
<code>in</code>	x, o	Select the child (inner) node: shrinks the selection again
<code>]n</code> / <code>[n</code>	x	Select the next / previous node
<code>]N</code> / <code>[N</code>	x	Select the next / previous sibling node

More Text Objects

Keymap	Mode	Description
<code>ii</code> / <code>ai</code>	x, o	The current indent scope (mini.indentscope); <code>ai</code> includes its border lines. <code>[i</code> / <code>]i</code> jump to the top / bottom of the scope
<code>i%</code> / <code>a%</code>	x, o	Inside / around a matching pair, also keywords like <code>if ... end</code> (vim-matchup)
<code>ia</code> / <code>aa</code>	x, o	Inside / around a function argument (targets.vim: on <code>b</code> in <code>f(a, b, c)</code> , <code>daa</code> gives <code>f(a, c)</code>)
<code>iq</code> / <code>aq</code>	x, o	Inside / around the nearest quotes of any kind (targets.vim)
<code>i,</code> / <code>a,</code>	x, o	Between separators such as <code>,</code> <code>;</code> <code>:</code> <code>+</code> <code>-</code> <code>=</code> <code>/</code> <code> </code> <code>&</code> (targets.vim)
<code><Space>iB</code>	x, o	The whole buffer (<code>y<Space>iB</code> copies everything)
<code><Space>iu</code>	x, o	The URL under the cursor (<code>d<Space>iu</code>)
<code><Alt-i></code>	x, o	The LSP/Treesitter reference under the cursor (vim-illuminate)

Markdown Code Block Text Objects

In markdown files only:

Keymap	Description
<code>vic</code>	Select inside a fenced code block (the object is <code>ic</code> , so <code>dic</code> , <code>yic</code> and <code>cic</code> work too)
<code>vac</code>	Select the code block including the fences (the object is <code>ac</code> : <code>dac</code> , <code>yac</code> , <code>cac</code>)

Precision Selection: From the Cursor to an Exact Spot

How to select (or delete/copy/change) from the cursor to a specific character, word, or place, including on another line.

Golden rules

- The character **under the cursor is always included** in a visual selection, both at the start and at the end.
- Everything below works after `v`. Most of it also works after an operator (`d`, `y`, `c`) in place of `v`: for example `vt` -> `dt`, `yt`, `ct`). Exceptions are noted below.
- A *selection* can only be one continuous area. You cannot select two separate places at once (see "Non-contiguous lines" below).

Quick lookup

I want to select from the cursor to...	Keys
the end of the line	<code>vL</code> (or <code>v\$</code>)
just before the next <code>x</code> on this line	<code>vtX</code>
including the next <code>x</code> on this line	<code>vtXL</code>
just before the 2nd <code>x</code> on this line	<code>v2tX</code>
the next <code>x</code> anywhere (crosses lines)	<code>v/X<Enter></code> (lands on the start of the match)
the end of the next word/text <code>foo</code>	<code>v/foo/e<Enter></code>
the 2nd <code>foo</code> after the cursor, anywhere in the file	<code>v2/foo/e<Enter></code>
something visible on screen, no counting	<code>v f xx <label></code> , then adjust with <code>l/h/e</code>
a target N lines below, count unknown	<code>Nj</code> , <code>L</code> , <code>?foo<Enter></code> , <code>viw</code>

End of line: `L`

- `L` is mapped to `g_` (last **non-blank** character of the line) in normal and visual mode; `H` is the first non-blank.
- `vL` selects from the cursor through the last character of the line (for example a trailing `;`). `v$` does the same (`$` is remapped to `g_` in visual mode only, so the newline is not selected).
- The mapping covers normal and visual mode only, **not operator-pending**. After an operator, `L` is the built-in "bottom of the screen" motion and works on whole lines. **Do not use** `dL` / `yL` / `cL`: `dL` was tested and deleted whole lines from the current one to the bottom of the window. Use `dg_` / `yg_` / `cg_` (tested: `dg_` deletes from the cursor through the last non-blank character), or `vL` then `d` / `y`.
- With a count, `2L` goes to the end of the line below.

`t` and `T`: stop just before a character

- `t<char>` jumps forward to the character **just before** the next `<char>`. `T<char>` does the same backward and stops just **after** it.
- It only searches the **current line**. If `<char>` is not on the line, nothing happens.
- It finds the next occurrence, not the last. The character under the cursor is not counted as a match.
- To **include** the target, add `l` after it: `vt:l` (up to and including the next `:`).
- `f` cannot be used for this, because `f` is hop.nvim here (it asks for 2 characters and shows labels). `vf:` does not do what you expect. After an operator `f` is still hop, but it is **not limited to the current line**: see the `df`, note below.
- Works with operators: `dt)` deletes up to before `)`, `yt"` copies up to before `"`, `ct)` changes up to before `)`. To include the target character with an operator, use `v` first: `vt)l` then `d`.
- **`f` after an operator (`df`, `,`, `cf`, `)` is hop, not the built-in `f`** (tested): `df`, deleted from the cursor through a comma several lines below (it was the only comma in the visible text), so it is **not limited to the current line** and the target character is **deleted too**. `cf`, deleted the same kind of range (tested with one match on screen: `cf`, deleted the same range as `df`, and left you in Insert mode; with several matches you pick the label first). With several matches on screen, hop

shows labels and you choose the target, so the deleted range depends on the label you press; check what you typed before pressing `d/c` on a big range (`u` undoes it).

Counting: `v2tX` is not `2vtX`

- Put the count **after** `v`, directly before `t`: `v2to` stops just before the 2nd `o` on the line. `v3t,` stops before the 3rd `,`.
- `2vt` is a **different** thing: a count typed before `v` reselects a region of that size, it does not mean "2nd occurrence".
- Counts also work with operators: `d2to`, `y2t`, `c3t`, .
- `T` counts backward: `v2T` .
- If the line has fewer occurrences than the count, nothing is selected or moved.

`)` and `(` are sentence motions, not parentheses

- `)` jumps to the **start of the next sentence**, and `(` to the start of the current/previous one. They do **not** mean "closing parenthesis". The search is not limited to the current line, so they can cross many lines.

What counts as the end of a sentence (standard Vim rules):

- A `.`, `!` or `?` that is followed by a **space, a tab or the end of the line**. Closing characters `)`, `]`, `"` or `'` may sit between the `./!/?` and that space (for example `done.")` `Next` still ends a sentence).
- A **blank line** also separates sentences (it is a paragraph boundary).
- A `.` with **no space after it does not count**: `obj.method()` or `file.txt` do not end a sentence. In code, only a `./!/?` followed by a space or at the end of a line counts (for example inside a string or a comment).

Where the cursor lands: on the **first non-blank character of the next sentence**, which is the first character after the terminator and the spaces that follow it.

Where a selection ends: in Visual mode the character under the cursor is included, so `v)` selects from the cursor **through that first letter of the next sentence** (the spaces before it are selected too). With an operator (`d`), (`y`) the motion stops just before that letter and does not include it (tested with `d`): with the cursor on the `t` of `there`, it deleted `there.` and the cursor ended on the `H` of `How`; `y)` was not tested).

Concrete example. Text on one line, cursor on the `t` of `there`:

```
Hello there. How are you? Fine!
```

Keys	What is selected / where the cursor ends
<code>v)</code>	<code>there. H</code> : ends on the <code>H</code> of <code>How</code> , the first letter of the next sentence (the <code>.</code> after <code>there</code> is the boundary)
<code>v))</code>	<code>there. How are you? F</code> : ends on the <code>F</code> of <code>Fine</code>
<code>v(</code>	from the start of <code>Hello</code> up to the <code>t</code> , because <code>(</code> goes back to the start of the current sentence

Tested once on real code: starting inside a line of code, `v)` ran across several lines and stopped on the first letter after a `.` that was inside a string literal (`"... text. Next ..."`, it stopped on the `N`), because that `.` counts as a sentence boundary even in code. All three rows of the table above were tested with the example text and gave exactly the results listed. Also tested: on a line like `see file.txt now. Next one.` with the cursor on `see`, `v)` selected `see file.txt now. N`, so the `.` in `file.txt` (no space after it) did not stop it.

Tip when testing these: type the sequence in one go (`v)`), not `v` and then `)` as separate steps). Pressing `v` again while already in Visual mode leaves Visual mode, and the which-key popup that appears after `v` is only a help list.

- To reach a closing parenthesis: `vt)l` (up to and including it), or `v/)<Enter>`.
- `%` jumps between a bracket and its match; it will not take you to a closing bracket from a plain character (it first finds the next bracket on the line and jumps to its partner, which can be backward).
- `va( / vi(` select the whole group / its inside, but always starting from the opening `(`, never from the cursor. Thanks to `targets.vim` the cursor may also be in front of the pair on the same line (tested: `vi(` on `foo (bar) baz` with the cursor on `f` selects `bar`).

Search as a selection motion (crosses lines)

Searching with `/` (forward) or `?` (backward) works as a motion after `v`:

Keys	The selection ends at
<code>v/foo<Enter></code>	the first character of the next <code>foo</code> (included)
<code>v/foo/e<Enter></code>	the last character of <code>foo</code> (included)
<code>v/foo/e-1<Enter></code>	one character before the end of <code>foo</code>
<code>v/foo/e+1<Enter></code>	one character after the end of <code>foo</code>
<code>v2/foo<Enter></code>	the start of the 2nd <code>foo</code> after the cursor
<code>v2/foo/e<Enter></code>	the end of the 2nd <code>foo</code> after the cursor

- The text after the second `/` (`e`, `e-1`, `e+1`) is a search **offset**; `e` means "end of the match".
- **The count counts matches, not lines.** `2/foo` means "the 2nd `foo` after the cursor, wherever it is". It does not care on which line a match is. If a line you expected has no `foo`, the count simply moves on to the next match.
- Because it counts matches, it also works **inside a single line**: if a line contains `foo` twice, `v2/foo/e<Enter>` selects up to the end of the second `foo` on that same line (tested: on a line with `Nome` and `nome`, the selection ended on the last letter of the second match, `nome`). (Unlike `t`, which also works on one line, search lets you target a whole word or phrase, not just one character.)
- If there are fewer matches than the count, the search **wraps** to the top of the file (`wrapscan` is on by default and this config does not change it) and keeps counting, so you can end up before the cursor. Check the `hlslens` `[n/total]` overlay.
- `ignorecase smartcase` is on, so a lowercase pattern (`foo`) also matches `Foo`. Type a capital letter to make it case-sensitive. (This is for `/` and `?`; `:s` and `:g` always ignore case here, see the Substitution section.)

- `/` is not remapped in this config.
- With operators, `d/foo<Enter>` deletes up to (not including) the match; `d/foo/e<Enter>` includes the last letter of the match (tested: with the cursor on the start of `two words`, `d/words/e<Enter>` deleted everything up to and including the last letter of `words`).

Hop: select to something you can see (no counting)

1. `v` starts the selection at the cursor.
2. `f` then type **2 characters**. Every matching place on screen gets a label (case insensitive).
3. Press the label letter. The cursor jumps there, and the selection now ends **on the first character of that match**, included.
4. Extend or trim with a normal motion:

Key	Effect
<code>l</code>	extend the selection one character to the right (<code>3l</code> = three characters)
<code>h</code>	move the end one character to the left : use it to pull back if you went one too far, or to extend leftward if the target is before the start
<code>e</code>	extend to the end of the word (no need to count its letters)
<code>b</code>	move back to the start of the word

Example: to select from the cursor to the end of a 4-letter word you can see: `v`, `f`, the word's first 2 letters, the label, then `ll` (first letter + 3 more) or simply `e`.

- Hop only reaches text that is **visible on screen**.
- Two-letter patterns match many places (for example `no` in `non`, `nome`, `Nome`), so look at the labels and pick the right one.

Target on another line, count unknown: use the relative numbers

With relative line numbers the gutter shows how far each line is from the cursor (the current line shows its absolute number).

1. `Nj` moves down `N` lines, where `N` is the number shown next to the target line (with a count, `j` moves real lines, not wrapped ones). Use `Nk` to go up.
2. `L` goes to the end of that line.
3. `?foo<Enter>` searches **backward** from the end of the line, which finds the **last** `foo` on that line (tested in a real terminal). In general `?foo` goes to the start of the `foo` the cursor is inside; from the first letter of a `foo` it goes to the previous `foo`. (Searching forward from the middle of the line could hit an earlier, unwanted match or a capitalised one.)
4. `viw` selects the word, or `ve` selects from the match start to the word end.

Related: `v3j` selects the current line and 3 below; `d3j` deletes 4 lines; `10G` or `;10` (Enter) jumps to absolute line 10.

Typing before or after: `i a I A`

Key	Where the typed text goes
<code>i</code>	before the character under the cursor
<code>a</code>	after the character under the cursor
<code>I</code>	before the first non-whitespace character of the line (not column 0: on an indented line it goes after the indentation; for the true start use <code>0</code> then <code>i</code>)
<code>A</code>	after the last character of the line (the real end, even past trailing spaces)

Typing at the absolute start of an indented line (before the indentation):

Keys	Where you start typing
<code>I</code>	after the indentation, before the first non-blank character
<code>0</code> then <code>i</code>	at column 0, before the indentation (preferred)
<code>gI</code>	the same as <code>0</code> then <code>i</code> , in one key (standard Vim; not remapped in this config)

- `0` is remapped to `g0` (start of the *screen* line), but `set nowrap` is on, so lines never wrap and it is the same as the real column 0.
- Tested: `gI` and `0` then `i` both start typing at column 0, before the indentation.
- These only insert from **Normal mode**. In Visual mode `i` and `a` do not insert: they start a text object (`iw`, `i(`, `aw`...). Press `<Esc>` first.
- **Typing after a selection that ends at the end of the line** (for example after `v$` or `vL`): press `<Esc>` (the cursor stays on the last character), then `a`. Using `i` would put the text *before* that last character (for example before a final `;`).
- **Shortest way to type at the end of the line:** `A`, from anywhere on the line, with no selection needed.
- `L` and `g_` stop on the last **non-blank** character, so `<Esc>` + `a` after `vL` types right after the last visible character, before any trailing spaces. `A` goes after the trailing spaces.

Non-contiguous lines (for example line 3 and line 10 together)

Not possible. Vim has no selection of separate pieces, and this config has no multi-cursor plugin (`vim-visual-multi` is commented out in `lua/plugin_specs.lua`). Do it in steps:

- **Repeat with** `.`: do the edit on one line, jump to the other, press `.`. Mind that deleting a line shifts the numbers below it.
- **Ex commands with line numbers:** `;3d` then `;9d`, or `;3,10d` for the whole range 3 to 10 (tested: `;3,10d` deleted lines 3 to 10 **inclusive**, 8 lines).
- **Bring them together:** `;3m10` moves line 3 **below** line 10, `;3t10` copies line 3 below line 10. Then select both with `v`. Tested: `;4m11` moved line 4 (an `import` line) to just below line 11, and `;4t11` copied line 4 below line 11 and kept the original in place. The line numbers refer to the file **before** the move, and afterwards the cursor sits on the moved/copied line.
- **The numbers are absolute file line numbers**, not relative to the cursor, and it does not matter where the cursor is. With relative numbers on, only the cursor line shows its absolute number in the gutter; move onto a line to read it. Line 1

counts even if it is blank (a file that starts with an empty line has its first real line at number 2).

- **Relative addresses:** `.` is the current line and `+N` / `-N` are N lines after/before it, so they match the relative numbers in the gutter. Tested: `;.t.` duplicated the current line and left the cursor on the new copy. Also tested: `;.m+2` moves the current line to below the line 2 lines further down, and `;.+,+3d` deletes the current line and the next 3 (4 lines in total).
- **Blank lines hide the effect:** moving or copying a blank line next to another blank line changes nothing you can see. Use a line with text to check `m` and `t`.
- **Same spot on adjacent lines:** `<Ctrl-v>` block mode, but only for adjacent lines (see the Visual Block Editing section).
- **Matching by content instead of number:** the `:g` command (see its section).

Line Range Yanking (Command Mode)

Command	Description
<code>:-5,+10yank</code>	Yank from 5 lines before to 10 lines after cursor
<code>:2,10yank</code>	Yank from line 2 to line 10 (absolute)

6. Working with Parentheses, Quotes, and Brackets

This section covers everything about matching, jumping to, selecting inside, changing, adding, and removing surrounding characters.

Jumping to Matching Pair

Keymap	Description
<code>%</code>	Jump between matching <code>()</code> , <code>[]</code> , <code>{}</code> , <code><></code> , and language keywords. The vim-matchup plugin extends this to work with <code>if/else/end</code> , <code>do/while</code> , <code>try/catch</code> , etc. If the match is offscreen, a popup shows the matching line.

Selecting Inside/Around Pairs

See the full table in the Selection section above. Quick summary:

Pattern	Inside	Around (including delimiters)
Parentheses <code>()</code>	<code>vi(</code>	<code>va(</code>
Braces <code>{}</code>	<code>vi{</code> OR <code>viB</code>	<code>va{</code> OR <code>vaB</code>
Brackets <code>[]</code>	<code>vi[</code>	<code>va[</code>
Angle brackets <code><></code>	<code>vi<</code>	<code>va<</code>
Double quotes <code>""</code>	<code>vi"</code>	<code>va"</code>
Single quotes <code>''</code>	<code>vi'</code>	<code>va'</code>
Backticks <code>``</code>	<code>vi`</code>	<code>va`</code>
HTML/XML tags	<code>vit</code>	<code>vat</code>
Nearest of <code>() [] {}</code> (targets.vim)	<code>vib</code>	<code>vab</code>

Changing Text Inside Pairs

Keymap	Description
<code>ci(</code>	Delete everything inside <code>()</code> and enter insert mode to type replacement
<code>ci"</code>	Delete everything inside <code>"</code> and enter insert mode
<code>ci{</code>	Delete everything inside <code>{}</code> and enter insert mode
<code>ci[</code>	Delete everything inside <code>[]</code> and enter insert mode
<code>ci'</code>	Delete everything inside <code>'</code> and enter insert mode
<code>cit</code>	Delete everything inside an HTML tag and enter insert mode

Deleting Text Inside Pairs

Keymap	Description
<code>di(</code>	Delete everything inside <code>()</code> (parentheses remain empty)
<code>di"</code>	Delete everything inside <code>"</code>
<code>di{</code>	Delete everything inside <code>{}</code>
<code>da(</code>	Delete everything including the <code>()</code> themselves
<code>da"</code>	Delete everything including the <code>"</code> themselves

Adding Surrounding Pairs (vim-sandwich Plugin)

The `sa` command adds surrounding characters. `s` key alone is disabled (use `cl` instead).

Keymap	Description	Example
<code>saiw"</code>	Add double quotes around the current word	<code>hello</code> becomes <code>"hello"</code>
<code>saiw(</code>	Add parentheses around the current word	<code>hello</code> becomes <code>(hello)</code>
<code>saiw{</code>	Add curly braces around the current word	<code>hello</code> becomes <code>{hello}</code>
<code>saiw[</code>	Add square brackets around the current word	<code>hello</code> becomes <code>[hello]</code>
<code>saiw'</code>	Add single quotes around the current word	<code>hello</code> becomes <code>'hello'</code>
<code>sa\$"</code>	Add quotes from cursor to end of line	
(visual) <code>sa"</code>	First select text with <code>v</code> , then <code>sa"</code> adds quotes around selection	

Removing Surrounding Pairs (vim-sandwich Plugin)

Keymap	Description	Example
<code>sd"</code>	Delete surrounding double quotes	<code>"hello"</code> becomes <code>hello</code>
<code>sd'</code>	Delete surrounding single quotes	<code>'hello'</code> becomes <code>hello</code>
<code>sd(</code>	Delete surrounding parentheses	<code>(hello)</code> becomes <code>hello</code>
<code>sd{</code>	Delete surrounding curly braces	<code>{hello}</code> becomes <code>hello</code>
<code>sdb</code>	Delete the nearest surrounding pair, whatever it is (<code>()</code> , <code>[]</code> , <code>{}</code> or quotes)	<code>[hello]</code> becomes <code>hello</code>
<code>sd[</code>	Delete surrounding square brackets	<code>[hello]</code> becomes <code>hello</code>

Replacing Surrounding Pairs (vim-sandwich Plugin)

Keymap	Description	Example
<code>sr"</code>	Replace <code>"</code> with <code>'</code>	<code>"hello"</code> becomes <code>'hello'</code>
<code>sr{</code>	Replace <code>()</code> with <code>{}</code>	<code>(hello)</code> becomes <code>{hello}</code>
<code>sr[</code>	Replace <code>{}</code> with <code>[]</code>	<code>{hello}</code> becomes <code>[hello]</code>
<code>sr' (</code>	Replace <code>'</code> with <code>()</code>	<code>'hello'</code> becomes <code>(hello)</code>
<code>srb'</code>	Replace the nearest surrounding pair, whatever it is	<code>"hello"</code> becomes <code>'hello'</code>

Auto-Pairing (nvim-autopairs Plugin)

When typing in insert mode, opening characters automatically insert their closing pair:

- Type `(` and `)` appears: `()` (cursor between them)
- Type `"` and closing `"` appears: `"|"`
- Type `{` and `}` appears: `{|}`
- Type `[` and `]` appears: `[|]`

16. Code Commenting

vim-commentary (Plugin)

Keymap	Mode	Description
<code>gcc</code>	n	Toggle comment on current line
<code>gc</code> + motion	n	Toggle comment on a motion (e.g., <code>gcip</code> comments a paragraph)
<code>gc</code>	v	Toggle comment on selected lines
<code>gc</code>	o	Comment text object: <code>dgc</code> deletes the comment block under the cursor, <code>ygc</code> yanks it
<code>gcu</code>	n	Uncomment the adjacent commented lines
<code>:[range]Commentary</code>	cmd	Toggle comment on a range (<code>:2,3Commentary</code>)

vim-commentary loads right after the first screen (VeryLazy), so these commands and the `gc` text object exist from then on.

Smart Commenting (Custom)

String-aware, multi-line-capable comment add/remove (`lua/smart_comment/`).

Keymap	Mode	Description
<code>gcs</code> + motion	n	Comment the rows a motion covers (<code>gcsip</code> , <code>gcs3j</code> , <code>gcsG</code>); <code>.</code> repeats
<code>{count}gcs</code>	n	Comment count rows from the cursor down (<code>200gcs</code>)
<code>gcSS</code>	n	Comment current line(s) (<code>3gcSS</code> = 3 rows); <code>.</code> repeats
<code>gcs</code>	x	Comment the selected rows
<code>gcr</code> + motion	n	Uncomment the rows a motion covers (<code>gcrip</code> , <code>gcr200j</code>); <code>.</code> repeats
<code>{count}gcr</code>	n	Uncomment count rows from the cursor down (<code>200gcr</code>)
<code>gcrR</code>	n	Uncomment current line(s) (<code>3gcrR</code> = 3 rows); <code>.</code> repeats
<code>gcr</code>	x	Uncomment the selected rows

Rule: `gcs` adds one marker per row and never nests (redundant markers already in the rows are removed); `gcr` removes every marker level (`##a` -> `a`). Only real comment markers of the file type count (a `#` in a colour, URL or one-line string is not a comment).

With a motion, put the count AFTER `gcs` / `gcr` (`gcs3j` , `gcr200j`): a count before them always means rows, so `3gcsip` comments 3 rows and then `ip` runs as normal keys. If you pause after `gcs` longer than 'timeoutlen', a following `s` no longer makes `gcSS` (`gcs` is then waiting for a motion and `s` cancels it); `gcsip` typed in one go is not affected.

`gcs` / `gcr` always change WHOLE rows (a charwise motion such as `gcse` changes every row it touches); the cursor stays on its text row; `.` repeats with the same motion, count or number of Visual rows. In a non-modifiable buffer they show one warning. Avoid `gcs gcs` (the second `gc` is the comment text object).

Fully supported languages: asm, bash, c, cmake, conf, cpp, cs, css, dockerfile, elixir, fish, gitconfig, go, haskell, hcl, html, i3config, java, javascript, julia, kitty, kotlin, lisp (Emacs Lisp / Common Lisp), lua, make, markdown, nix, perl, php, ps1 (PowerShell), python, r, ruby, rust, scala, sh, sql, swift, terraform, tex (LaTeX), tmux, toml, typescript, typst, vim, xml, yaml, zig, zsh. In gitconfig an UNQUOTED `#ff0000` is a comment for git itself, so `gcr` removes it: quote colours there.

Other filetypes use their 'commentstring': a marker counts only as the first non-blank character of the row.

17. Surrounding Pairs (`vim-sandwich` + `nvim-autopairs`)

See [Section 6: Working with Parentheses, Quotes, and Brackets](#) for the complete guide.

Quick reference:

Keymap	Description
<code>saiw"</code>	Add <code>"</code> around word
<code>sd"</code>	Delete surrounding <code>"</code>
<code>sr"'</code>	Replace <code>"</code> with <code>'</code>
<code>%</code>	Jump to matching bracket

24. Yank History (`yanky.nvim`)

Keymap	Mode	Description
<code>p</code> / <code>P</code>	n, x	Paste after / before (with 300ms highlight)
<code>[y</code>	n	After pasting, cycle to previous yank entry
<code>]y</code>	n	After pasting, cycle to next yank entry

Command: `:YankyRingHistory` to browse all yank history. `yanky.nvim` loads right after the first screen (VeryLazy), so the yank history contains every yank of the session. In Visual mode `p` is yanky's: it overwrites the unnamed register with the replaced text (there is no separate keep-register map any more).

25. Undo History

Keymap	Description
<code><Space>u</code>	Toggle Neovim's builtin undo tree (<code>nvim.undotree</code>) in a 30-column panel on the far left

Inside the panel there are no extra keys: moving the cursor onto an entry switches the buffer to that undo state (documented in `:h undotree.open()`). Close it with `<Space>u` again or `:q`.

29. Registers & Macros

Registers

Keymap	Description
<code>"3y</code>	Yank to register 3
<code>"3p</code>	Paste from register 3
<code>"*y</code> / <code>"+y</code>	Yank to system clipboard
<code>:reg</code>	View all registers

Note: When a clipboard tool (wl-clipboard, xclip, ...) is installed, `clipboard` is `unnamedplus`, so `y/p` already use the system clipboard by default.

Macros

`Q` is an extra key for recording: `Qa` and `qa` both start recording into register `a` (tested: plain `q` works too, `q` is mapped to `q`).

Keymap	Description
<code>Qh</code>	Start recording macro to register <code>h</code>
<code>q</code>	Stop recording
<code>@h</code>	Play macro from register <code>h</code>
<code>5@h</code>	Play macro 5 times
<code>@@</code>	Replay the last played macro

60. Macros In Depth

Macros record a sequence of keystrokes and replay them. They are one of the most powerful features in Vim for repetitive editing.

Key mapping: In this config `q` is mapped to `qq`, so `q` and `qq` both start recording (tested: plain `q` works too). Stopping is always `q`.

Recording a Macro

1. Press `q` followed by a register letter (e.g., `qa` to record into register `a`)
2. The command line (bottom line) shows `recording @a` (tested in a real terminal) – everything you do now is being recorded
3. Perform the editing actions you want to repeat
4. Press `q` to stop recording

Playing a Macro

Keymap	What it does
<code>@a</code>	Play macro from register <code>a</code> once
<code>5@a</code>	Play macro 5 times
<code>@@</code>	Replay the last played macro
<code>100@a</code>	Play 100 times (stops early if it hits an error, e.g., end of file)

Scenario: Add Semicolons to the End of 20 Lines

1. Place cursor on the first line
2. `qa` – start recording to register `a`
3. `A;<Esc>` – go to end of line, add semicolon, back to normal mode
4. `j` – move down one line
5. `q` – stop recording
6. `19@a` – replay 19 more times (20 lines total)

Scenario: Wrap Each Line in Double Quotes

Starting with:

```
apple
banana
cherry
```

1. Cursor on line 1
2. `qa` – start recording
3. `I"<Esc>` – insert `"` at beginning
4. `A"<Esc>` – append `"` at end
5. `j` – move down
6. `q` – stop recording
7. `2@a` – replay for remaining lines

Result:

```
"apple"
"banana"
"cherry"
```

Scenario: Append the Same Text to a Block of Lines (Tested)

Starting with 4 consecutive lines that each end with `;`, add `// ok` to the end of every one.

1. Put the cursor on the first of the 4 lines, at any column.
2. `0a` – start recording to register `a`
3. `A // ok<Esc>` – jump to the end of the line, type the text, back to Normal mode
4. `j` – move down one line, ready for the next run
5. `q` – stop recording. Only line 1 has changed so far (you did the edit while recording).
6. `3@a` – replay 3 more times for the other 3 lines (total lines - 1)

Result: all 4 lines end with `; // ok`.

- **Why it repeats well:** `A` goes to the end of the line wherever the cursor is, so the column does not matter, and the macro ends on `j`, so the next run starts on the next line.
- **Where the cursor ends:** one line **below** the last processed line (the final `j` of the last run). If that line is blank, the cursor is on the blank line. When the block is the last in the file the final `j` fails and the cursor stays on the last line.
- **Count:** use `number of lines - 1`, because the recording already did the first line. A count that goes past the last line stops early with an error, which is harmless.
- **Undo:** one `u` undoes the whole replay (lines 2-4); the line you edited by hand while recording needs a second `u` (tested).

Scenario: Convert a List of Variables to Assignments

Starting with:

```
name
age
email
```

Turn each into `self.name = name`:

1. Cursor on `name`
2. `0a` – start recording
3. `0yiw` – yank the word
4. `Iself.<Esc>` – prepend `self.`
5. `A = <Esc>p` – append `=` and paste the word
6. `j` – next line
7. `q`, then `2@a` (tested: gives `self.name = name`, `self.age = age`, `self.email = email`)

Scenario: Turn CSV into SQL VALUES

Starting with `John,30,john@email.com`:

1. `0a` – start recording
2. `I('<Esc>` – prepend `('`
3. `:s/,/,',/g<CR>` – replace the commas on this line only (use `:s`, not `:%s`)
4. `A'),<Esc>` – append `')`,
5. `j` – next line
6. `q` then replay

Tips for Writing Macros

- **Start from a consistent position:** Begin each macro from a fixed position in the line (`0` start of the line, `H` first non-blank) or from a search result. This makes the macro repeatable.
- **Use word motions, not character motions:** `w`, `e`, `b` work regardless of word length. `3l` only works for a specific column.
- **End on the next line:** If processing line-by-line, end the macro with `j` (move down) so replaying it processes subsequent lines.
- **Test with `@a` once:** Play the macro once to verify before running `100@a`.
- **Check existing registers:** `:reg` shows what's stored in each register. Macros and yanks share registers, so recording to `a` overwrites whatever was yanked to `a`.
- **Use a high count:** `999@a` will replay until it fails (e.g., end of file). Vim stops automatically on error.

Visual Mode Macros

You can apply a macro to every line in a visual selection:

1. Select lines with `v` + `j/k`
2. Type `:normal @a` and press Enter
3. The macro runs on each selected line

61. The Dot Command (`.`) - Repeating Actions

The `.` key repeats the last change. This is arguably the most important efficiency tool in Vim.

What Counts as a "Change"

- Any editing in insert mode between `i ... <Esc>` (typed text, deletions, etc.)
- Any operator command: `dd`, `dw`, `ciw`, `>>`, `gcc`, etc.
- Surroundings: `saiw`", `sd`", `sr`"'
- Plugin actions (via vim-repeat): sandwich, commentary, etc.

Scenario: Change a Variable Name One-by-One

1. Place cursor on the word `oldName`
2. `*` - search for it (highlights all occurrences)
3. `ciw` - change inner word, type `newName`, press `<Esc>`
4. `n` - jump to next occurrence
5. `.` - repeat the change (replaces `oldName` with `newName`)
6. `n` - next occurrence
7. `.` - repeat again
8. Skip an occurrence? Just press `n` without `.`

This gives you manual control over each replacement, unlike `:%s` which replaces all at once.

Scenario: Add a Prefix to Multiple Lines

1. On the first line: `I// <Esc>` (insert `//` at start)
2. `j` - move down
3. `.` - repeat (adds `//` to this line too)
4. `j.j.j.` - keep going

Scenario: Delete the First Word on Several Lines

1. On the first line: `0dw` (go to start, delete word)
2. `j` - move down
3. `.` - repeat
4. Continue `j.` as needed

Scenario: Indent Multiple Blocks

1. On a line: `>>` (indent right)
2. `.` - indent again (double indent)
3. Move to another line, `.` - indent that line too

Combining `.` with Counts

- `3.` repeats the last change 3 times
 - `5>>` then `.` repeats the 5-line indent
-

62. Visual Block Editing (Multi-Cursor-Like)

Visual block mode (`<Ctrl-v>`) lets you edit rectangular columns of text. This is the closest thing to multi-cursor editing.

Scenario: Add a Prefix to Multiple Lines at Once

```
line one
line two
line three
```

1. Place cursor at the start of `line one`
2. `<Ctrl-v>` - enter block visual mode
3. `2j` - extend selection down 2 lines (column is now selected on 3 lines)
4. `I` - enter insert mode (capital I, for block insert)
5. Type `//` (or any prefix)
6. Press `<Esc>` - the prefix appears on ALL three lines

Result:

```
// line one
// line two
// line three
```

Scenario: Append Text to Multiple Lines

```
item1
item2
item3
```

1. `<Ctrl-v>` then `2j` - select the column
2. `$` - extend selection to end of each line
3. `A` - enter append mode (capital A)
4. Type `,` (or any suffix)
5. `<Esc>` - applied to all lines

Result:

```
item1,  
item2,  
item3,
```

Scenario: Delete a Column

If you have aligned text and want to remove a column:

1. `<Ctrl-v>` – block visual
2. Move to select the rectangular region (e.g., `3j10l`)
3. `d` – delete the block

Scenario: Replace a Column

1. `<Ctrl-v>` – select the column
 2. `c` – change (deletes the block and enters insert mode)
 3. Type the replacement
 4. `<Esc>` – applied to all lines
-

64. Everyday Editing Scenarios

Swap Two Lines

1. On the first line: `dd` (cut it)
2. Move to where you want it: `j` or `k`
3. `P` (paste above) or `p` (paste below)

Or use `<Alt-j>` / `<Alt-k>` to move lines up/down without cutting.

Swap Two Words

Plugin: **vim-swap**. Put the cursor on an item of a comma-separated list (function arguments) and press `gs`: this starts "swap mode". There `h/l` move the current item left/right (swap with its neighbour), `j/k` choose another item, `1-9` choose the nth item, `s/S` sort ascending/descending, `r` reverses, `u/<Ctrl-r>` undo/redo, `<Esc>` leaves swap mode.

For manual word swap:

1. On the first word: `diw` (delete inner word)
2. Move to the second word: `w` or `f`
3. `viwp` – select the second word and paste (swaps them)

Duplicate a Line

1. `yy` – yank the line
2. `p` – paste below

Or: `yyP` (same thing).

Duplicate a Block of Code

1. Select the block with `v + j/k`
2. `y` – yank
3. Navigate to destination
4. `p` – paste

Fix Indentation of Entire File

1. `gg=G` – go to top, auto-indent everything to bottom

Remove All Blank Lines

1. `:%g/^$/d` – globally delete lines matching "empty"

Sort Lines

1. Select lines with `v` + movement
2. `:sort` – sort alphabetically
3. `:sort!` – reverse sort
4. `:sort n` – numeric sort
5. `:sort u` – sort and remove duplicates

Convert Tabs to Spaces (or Vice Versa)

1. `:set expandtab` (already set by default)
2. `:retab` – convert all tabs to spaces in the file
3. Or `:set noexpandtab` then `:retab!` for spaces-to-tabs

Wrap a Selection in a Tag/Function

1. Select text with `v` or `V`
2. `sa` + the surrounding character (vim-sandwich)
3. For example: select `myVar`, then `sa"` wraps it as `"myVar"`
4. For function: type `sa` then `f` then the function name – wraps as `funcName(myVar)`

65. Swapping Function Arguments (vim-swap)

Plugin: **vim-swap**. Swap delimited items (function arguments, list elements, etc.) without cutting and pasting.

Place your cursor on one of the arguments inside parentheses:

Keymap	What it does
<code>gs</code>	Start swap mode (then <code>h/l</code> swap with the neighbour, <code>j/k</code> choose an item, <code>l</code> - <code>g</code> pick an item, <code><Esc></code> exit)

Example: Given `func(a, b, c)` with the cursor on `b`, press `gs`, then `l`: `b` moves one place right, giving `func(a, c, b)`. Press `<Esc>` to leave swap mode (tested).

Works with any comma-separated list: function arguments, array literals, dictionary entries, etc.

68. Useful Vim Tricks

Run a Normal-Mode Command on Every Line

`:g/pattern/normal @a` – run macro `a` on every line matching `pattern` `:g/pattern/normal dd` – delete every line matching `pattern` `:v/pattern/normal dd` – delete every line NOT matching `pattern` (inverse)

Execute a Command on a Range

`:10,20normal A;` – append semicolons to lines 10-20 `:10,20normal I//` – comment out lines 10-20 `:'<,'>normal @a` – run macro `a` on visually selected lines

Increment/Decrement Numbers

Keymap	What it does
<code><Ctrl-a></code>	Increment the number under cursor
<code><Ctrl-x></code>	Decrement the number under cursor
<code>10<Ctrl-a></code>	Add 10 to the number
<code>g<Ctrl-a></code> (visual block)	Sequential increment: line 1 gets +1, line 2 gets +2, line 3 gets +3...
<code>g<Ctrl-x></code> (visual block)	Sequential decrement (same idea, subtracting)

Scenario: Generate a numbered list. Type `0.` on 5 lines, select them with `<Ctrl-v>`, then `g<Ctrl-a>` turns them into `1. 2. 3. 4. 5.`

Important: `g<Ctrl-a>` doesn't "know" what you want incremented – it purely acts on whatever number the highlighted column(s) overlap on each line, ignoring every other number on the line. Wherever you place the block is what gets incremented. Only the digits inside the block count as the number.

Gotcha - starting value: because it *adds* `1xn` to each line, starting from `1` gives `1+1=2, 1+2=3, 1+3=4...` (starts at 2, not 1). If you want the sequence to start at 1, your placeholder number must start at `0`.

Example 1 - single number on the line (increments normally):

```
imgur_japan_0.jpg
imgur_japan_0.jpg
imgur_japan_0.jpg
```

Put the cursor on the `0`, `<Ctrl-v>`, `G` to extend the block down that same column to the last line, then `g<Ctrl-a>`:

```
imgur_japan_1.jpg
imgur_japan_2.jpg
imgur_japan_3.jpg
```

Example 2 - two numbers on the line, only one should change:

```
0001-photo-0.jpg
0001-photo-0.jpg
0001-photo-0.jpg
```

Here `0001` is a fixed ID that must stay identical on every line, and only the trailing `0` should become sequential. Place the cursor on the trailing `0` (not on `0001`), `<Ctrl-v>`, `G`, `g<Ctrl-a>`:

```
0001-photo-1.jpg
0001-photo-2.jpg
0001-photo-3.jpg
```

The `0001` is left untouched because the visual-block column never overlapped it – only the number your block touches gets incremented, no matter how many other numbers appear elsewhere on the line.

Common use case: renaming a batch of files. Open the filenames in a buffer (e.g. via a bulk-rename tool that spawns `$EDITOR` with one filename per line, in file order), reduce every line to the same placeholder with `:%s/./newname_0.jpg/`, then apply the block-select + `g<Ctrl-a>` trick above to turn the shared `0` into a sequence.

Example 3 - starting the sequence at 0 instead of 1:

`g<Ctrl-a>` always adds `1xn`, so it naturally starts at 1. To start at 0, do the sequential increment first, then shift the whole result down by one with a **plain** (non-`g`) `<Ctrl-x>` on the same block – plain block increment/decrement applies the *same* amount to every line instead of a growing amount:

newname_0.jpg		newname_1.jpg		newname_0.jpg
newname_0.jpg	<code>g<C-a></code>	newname_2.jpg	<code><C-x></code>	newname_1.jpg
newname_0.jpg	<code>-----></code>	newname_3.jpg	<code>-----></code>	newname_2.jpg
newname_0.jpg		newname_4.jpg		newname_3.jpg

Reselect the exact same column block before pressing `<Ctrl-x>` (block selections don't persist across a `g<Ctrl-a>` – you need `<Ctrl-v>` + `g` again). This same "sequential increment, then uniform shift" combo also works for shifting a sequence to start at any number.

Example 4 - multi-digit, zero-padded numbers:

Vim preserves the digit width/leading zeros automatically as long as you don't overflow it:

```
photo_001.jpg
photo_001.jpg
photo_001.jpg
```

Put the cursor on the first `0` of `001`, widen the block over all three digits (`<Ctrl-v>ll`, then `g`), then `g<Ctrl-a>`:

```
photo_002.jpg
photo_003.jpg
photo_004.jpg
```

Warning: a block that covers only part of the digits increments only those digits (cursor on the first `0` with a 1-column block gives `photo_101.jpg`, `photo_201.jpg`, ...). Leading zeros are kept as long as the number does not outgrow its width (tested).

If you want it to start at `001` instead of `002`, apply the same Example 3 shift: reselect the block, plain `<Ctrl-x>` once.

Letter Sequences (a, b, c...) Instead of Numbers

`<Ctrl-a>/<Ctrl-x>` and their `g`-prefixed block variants also work on **letters**, but only once you opt in, since alphabetic increment isn't in Neovim's default `nrformats`:

```
:set nrformats+=alpha
```

(Add this to your Neovim config if you want it permanently; otherwise it only applies to the current session.)

With that set, bulk-renaming to `a, b, c, d...` works the same way as numbers:

```
a
a      g<C-a>    b
a  ----->    c
a      d
```

Note this starts at `b`, not `a` – same off-by-one as numbers, fixed the same way (Example 3): reselect the block and press plain `<Ctrl-x>` once to shift down:

```
b      a
c  <C-x> b
d  -----> c
e      d
```

`<Ctrl-a>/<Ctrl-x>` do NOT wrap at the alphabet boundary: `<Ctrl-a>` on `z` (or `Z`) leaves it unchanged (tested). Case is kept (`a` -> `b`, `A` -> `B`).

Open the File Under Cursor

Keymap	What it does
<code>gf</code>	Open the file path under cursor (if it exists)
<code><Ctrl-w>f</code>	Open file under cursor in a split
<code>gx</code>	Open the URL or file under the cursor (<code>gx.nvim</code> ; also on a Visual selection)

Change Case

Keymap	What it does
<code>~</code>	Toggle case of character(s). Since <code>tildeop</code> is set, use with a motion: <code>~w</code> toggles case of a word, <code>~e</code> to end of word.
<code>gUw</code>	Uppercase the entire word
<code>guiw</code>	Lowercase the entire word
<code>gUU</code>	Uppercase the entire line
<code>guu</code>	Lowercase the entire line
(in insert mode) <code><Ctrl-u></code>	Uppercase the current word (custom; the word touching the cursor, or the previous word if only spaces are before the cursor)
(in insert mode) <code><Ctrl-t></code>	Toggle the case of the first letter of the current word (custom; <code>foo</code> <-> <code>Foo</code>)

Align Text

Plugin: **tabular**. Aligns text around a character. `:Tabularize` works in every filetype from a fresh start (it loads on the first `:Tabularize`, and with the first Markdown file).

Command	What it does
<code>:Tabularize /=</code>	Align all <code>=</code> signs in a selection or file
<code>:Tabularize /:</code>	Align colons (for JSON/YAML-like structures)

To align the pipes of a Markdown table, type `:Tabularize /|` (shown outside the table because a pipe breaks a table row).

Command Abbreviations

This config sets some command abbreviations (type the short form as the first word of the command line, then press space or Enter):

Short	Expands to
<code>git</code>	<code>Git</code> (fugitive; only inside a git repository)
<code>man</code>	<code>Man</code> (manual pages)
<code>edit</code>	<code>Edit</code> (multi-file edit)
<code>pi</code>	Lazy <code>install</code>
<code>pud</code>	Lazy <code>update</code>
<code>pc</code>	Lazy <code>clean</code>
<code>ps</code>	Lazy <code>sync</code>
<code>z</code>	<code>z</code> (zoxide jump; only when it is the whole command so far)
<code>norm</code>	<code>Norm</code> (<code>:norm</code> with a live preview while you type)

Two insert-mode abbreviations fix typos: `require -> require`, `serveral -> several`.

71. The Global Command (`:g`)

The global command runs an Ex command on every line matching a pattern. It's one of the most powerful built-in features.

Syntax: `:g/pattern/command`

The inverse (`:v`) runs on lines that do NOT match: `:v/pattern/command`

Common Uses

Command	What it does
<code>:g/TODO/d</code>	Delete every line containing <code>TODD</code>
<code>:v/TODO/d</code>	Delete every line that does NOT contain <code>TODD</code> (keep only <code>TODD</code> lines)
<code>:g/^\$/d</code>	Delete all blank lines
<code>:g/^\s*\$/d</code>	Delete all blank lines (including whitespace-only)
<code>:g/console\.log/d</code>	Delete all <code>console.log</code> lines
<code>:g/pattern/normal @a</code>	Run macro <code>a</code> on every matching line
<code>:g/pattern/normal A;</code>	Append semicolon to every matching line
<code>:g/pattern/normal I//</code>	Comment out every matching line
<code>:g/pattern/t \$</code>	Copy every matching line to the end of file
<code>:g/pattern/m 0</code>	Move every matching line to the top of file
<code>:g/^import/normal >></code>	Indent all import lines

Everyday Scenarios

Delete All Print/Debug Statements

```
:g/print(/d          -- Python: delete all print() lines
:g/console\.log/d    -- JS: delete all console.log lines
:g/System\.out\.print/d -- Java: delete all System.out.println lines
:g/fmt\.Print/d      -- Go: delete all fmt.Print lines
```

Keep Only Lines Matching a Pattern

```
:v/error/d          -- keep only lines containing "error"
:v/\v(import|from)/d -- keep only import statements
```

Extract All Function Signatures

```
:v/\vdef \w+\(/d      -- Python: keep only function definitions
:v/\v(public|private|protected)/d -- Java: keep only method/field declarations
```

Add Prefix/Suffix to Matching Lines

```
:g/TODO/normal I[URGENT] -- add "[URGENT] " before every TODO line
:g/^#/normal A <!-->    -- add comment marker after every markdown heading
```

Sort Lines Matching a Pattern to Top

```
:g/import/m 0          -- move all import lines to the top of the file
```

76. Common Editing Power Combos

Quick-reference card of the most powerful editing combinations for daily use.

Changing Text

Combo	What it does	Example
<code>ciw</code>	Change the word under cursor	<code>foo</code> -> type <code>bar</code> -> <code>bar</code>
<code>ci"</code>	Change text in double quotes	<code>"old"</code> -> type <code>new</code> -> <code>"new"</code>
<code>ci(</code>	Change text in parentheses	<code>func(old)</code> -> type <code>new</code> -> <code>func(new)</code>
<code>ci{</code>	Change text in braces	<code>{old}</code> -> type <code>new</code> -> <code>{new}</code>
<code>cit</code>	Change text in HTML tag	<code><p>old</p></code> -> type <code>new</code> -> <code><p>new</p></code>
<code>cc</code>	Change entire line	Clears line, insert mode
<code>C</code>	Change from cursor to end of line	Deletes rest of line, insert mode
<code>c\$</code>	Same as <code>c</code>	
<code>ct)</code>	Change from cursor to before <code>)</code>	Useful inside function arguments
<code>cf, + char + label</code>	Change from the cursor through a <code>,</code> picked with <code>hop</code> : <code>f</code> is <code>hop</code> 's 2-character jump, so type <code>,</code> and the character after it, then the label (may be several lines away; see "Precision Selection")	<code>f</code> is <code>hop.nvim</code> here, not the built-in

Deleting Text

Combo	What it does
<code>diw</code>	Delete word under cursor
<code>daw</code>	Delete word + surrounding spaces
<code>di"</code>	Empty out double-quoted string
<code>da"</code>	Delete the entire quoted string including quotes
<code>di(</code>	Empty out parentheses
<code>da(</code>	Delete parentheses and their contents
<code>dip</code>	Delete paragraph
<code>dd</code>	Delete line
<code>D</code>	Delete from cursor to end of line
<code>dt)</code>	Delete from cursor to before <code>)</code>

Copying Text

Combo	What it does
<code>yiw</code>	Copy word under cursor
<code>yi"</code>	Copy text inside double quotes
<code>yi(</code>	Copy text inside parentheses
<code>yap</code>	Copy paragraph
<code>yy</code>	Copy line
<code>y\$</code>	Copy from cursor to end of line

Selecting Text

Combo	What it does
<code>viw</code>	Select word
<code>vi"</code>	Select inside quotes
<code>vi(</code>	Select inside parentheses
<code>vip</code>	Select paragraph
<code>V5j</code>	Select 5 lines down
<code>ggVG</code>	Select entire file

Quick Transformations

Combo	What it does
<code>gUiw</code>	Uppercase word
<code>guiw</code>	Lowercase word
<code>~w</code>	Toggle case of word
<code>>></code>	Indent line
<code><<</code>	Deindent line
<code>==</code>	Auto-indent line
<code>gg=G</code>	Auto-indent entire file
<code>gcc</code>	Comment/uncomment line
<code>gcip</code>	Comment/uncomment paragraph
<code>saiw"</code>	Surround word with <code>"</code>
<code>sd"</code>	Remove surrounding <code>"</code>
<code>sr"'</code>	Replace <code>"</code> with <code>'</code> around current text
<code>J</code>	Join current line with next

The Most Powerful Patterns

Pattern	How it works
<code>* then ciw then n.n.n.</code>	Find-and-replace one at a time with full control
<code>Qa ... q then @a</code>	Record and replay any sequence of actions
<code>V select then :norm @a</code>	Run a macro on selected lines
<code>:g/pattern/command</code>	Run a command on every matching line
<code>:grep "text" then :cfd ...</code>	Project-wide search and replace (the substitute + update recipe is in section 67)
<code><Space>rn</code>	Intelligent rename across project
<code>qf list + :cnext / :cprev</code>	Jump through search results or errors
<code>.</code>	Repeat last change (combine with <code>n</code> for find-and-repeat)

Chapter 5: Completion and snippets

14. Autocompletion (`nvim-cmp`)

Plugin: `nvim-cmp`. Sources: LSP, UltiSnips snippets, file paths, buffer words.

Keymap	Description
<code><Tab></code>	If menu is open: select next item. Otherwise: normal tab.
<code><CR></code> (Enter)	Confirm an item you picked with <code><Tab></code> / <code><Ctrl-n></code> ; if nothing is selected it is a plain newline
<code><Ctrl-e></code>	Dismiss / close the completion menu
<code><Esc></code>	Close the completion menu
<code><Ctrl-d></code>	Scroll documentation popup up
<code><Ctrl-f></code>	Scroll documentation popup down
<code><Ctrl-n></code> / <code><Ctrl-p></code>	Open the menu / select the next / previous item (inserts it)
<code><Ctrl-y></code>	Confirm the selected item
<code><Down></code> / <code><Up></code>	Select the next / previous item without inserting it
<code><Tab></code> / <code><S-Tab></code>	In the <code>:</code> and <code>/</code> command line: open / move through the completion menu

Completion labels are coloured like code (`colorful-menu.nvim`); match and kind colours follow the active colorscheme.

15. Snippets (`UltiSnips`)

Plugin: `UltiSnips` + `vim-snippets`. Custom snippets in `my_snippets/` directory.

Keymap	Description
<code><Ctrl-j></code>	Expand snippet / jump to next placeholder
<code><Ctrl-k></code>	Jump to previous placeholder

Available snippet files: `all`, `cpp`, `java`, `markdown`, `nix`, `python`, `snippets`, `tex`, `vim`

Java Snippets

Trigger	Expansion
<code>fdjscanner</code>	Java Scanner input template
<code>jarr</code> / <code>jarrlit</code>	Array / array with literal values
<code>jdict</code> / <code>jdictfull</code>	HashMap / HashMap with import
<code>jfor</code> / <code>jforeach</code>	For loop / enhanced for loop
<code>jwhile</code> / <code>jdowhile</code>	While / do-while loop
<code>jif</code> / <code>jifelse</code> / <code>jifelif</code>	If / if-else / if-else if-else
<code>jswitchtraditional</code> / <code>jswitchmulti</code> / <code>jswitcharrow</code> / <code>jswitcharrowmulti</code> / <code>jswitchyield</code> / <code>jswitchyieldblock</code>	Switch variants
<code>jtrycatch</code> / <code>jtryfinally</code>	Try-catch / try-catch-finally
<code>jwhilescannerbreak</code>	While loop with Scanner and break condition

Other Snippets

File	Triggers
all	<code>arw</code> (right arrow), <code>ltx</code> (LaTeX symbol)
cpp	<code>bare</code> (barebone template), <code>icd</code> (<code>#include</code>), <code>incvec</code> <code>incmap</code> <code>incset</code> <code>incqueue</code> <code>incstr</code> <code>incstack</code> (include that header), <code>vec</code> <code>map</code> <code>umap</code> <code>set</code> <code>uset</code> <code>queue</code> <code>stack</code> (std containers), <code>cout</code> , <code>plist</code> (print vector), <code>pmap</code> (print list of lists), <code>pqueue</code> (print queue), <code>random</code> (random list), <code>sol</code> (solution), <code>for</code> , <code>if</code> , <code>ifelse</code>
markdown	<code>meta</code> (YAML front matter), <code>h1</code> ... <code>h6</code> (header), <code>link</code> , <code>rlink</code> (reference link), <code>img</code> , <code>font</code> , <code>more</code> , <code>detail</code> (clickable details), <code>k1</code> / <code>kbd</code> , <code>k2</code> , <code>k3</code> (keyboard keys), <code>info</code> <code>warn</code> <code>error</code> <code>success</code> (boxes), <code>td</code> (too long, did not read), <code>yh</code> (corner quotes)
nix	<code>homepackages</code> , <code>systempackages</code> , <code>excludepackages</code> , <code>delibheaderhome</code> , <code>delibheadersystem</code> , <code>delibheaderhomealways</code> , <code>delibheadersystemalways</code> , <code>let</code> , <code>mkshell</code> , <code>mkderiv</code> , <code>flake</code> , <code>homefile</code> , <code>fetchgit</code> , <code>systemd</code>
python	<code>print</code> , <code>impa</code> (import as), <code>main</code> (main boilerplate), <code>sol</code> (solution)
snippets	<code>snip</code> (UltiSnips snippet definition)
tex	<code>use</code> (<code>\usepackage{}</code>), <code>eqa</code> (equation environment)
vim	<code>fun</code> (function), <code>aug</code> (augroup)

45. Autocompletion In Depth

How It Works

When you type in insert mode, **nvim-cmp** queries multiple sources and shows a popup menu with suggestions:

1. **LSP** (highest priority): Function names, variables, methods, types from the language server
2. **UltiSnips**: Snippet triggers (e.g., type `jfor` in a Java file)
3. **Path**: File paths when you start typing a path
4. **Buffer** (lowest priority, min 2 chars): Words already in the current buffer

For LaTeX files the sources are **omni** (BibTeX/citations), the texlab LSP (only when `texlab` is on PATH, LaTeX devShell), UltiSnips, buffer and path.

The Smart Tab Behavior

`<Tab>` has two behaviors depending on context:

1. **Completion menu is visible:** Selects the next item in the menu
2. **Otherwise:** Inserts a normal tab character

Completion Keymaps

Keymap	In completion menu	Outside menu
<code><Tab></code>	Select next item	Insert tab
<code><CR></code> (Enter)	Confirm the item you picked (with nothing picked: newline)	Insert newline
<code><Ctrl-e></code>	Close menu	Go to end of line
<code><Esc></code>	Close menu	Exit insert mode
<code><Ctrl-d></code>	Scroll docs up	Delete the character right of the cursor
<code><Ctrl-f></code>	Scroll docs down	(nothing)

Visual Indicators

- Each completion item shows an icon indicating its kind (function, variable, method, keyword, etc.) via `mini.icons`
- Deprecated items appear with strikethrough
- The completion menu is semi-transparent (5% blend)

52. Snippets for Developers

What Snippets Are

Plugin: **UltiSnips** + **vim-snippets**. Snippets are templates that expand into boilerplate code when you type a trigger word.

How to Use Snippets

1. In insert mode, type a trigger word (e.g., `jfor` in a Java file)
2. The trigger appears in the completion menu as a snippet
3. Press `<Ctrl-j>` to expand it
4. The snippet expands with **placeholders** (highlighted fields you need to fill in)
5. Press `<Ctrl-j>` to jump to the next placeholder
6. Press `<Ctrl-k>` to jump to the previous placeholder
7. Fill in each placeholder, and you're done

Custom Snippets

Custom snippets live in the `my_snippets/` directory. Each file targets a specific language:

File	Language	Notable snippets
<code>all.snippets</code>	All filetypes	General-purpose snippets
<code>java.snippets</code>	Java	Scanner, arrays, loops, conditionals, switch, try-catch (see full list in Snippets section)
<code>python.snippets</code>	Python	Python-specific patterns
<code>cpp.snippets</code>	C++	C++ templates
<code>nix.snippets</code>	Nix	Nix language patterns
<code>tex.snippets</code>	LaTeX	LaTeX environments and commands
<code>markdown.snippets</code>	Markdown	Markdown structures
<code>vim.snippets</code>	Vimscript	Vim plugin development
<code>snippets.snippets</code>	Snippet files	<code>snip</code> : template for a new snippet definition

Creating Your Own Snippets

Edit the appropriate file in `my_snippets/` (e.g., `my_snippets/python.snippets`):

```
snippet trigger "Description" b
def ${1:function_name}(${2:args}):
    ${3:pass}
endsnippet
```

- `trigger` is what you type
- `b` means it only triggers at the beginning of a line
- `${1}`, `${2}`, `${3}` are tab-stop placeholders (jump between them with `<Ctrl-j>`)

Chapter 6: Searching, replacing, files and the file tree

10. Searching, Replacing, and Refactoring Text

This is one of the most important sections in the guide. It covers searching within a file, replacing text with various levels of control, and doing all of this across the entire project.

Searching in the Current File

Keymap	Description
<code>/pattern</code>	Search forward for <code>pattern</code> . Press <code><Enter></code> to start the search.
<code>?pattern</code>	Search backward for <code>pattern</code>
<code>n</code>	Jump to the next match (with <code>hlslens</code> showing <code>[x/y]</code> count)
<code>N</code>	Jump to the previous match
<code>*</code>	Search forward for the exact word under cursor (whole word, literal; the cursor stays on the word, <code>3*</code> jumps to the 3rd match after the cursor)
<code>#</code>	Search backward for the exact word under cursor
<code>;noh<Enter></code> (<code>;</code> is <code>:</code> here)	Clear the yellow search highlight (tested). The search itself is kept, so <code>n</code> / <code>N</code> still work; the highlight comes back on the next search or <code>*</code> .

Search Modifiers

Modifier	Where to put it	What it does	Example
<code>\c</code>	Anywhere in pattern	Force case-insensitive	<code>/\chello</code> finds <code>Hello</code> , <code>HELLO</code> , <code>hello</code>
<code>\C</code>	Anywhere in pattern	Force case-sensitive	<code>/\Chello</code> only finds <code>hello</code>
<code>\v</code>	At start of pattern	Very magic: regex works like Perl/Python (no need to escape <code>()</code> , <code> </code> , <code>+</code> , etc.)	<code>/\vfunction\(.*\)</code>
<code>\<</code> and <code>\></code>	Around pattern	Whole word match only	<code>/\<count\></code> finds <code>count</code> but not <code>counter</code>

By default, search is case-insensitive but becomes case-sensitive if you type any uppercase letter (smart case). This is for `/` and `?` only: while you type a `:` command, smart case is off, so `:s` and `:g` ignore case completely (use `\c` or the `I` flag of `:s` for an exact match).

Search Examples

Search	What it finds
<code>/hello</code>	hello, Hello, HELLO (smart case: all lowercase = case-insensitive)
<code>/Hello</code>	Only Hello (smart case: has uppercase = case-sensitive)
<code>/\vdef \w+\(</code>	All Python function definitions (very magic regex)
<code>/\v(TODO FIXME HACK)</code>	Any of these three words (very magic <code> </code> for alternation)
<code>/\<user\></code>	Only the word <code>user</code> , not <code>username</code> or <code>superuser</code>
<code>/error\c</code>	error, Error, ERROR (forced case-insensitive)

Substitution (Find & Replace in Current File)

The substitute command has this structure: `: [range]s/old/new/[flags]`

Understanding the Range (Where to Replace)

The range tells Vim which lines to search. If omitted, only the current line is affected.

Range	Meaning	Example
(none)	Current line only	<code>:s/old/new/</code>
<code>%</code>	Entire file (all lines)	<code>:%s/old/new/g</code>
<code>.</code>	Current line (same as no range)	<code>:.s/old/new/g</code>
<code>\$</code>	Last line of file	
<code>., \$</code>	From current line to end of file	<code>., \$s/old/new/g</code>
<code>1, .</code>	From first line to current line	<code>:1, .s/old/new/g</code>
<code>20, 30</code>	From line 20 to line 30 (absolute)	<code>:20, 30s/old/new/g</code>
<code>-3, +3</code>	From 3 lines above to 3 lines below cursor (relative)	<code>:-3, +3s/old/new/g</code>
<code>'<, '></code>	Current visual selection (auto-filled when you press <code>:</code> in visual mode)	<code>:'<, '>s/old/new/g</code>

Understanding the Flags (How to Replace)

Flags go at the very end, after the last `/`.

Flag	What it does
(none)	Replace only the first occurrence on each line in range
<code>g</code>	Global : replace all occurrences on each line (not just the first)
<code>c</code>	Confirm : ask <code>y/n</code> for each replacement. You see the match highlighted and choose.
<code>i</code>	Case- insensitive matching
<code>I</code>	Case- sensitive matching. Needed here: <code>:s</code> ignores case completely, even with capitals in the pattern (tested: <code>:%s/old/X/g</code> on <code>old old OLD</code> gave <code>x x x</code> , with <code>gI</code> only <code>old</code> changed)
<code>n</code>	Count only : show how many matches there are without replacing anything
<code>e</code>	Suppress "pattern not found" error

Flag Combinations

Command	What happens
<code>:%s/old/new/</code>	Replace the first <code>old</code> on each line in the file
<code>:%s/old/new/g</code>	Replace every <code>old</code> in the entire file
<code>:%s/old/new/gc</code>	Replace every <code>old</code> , but ask confirmation for each one
<code>:%s/old/new/gi</code>	Replace every <code>old</code> case-insensitively (<code>old</code> , <code>OLD</code> , <code>oLd</code> all match)
<code>:%s/old/new/gn</code>	Count how many <code>old</code> exist in the file (no replacement)
<code>:%s/old/new/gce</code>	Confirm each, and don't error if not found

Quick difference between the three common endings (same `:%s/old/new` start, only the end changes):

Ending	Replaces	Asks you?
<code>/</code> (no flag)	only the first <code>old</code> on each line	no
<code>/gc</code>	every <code>old</code> on each line	yes, <code>y/n</code> for each one
<code>/gcc</code>	every <code>old</code> on each line (tested: the second <code>c</code> switches confirmation off again, so <code>/gcc</code> behaves like <code>/g ; /gccc</code> asks again)	no

Do not confuse `/gcc` with the normal-mode `gcc` (toggle comment on the current line): inside `:s/.../.../` the letters are flags, outside it `gcc` is a command.

Confirmation Mode (`c` Flag) Controls

When you use the `c` flag, Vim highlights each match and asks what to do:

Key	Action
<code>y</code>	Yes , replace this one and move to next
<code>n</code>	No , skip this one and move to next
<code>a</code>	All : replace this one and all remaining (stop asking)
<code>q</code>	Quit : stop replacing now
<code>l</code>	Last : replace this one and then stop
<code><Ctrl-e></code>	Scroll down to see more context
<code><Ctrl-y></code>	Scroll up to see more context

Changing the Delimiter

If your search/replace text contains `/`, use a different delimiter to avoid confusion:

Command	What it does
<code>:%s#/usr/local/bin#/opt/bin#g</code>	Replace path using <code>#</code> as delimiter
<code>:%s old new g</code>	Use <code> </code> as delimiter

You can use almost any character as a delimiter. Just use the same character for all three separators.

Special Replacement Patterns

Pattern in replacement	What it means
<code>&</code>	The entire matched text
<code>\1</code> , <code>\2</code> , etc.	Capture group 1, 2, etc. from <code>\(\)</code> in the search
<code>\u</code>	Uppercase the next character
<code>\U</code>	Uppercase everything after this
<code>\l</code>	Lowercase the next character
<code>\L</code>	Lowercase everything after this
<code>\r</code>	Newline (line break)

Substitution Examples

Command	What it does
<code>:%s/foo/bar/g</code>	Replace all <code>foo</code> with <code>bar</code>
<code>:%s/<foo>/bar/g</code>	Replace only whole-word <code>foo</code> (not <code>foobar</code>)
<code>:%s/foo/bar/gc</code>	Replace all, confirming each one
<code>:%s/foo//g</code>	Delete all occurrences of <code>foo</code>
<code>:%s/\v(\w+), (\w+)/\2, \1/g</code>	Swap two comma-separated words: <code>last, first</code> becomes <code>first, last</code>
<code>:%s/<(\w\w)/\u\1/g</code>	Capitalize the first letter of every word
<code>:%s/\$/;/</code>	Add a semicolon at the end of every line
<code>:%s/^\s*\$\n//g</code>	Delete all blank lines
<code>:20,30s/TODO/DONE/g</code>	Replace only between lines 20-30
<code>: '<, '>s/old/new/g</code>	Replace only in the visual selection

Searching in the Current Buffer with `*` and `#`

These are the fastest ways to search for a word:

1. Place cursor on any word
2. Press `*` - all occurrences highlight, the hlsens overlay shows `[1/N]`
3. Press `n` to jump forward, `N` to jump backward
4. With no count the cursor stays on the word you pressed `*` on (custom behavior in this config: a whole-word, literal search; `3*` jumps to the 3rd match after the cursor instead)

This is often combined with `ciw` + `.` for selective replacement (see below).

Using `ciw` + `.` for Selective Single-File Replacement

This is the **most practical replacement method** for everyday use. It gives you full control, replacing one occurrence at a time:

1. Place cursor on the word you want to replace (e.g., `oldName`)
2. `*` - search for it (all occurrences highlight)
3. `ciw` - delete the word and enter insert mode

4. Type the new word (e.g., `newName`), then press `<Esc>`
5. `n` – jump to the next occurrence
6. Decide: press `.` to replace this one too, or `n` to skip it
7. Repeat step 5-6 until done

Why this is great: Unlike `:%s`, you see each occurrence in context and can decide whether to replace it. Unlike `:%s/old/new/gc`, you stay in normal mode between replacements and can scroll around.

67. Multi-File Search and Replace (Complete Guide)

This is the section you need when you want to find or replace text across your entire project – not just the current file.

Quick Decision Guide: Which Method to Use

Scenario	Best method
Rename a function/variable/class (code-aware)	LSP Rename (<code><Space>rn</code>)
Replace a plain string in many files	<code>:grep</code> + <code>:cfd</code>
Replace only in certain file types (e.g., only <code>.py</code>)	<code>:grep --type</code> or <code>:vimgrep</code> , then <code>:cfd</code>
Just find where something is used (no replace)	<code><Space>fg</code> (live grep)
Replace with confirmation for each occurrence	<code>:cfd</code> with the <code>gc</code> flag

Method 1: LSP Rename (Best for Code Symbols)

If you're renaming a function, variable, class, or any code symbol, this is the best method because it understands scope and language semantics.

1. Place cursor on the symbol you want to rename
2. Press `<Space>rn`
3. Type the new name
4. Press `<Enter>`

What happens: The LSP server finds every reference to that symbol across the entire project and renames them all. It's smart: renaming `count` in one function won't affect `count` in another function.

Limitations: Only works for code symbols (not arbitrary text), and requires an LSP server that supports rename.

Method 2: `:grep` + `:cfd` (Best for Plain Text)

This is the most versatile method. It uses `ripgrep` (very fast) to search the entire project, puts results in the quickfix list, then runs a command on each file of the list.

Why `:cfd` and not `:cdo`: `:grep` here creates ONE quickfix entry per match. `:cdo s/x/y/g` visits a line once per match; after the first visit replaced every `x` on the line, the next visit finds nothing and stops with `E486: Pattern not found` (tested), so the rest is NOT replaced. `:cfd %s/x/y/g` runs once per file and avoids this. If you prefer `:cdo`, add the `e` flag: `:cdo s/x/y/ge | update`.

Step-by-Step: Replace All Without Confirmation

```
:grep "oldFunction"          -- search entire project
:cfdo %s/oldFunction/newFunction/g | update  -- replace in every matching file and save it
```

Step-by-Step: Replace with Confirmation for Each Occurrence

```
:grep "oldFunction"          -- search entire project
:cfdo %s/oldFunction/newFunction/gc | update  -- 'c' flag asks y/n for EACH occurrence
```

When the `c` flag is active, for each match you see it highlighted and can press:

- `y` to replace this one
- `n` to skip this one
- `a` to replace all remaining in this file (then moves to the next file, where you are prompted again; tested)
- `q` to stop entirely

Step-by-Step: Review Results Before Replacing

```
:grep "oldFunction"          -- search entire project
:copen                       -- open the quickfix window to review all results
```

Now you can see every file and line that matches. Use `:cnext / :cprev` (or `j / k` in the quickfix window then `<Enter>`) to jump through them. Once satisfied:

```
:cfdo %s/oldFunction/newFunction/g | update  -- replace and save
```

Using Regex with `:grep`

`:grep` passes the pattern directly to ripgrep, so you can use ripgrep regex. It is smart-case: an all-lowercase pattern ignores case, a pattern with a capital is exact (add `-s` to force exact case):

Command	What it finds
<code>:grep "TODO"</code>	All lines containing <code>TODO</code>
<code>:grep "TODO FIXME"</code>	Lines with <code>TODO</code> or <code>FIXME</code> (type <code> </code> with the backslash; a rendered table may hide it)
<code>:grep "\buser\b"</code>	Only the whole word <code>user</code>
<code>:grep "def \w+\("</code>	Python function definitions
<code>:grep "console\.log"</code>	All <code>console.log</code> calls

Limiting to Specific File Types

Ripgrep supports file type filters:

Command	What it searches
<code>:grep "pattern" --type py</code>	Only Python files
<code>:grep "pattern" --type js</code>	Only JavaScript files
<code>:grep "pattern" --type java</code>	Only Java files
<code>:grep "pattern" src/</code>	Only files in the <code>src/</code> directory
<code>:grep "pattern" --glob "*.tsx"</code>	Only <code>.tsx</code> files

Method 3: `:vimgrep` + `:cfd` (Built-in, Slower but Portable)

`:vimgrep` is Vim's built-in search (doesn't require ripgrep). It's slower but lets you use Vim regex and file glob patterns:

Command	What it does
<code>:vimgrep /pattern/ **/*</code>	Search all files recursively
<code>:vimgrep /pattern/ **/*.py</code>	Search only Python files
<code>:vimgrep /pattern/ **/*.{js,ts}</code>	Search JS and TS files
<code>:vimgrep /pattern/ src/**/*</code>	Search only in <code>src/</code> directory

Then use `:cfd` as before:

```
:vimgrep /oldName/ **/*.java
:cfd %s/oldName/newName/g | update
```

Method 4: `<Space>fg` for Finding (No Replace)

`<Space>fg` (live grep via fzf-lua) is the fastest way to **find** where something is used, but it doesn't directly support replace. Use it for:

- Exploring: "Where is this function called?"
- Investigating: "Which files reference this config key?"
- Planning: "How many places use this pattern?" before deciding on a replace strategy

After reviewing results in fzf, you can then use `:grep` + `:cfd` for the actual replacement.

Complete Examples

Example 1: Rename an API Endpoint Across the Project

You renamed `/api/users` to `/api/v2/users`:

```
:grep "/api/users"          -- find all references
:copen                     -- review: make sure you're not catching wrong
things
:cfd %s#/api/users#/api/v2/users#g | update -- replace (using # as delimiter since / is
in the text) and save
```

Example 2: Replace a Deprecated Function Name (with Confirmation)

```
:grep "getUser"  
:cfdo %s/getUser/fetchUser/gc | update -- confirm each one ('y' to replace, 'n' to skip)
```

Example 3: Delete All Console.log Statements in JavaScript Files

```
:grep "console\.log" --type js -- find them  
:cfdo g/console\.log/d -- delete every line containing the match  
:cfdo update -- save the files
```

Example 4: Add a Comment Before Every TODO

```
:grep "TODO"  
:cfdo %s/TODO/NOTE: was TODO/g | update
```

Example 5: Replace Only in Python Files in the src/ Directory

```
:grep "old_function" --type py src/  
:cfdo %s/old_function/new_function/g | update
```

Example 6: Case-Insensitive Project-Wide Replace

```
:grep -i "oldname" -- ripgrep's -i flag for case-insensitive  
:cfdo %s/oldname/newname/g | update -- :s already ignores case in this config
```

Understanding :cdo vs :cfdo vs :bufdo

Command	What it does
:cdo {cmd}	Run {cmd} on every line in the quickfix list (may visit the same file multiple times)
:cfdo {cmd}	Run {cmd} once per file in the quickfix list (visits each file only once)
:bufdo {cmd}	Run {cmd} on every open buffer (not just quickfix results)

For search-and-replace use :cfdo %s/old/new/g (see Method 2 for why a plain :cdo s/old/new/g can stop early). :cdo is fine for commands that act on the entry's line once, or with the e flag.

Undoing a Multi-File Replace

If the replace went wrong, each file has its own undo history:

1. :cfdo undo – undo the last change in every affected file
2. :cfdo update – save the reverted files

Or use :cfdo earlier lf to go back one save-state in each file (both recipes tested: both files were restored exactly).

12. Fuzzy Finding & Project-Wide Search (fzf-lua)

Plugin: **fzf-lua**. A powerful popup interface that connects to FZF (a command-line fuzzy finder). It lets you search file names, search text inside files, browse buffers, and more. The popup opens centered on screen at 70% height.

Moving Inside Any Picker (Lists With a Search Bar)

Several things look the same: a search bar on top and a filtered list below it. They are the fzf-lua pickers (`<Space>ff`, `<Space>fg`, `<Space>fb`, `<Space>fr`, `<Space>gbl` ...), the snacks pickers (the branch menu you get by clicking the branch in the statusline, the code-action menu `<Space>ca`, every other `vim.ui.select` list) and Telescope (`:Telescope`, `<Space>db`, the devdocs commands). The search bar takes your typing, so `j` and `k` type the letters `j` and `k` there. Move through the list with these keys (all tested in a real terminal):

Keys	What it does
<code><Ctrl-n></code> / <code><Ctrl-p></code> , or <code><Down></code> / <code><Up></code>	Next / previous item. Works in all three kinds of picker.
<code><Ctrl-j></code> / <code><Ctrl-k></code>	The same, but only in fzf-lua and the snacks pickers (NOT in Telescope)
<code><Tab></code>	Move down. In fzf-lua and Telescope it also marks the item, so you can choose several at once
<code><Enter></code>	Choose the item (or the marked ones)
<code><Esc></code>	fzf-lua: close the picker. snacks and Telescope: the first <code><Esc></code> leaves the search bar for Normal mode, where <code>j</code> / <code>k</code> move the list; a second <code><Esc></code> closes it

The same `<Ctrl-n>` / `<Ctrl-p>` also move through the completion menu (section 14); on the `:` command line use `<Tab>` / `<S-Tab>`.

Keymaps

Keymap	Description
<code><Space>ff</code>	Find files: search file names in the project
<code><Space>fg</code>	Live grep: search text content across all files in the project
<code><Space>fh</code>	Search Neovim help tags
<code><Space>ft</code>	Search tags (functions, classes) in the current buffer (needs <code>ctags</code> ; the Nix nvim wrapper provides universal-ctags)
<code><Space>fb</code>	Search currently open buffers
<code><Space>fr</code>	Search recently opened files
<code><Space>gbl</code>	Fuzzy-search git branches (<code><Enter></code> checks the branch out)
<code><Space>gB</code>	Branch menu (the same menu as clicking the branch name in the statusline): choose a branch and Neovim switches to it. Move with <code><Ctrl-n></code> / <code><Ctrl-p></code> , see "Moving Inside Any Picker"

`<Space>ff` has no preview window and shows git status icons next to modified/untracked files; `.gitignore` is respected.

Inside the FZF Popup

Key	What it does
Type text	Filters results in real-time
<Enter>	Open the selected result
<Esc>	Cancel and close the popup
<Ctrl-j> / <Ctrl-k>	Move down / up in the results list
<Ctrl-n> / <Ctrl-p>	Move down / up (alternative keys)

<Space>fg - Live Grep (Project-Wide Text Search) In Depth

This is one of the most important keymaps for developers. It searches inside every file in your project directory using **ripgrep** (`rg`) under the hood.

What It Does

1. Press `<Space>fg`
2. A popup appears with a search prompt
3. As you type, ripgrep searches **all files** in the project folder and shows matching lines in real-time
4. Results show: file path, line number, and the matching line
5. Press `<Enter>` to jump directly to that file and line

Plain Text Search

Just type normal text. For example, typing `getUserById` finds every file and line containing that string.

Regex Search

Live grep supports **full regex** (ripgrep regex syntax). You don't need to learn all of regex, but here are the most useful patterns:

Pattern you type	What it finds	Example matches
<code>TODO</code>	Literal text <code>TODO</code>	<code>// TODO: fix this</code>
<code>TODO FIXME</code>	<code>TODO</code> OR <code>FIXME</code>	Both <code>// TODO</code> and <code>// FIXME</code>
<code>def \w+\(</code>	Python function definitions	<code>def process_data(</code> , <code>def main(</code>
<code>class \w+</code>	Class declarations	<code>class UserService</code> , <code>class App</code>
<code>import.*from</code>	ES6-style imports	<code>import { foo } from 'bar'</code>
<code>console\.log</code>	<code>console.log</code> (dot is escaped)	<code>console.log("debug")</code>
<code>function\s+\w+</code>	JavaScript function declarations	<code>function handleClick</code>
<code>\berror\b</code>	Whole word <code>error</code> only	<code>error</code> but not <code>errors</code> or <code>errorHandler</code>
<code>https?://</code>	URLs (http or https)	<code>https://example.com</code>
<code>v[0-9]+\.[0-9]+</code>	Version strings	<code>v1.0</code> , <code>v2.13</code>

Use Cases for Live Grep

Scenario	What to search
Find where a function is called	Type the function name
Find all TODOs	Type <code>TODO</code>
Find a specific error message	Type part of the error string
Find all API endpoints	Type <code>@GetMapping</code> (Java) or <code>app.get()</code> (Express) or <code>@app.route</code> (Flask)
Find all imports of a module	Type <code>import.*moduleName</code> (regex)
Find environment variable usage	Type <code>process.env</code> or <code>os.environ</code>
Find hardcoded strings	Type the string in quotes

`<Space>ff` - Find Files (File Name Search)

Searches **file names** (not content). Useful when you know the file you want but not the exact path.

- Type `userserv` to find `UserService.java` (fuzzy matching)
- Type `config.py` to find configuration files
- Type `.env` to find environment files (only if they are not git-ignored: `.gitignore` is respected; a `.ignore` file containing `!.env` makes them visible)
- Type `test` to see all test files

The Difference Between Search Methods

Method	Keymap	What it searches	Best for
Live grep	<code><Space>fg</code>	Text inside files across the entire project	Finding where code/text is used
Find files	<code><Space>ff</code>	File names in the project	Opening a file by name
Buffer search	<code><Space>fb</code>	Names of currently open files	Switching between open files
Recent files	<code><Space>fr</code>	Files you recently edited	Returning to a file you had open earlier
Buffer tags	<code><Space>ft</code>	Functions/classes in current file (needs <code>ctags</code>)	Jumping to a function in the current file
In-file search	<code>/pattern</code>	Text in current file only	Finding something in the file you're editing
Word under cursor	<code>*</code>	Current word in current file	Quick highlight and jump to next occurrence

26. Quickfix & Location List

Commands

Command	Description
<code>:copen</code> / <code>:cclose</code>	Open / close quickfix window
<code>:cnext</code> / <code>:cprev</code>	Next / previous item
<code>:cfirst</code> / <code>:clast</code>	First / last item
<code>:cc [nr]</code>	Jump to specific entry
<code>:cdo {cmd}</code>	Run command for each entry
<code>:colder</code> / <code>:cnewer</code>	Navigate quickfix history
<code>:lopen</code> / <code>:lclose</code>	Location list (per-window)
<code>\x</code>	Close quickfix and location list windows

Inside the Quickfix Window (`nvim-bqf`, `quicker.nvim`)

`quicker.nvim` formats the list (grouped by file, file-name column at most 40 characters or half the screen); the list cannot be edited as a buffer. `nvim-bqf` adds these keys inside the quickfix window (the preview does not start by itself):

Key	Description
<code>p</code> / <code>P</code>	Toggle the preview of the item / toggle auto-preview while moving
<code>zp</code>	Toggle the preview between normal and maximum size
<code><Ctrl-f></code> / <code><Ctrl-b></code>	Scroll the preview down / up
<code><Tab></code> / <code><S-Tab></code>	Mark the item and move down / up; <code>z<Tab></code> clears the marks
<code>zn</code> / <code>zN</code>	New list from the marked / unmarked items
<code>zf</code>	Fuzzy filter the list (fzf)
<code><</code> / <code>></code>	Go to the older / newer quickfix list
<code>o</code>	Open the item and close the quickfix window
<code>O</code>	Open the item in a window that already shows the buffer (otherwise open it); the list stays open
<code>t</code> / <code>T</code>	Open in a new tab (<code>T</code> stays in the quickfix window)
<code><Ctrl-x></code> / <code><Ctrl-v></code> / <code><Ctrl-t></code>	Open in a horizontal split / vertical split / new tab

Trouble (Plugin)

Keymap / Command	Description
<code>:Trouble</code>	Open Trouble diagnostics viewer
<code><Space>dw</code>	Workspace diagnostics via Trouble

51. Quickfix Workflows for Developers

The quickfix list is a central tool for developers. It's a list of locations (file + line number) that you can jump through. Many features populate it.

What Populates the Quickfix List

Source	How to populate	Description
Project-wide search	<code>:vimgrep /pattern/ **/*</code>	Search results across all files
LSP diagnostics	<code><Space>qw</code>	Diagnostics of all open buffers
Buffer diagnostics	<code><Space>qb</code>	Errors/warnings in current file only
Build errors	<code>:make</code>	Compiler output
Grep	<code>:grep pattern</code>	Uses ripgrep (configured in this setup)

Navigating the Quickfix List

Command / Keymap	What it does
<code>:copen</code>	Open the quickfix window at the bottom
<code>:cclose</code> or <code>\x</code>	Close the quickfix window (<code>\x</code> also closes all location lists)
<code>:cnext</code>	Jump to the next item
<code>:cprev</code>	Jump to the previous item
<code>:cfirst</code> / <code>:clast</code>	Jump to the first / last item
<code>:cc 5</code>	Jump to item number 5
<code>:colder</code> / <code>:cnewer</code>	Go to the previous / next quickfix list (history)

Batch Operations on Quickfix Items

Command	What it does
<code>:cfdo %s/old/new/g update</code>	Run a substitution once in every file of the quickfix list, then save it
<code>:cdo s/old/new/ge update</code>	The same per quickfix entry (the <code>e</code> flag is needed, see the Multi-File Search and Replace section)

Example workflow: Rename a string across the project:

1. `:grep "oldName"` to populate quickfix with all occurrences
2. `:cfdo %s/oldName/newName/g | update` to replace in all files and save them

Trouble (Better Quickfix UI)

Plugin: **trouble.nvim**. A nicer interface for browsing diagnostics and quickfix items.

Keymap / Command	What it does
<code><Space>dw</code>	Open Trouble with workspace diagnostics
<code>:Trouble</code>	Open Trouble window

Trouble shows diagnostics grouped by file with icons and colors, making it easier to triage errors.

11. File Explorer (nvim-tree)

Plugin: `nvim-tree.lua`. A sidebar file tree. It loads on the first `<Space>s` or the first `:NvimTreeToggle`, `:NvimTreeOpen`, `:NvimTreeFocus`, `:NvimTreeFindFile` OR `:NvimTreeFindFileToggle`; `nvim <dir>` and the dashboard entry open it too. Which folder it shows: `nvim` or `nvim .`

shows the folder you started in; `nvim <folder>` shows that folder; `nvim <folder>/file` shows the folder you started in (not the file's folder). From then on it keeps that folder until the working folder changes: after `:cd`, `:tcd`, `:Z` or `:z` it shows the new folder (also when it is open, or was closed and is opened again).

Keymap	Context	Description
<code><Space>s</code>	global	Toggle the file explorer on/off
<code><Enter></code>	in tree	Open file (cursor moves to file) / expand directory. With 2 or more editor windows open and the file not already shown in one of them, it asks "Pick window:": a letter (A, B, C, ...) is drawn in the middle of each window's status line, press the letter of the window you want; any other key cancels. A file that is already open in a window is just focused. The dimming plugin vimade is switched off only while the letters are shown (otherwise they were nearly invisible) and switched on again right after. The tree itself always opens at the far left of the screen.
<code><Tab></code>	in tree	Open file but keep cursor in the tree (great for opening multiple files)
<code><BS></code>	in tree	Close (collapse) the directory under the cursor; on a file it closes its parent directory
<code>a</code>	in tree	Create a new file. Type the name and press Enter. Add <code>/</code> at the end for a directory.
<code>d</code>	in tree	Delete file/directory (asks for confirmation)
<code>r</code>	in tree	Rename file/directory
<code>c</code>	in tree	Copy file to clipboard
<code>x</code>	in tree	Cut file to clipboard
<code>p</code>	in tree	Paste from clipboard
<code>q</code>	in tree	Close the file explorer
<code>D</code>	in tree	Move file/directory to the trash (asks for confirmation; needs <code>trash</code>)
<code>g?</code>	in tree	Show the help with every key
<code><Ctrl-v></code> / <code><Ctrl-x></code> / <code><Ctrl-t></code>	in tree	Open in a vertical split / horizontal split / new tab
<code>-</code> / <code><Ctrl-]></code>	in tree	Make the parent / the folder under the cursor the root
<code>H</code> / <code>I</code>	in tree	Toggle dotfiles / git-ignored files
<code>f</code> / <code>F</code>	in tree	Live filter: start / clear
<code>R</code>	in tree	Refresh
<code>y</code> / <code>Y</code> / <code>gy</code>	in tree	Copy the name / relative path / absolute path
<code>]c</code> / <code>[c</code> , <code>]e</code> / <code>[e</code>	in tree	Next / previous git item, diagnostic item
<code>E</code> / <code>w</code>	in tree	Expand all / collapse all

`nvim <dir>` opens the tree on that directory and makes it the working directory.

Moving between tree and code: Use `<Ctrl-w>h` / `<Ctrl-w>l` or `<Left>` / `<Right>` arrow keys.

30. Working with Directories

Keymap / Command	Description
<code><Space>cd</code>	Change working directory to current file's directory (window-local)
<code>:cd <path></code>	Change directory globally
<code>:lcd <path></code>	Change directory for current window only
<code>:tcd <path></code>	Change directory for current tab
<code>:pwd</code>	Print current working directory
<code>:Z {keywords}</code> (or <code>:z {keywords}</code>)	<code>cd</code> to the best zoxide match (e.g. <code>:z nix nixos</code>) and print <code>cd <dir></code> ; without arguments it opens the fuzzy zoxide directory picker; without zoxide installed one warning

Path Modifiers (for use in commands)

Modifier	Meaning	Example
<code>%</code>	Current file path	<code>/home/user/project/src/main.lua</code>
<code>:%:h</code>	Directory of current file	<code>/home/user/project/src</code>
<code>:%:t</code>	Filename only	<code>main.lua</code>
<code>:%:p</code>	Full absolute path	<code>/home/user/project/src/main.lua</code>

57. File Management for Developers

File Operations

Plugin / Feature	What it does
nvim-tree (<code><Space>s</code>)	Visual file browser. Create (<code>a</code>), delete (<code>d</code>), rename (<code>r</code>), copy (<code>c</code>), cut (<code>x</code>), paste (<code>p</code>).
vim-eunuch	Unix file commands, all available from a fresh start (lazy <code>cmd</code> list): <code>:Rename <newname></code> , <code>:Move <path></code> (move the file, creating directories), <code>:Duplicate <name></code> , <code>:Copy <path></code> , <code>:Delete</code> / <code>:Remove</code> / <code>:Unlink</code> (delete the file; <code>:Delete</code> also the buffer), <code>:Mkdir <dir></code> (always creates the missing parent folders, like <code>mkdir -p</code> ; do NOT type <code>-p</code> , it would become part of the folder name; a <code>!</code> makes no difference; with no argument it creates the folder of the current file; an existing folder only gives the message "Directory already exists"), <code>:Chmod <mode></code> , <code>:Cfind</code> / <code>:Lfind</code> / <code>:Clocate</code> / <code>:Llocate</code> (find / locate into the quickfix / location list), <code>:SudoEdit</code> , <code>:SudoWrite</code> , <code>:Wall</code> (write all) and <code>:W</code> (= <code>:Wall</code>).
gx.nvim (<code>gx</code>)	Open the URL or file path under cursor in a browser.
<code>:CopyPath absolute</code>	Copy the full file path to clipboard.
<code>:CopyPath relative</code>	Copy path relative to project root.
<code>:CopyPath nameonly</code>	Copy just the filename.

Project Structure Navigation

Keymap	What it does
<code><Space>s</code>	Toggle file tree sidebar
<code><Space>ff</code>	Fuzzy-find any file in the project
<code><Space>fg</code>	Search for text across all project files
<code><Space>cd</code>	Change the working directory of THIS window only (<code>:lcd</code>) to the current file's folder and print it
<code><Space>t</code>	Toggle the symbol outline (aerial) for the current file

63. Working with Multiple Files

Opening Several Files

Method	How
From command line	<code>nvim file1.py file2.py file3.py</code> (opens all as buffers)
From inside Neovim	<code><Space>ff</code> to find and open files one at a time
Split open	<code>:vs file2.py</code> opens file2 in a vertical split next to current file
Tab open	<code>:tabe file2.py</code> opens in a new tab
From file tree	<code><Space>s</code> , navigate to file, press <code><Tab></code> to open without leaving the tree

Comparing Two Files Side by Side

1. Open the first file

2. `:vs second_file.py` – open the second file in a vertical split
3. Now both files are visible side-by-side
4. Use `<Ctrl-w>h` / `<Ctrl-w>l` to switch between them
5. Use `:diffthis` in each window to enable diff mode (highlights differences)
6. `:diffoff` to turn diff off

Or use the diffview plugin: `:DiffviewOpen` for git diffs.

Copying Between Files

1. In file A: select text with `v` or `V`, then `y` to yank
2. Switch to file B: `<Ctrl-w>l` or `gb` or `<Space>bp`
3. Navigate to where you want the text
4. `p` to paste

Since clipboard is `unnamedplus`, yanked text is shared across all buffers and even with external applications.

Running the Same Edit Across Multiple Files

Use the quickfix list:

1. `:grep "TODO"` – find all files with "TODO"
2. `:cfd %s/TODO/DONE/g | update` – replace in every file of the list and save it

Closing Files You're Done With

Keymap	What it does
<code>\d</code>	Close current buffer, keep window
<code>\D</code>	Close all other buffers EXCEPT those with unsaved changes or a still-running terminal (one message says how many were kept)
<code><Space>q</code>	Save the buffer if modified, then close the window (last window: quits nvim)

Chapter 7: Windows, buffers, terminal, sessions and the interface

7. Windows, Splits, and Buffers

This section explains how to open, navigate, resize, and close split windows entirely with the keyboard.

Key Concepts

- **Buffer:** A file loaded into memory. You can have many buffers open but only see some of them.
- **Window:** A visible area showing a buffer. You can split your screen into multiple windows.
- **Tab:** A collection of windows. Think of it as a different workspace layout.

Panel layout: short single-task panels open on the LEFT (undo tree `<Space>u`, `<Space>rr` output, `:help` on a screen of at least 200 columns, which then opens as a full-height split on the far left), persistent panels on the RIGHT (Claude Code). Inactive windows are dimmed (vimade); the current window is always full colour.

Creating Splits

Keymap / Command	Description
<code><Ctrl-w>s</code> OR <code>:sp</code>	Split the current window horizontally (new window appears below)
<code><Ctrl-w>v</code> OR <code>:vs</code>	Split the current window vertically (new window appears to the right)
<code><Space>-</code>	Split the current window horizontally (new window below, same buffer)
<code><Space> </code>	Split the current window vertically (new window to the right, same buffer)
<code>:sp <file></code>	Open <code><file></code> in a new horizontal split
<code>:vs <file></code>	Open <code><file></code> in a new vertical split

Config note: `splitbelow` and `splitright` are set, so new splits always open below/right.

Navigating Between Windows

Keymap	Description
<code><Ctrl-w>h</code> OR <code><Left></code>	Move to the window on the left
<code><Ctrl-w>j</code> OR <code><Down></code>	Move to the window below
<code><Ctrl-w>k</code> OR <code><Up></code>	Move to the window above
<code><Ctrl-w>l</code> OR <code><Right></code>	Move to the window on the right
<code><Ctrl-w>w</code>	Cycle to the next window
<code><Ctrl-w>W</code>	Cycle to the previous window
<code><Ctrl-w>p</code>	Jump to the previously active window

Resizing Windows

Keymap	Description
<code><Ctrl-w>=</code>	Make all windows equal size
<code><Ctrl-w>+</code>	Increase current window height by 1 line
<code><Ctrl-w>-</code>	Decrease current window height by 1 line
<code><Ctrl-w>></code>	Increase current window width by 1 column
<code><Ctrl-w><</code>	Decrease current window width by 1 column
<code>10<Ctrl-w>+</code>	Increase height by 10 lines
<code>10<Ctrl-w>></code>	Increase width by 10 columns
<code><Ctrl-w>_</code>	Maximize current window height (make it as tall as possible)
<code><Ctrl-w> </code>	Maximize current window width (make it as wide as possible)
<code>:resize 20</code>	Set window height to 20 lines
<code>:vertical resize 80</code>	Set window width to 80 columns

Auto-resize: When you resize your terminal, all windows resize equally, except the Claude Code panel, which goes back to 30% of the screen width.

Moving Windows Around

Keymap	Description
<code><Ctrl-w>H</code>	Move current window to the far left (becomes full height)
<code><Ctrl-w>J</code>	Move current window to the very bottom (becomes full width)
<code><Ctrl-w>K</code>	Move current window to the very top (becomes full width)
<code><Ctrl-w>L</code>	Move current window to the far right (becomes full height)
<code><Ctrl-w>r</code>	Rotate windows in the current row/column
<code><Ctrl-w>R</code>	Rotate windows in reverse
<code><Ctrl-w>x</code>	Swap current window with the next one
<code><Ctrl-w>T</code>	Move current window to a new tab

Buffer Tabs (the Top Line)

The top line shows one tab per open buffer (bufferline). Click a tab to switch to it. The `x` at the right of a tab closes it; while that buffer has unsaved changes the `x` is replaced by a `•`, and clicking the `•` closes it too. For a changed file Vim then asks `Save changes to "name"? [Y]es, [N]o, [C]ancel`: Yes saves and closes, No discards and closes, Cancel keeps the buffer and shows the warning "unsaved changes, buffer kept". A terminal whose program is still running is kept ("running terminal, buffer kept"). A right-click opens Neovim's own menu (see the Mouse table in the cheat sheet).

Closing Windows

Keymap / Command	Description
<code><Space>q</code>	Close the current window (saves if modified, <code>:x</code>). It fires after a short pause because <code><Space>qb</code> / <code><Space>qw</code> also exist
<code><Space>Q</code>	Force quit Neovim, discarding unsaved changes , after a Yes/No confirmation (default No)
<code>:q</code>	Close current window
<code>:q!</code>	Close current window discarding unsaved changes
<code>:only</code> or <code><Ctrl-w>o</code>	Close ALL other windows, keep only the current one
<code>\x</code>	Close the quickfix window and all location lists of the tab (the cursor stays in the current window)

Buffer Management

Keymap / Command	Description
<code>gb</code>	Go to the next buffer; <code>{N}gb</code> (e.g. <code>3gb</code>) goes to buffer number N (an invalid number warns "Invalid bufnr")
<code>gB</code>	Go to the previous buffer. Do not give it a count: <code>{N}gB</code> does nothing (an invalid number warns "Invalid bufnr"); use <code>{N}gb</code> to jump to buffer N
<code><Space>bp</code>	Pick a buffer: each open buffer shows a letter, press it to switch
<code>\d</code>	Close/delete the current buffer (window stays open, shows previous buffer). On the last buffer an empty buffer is left. A named file with changes is saved first by auto-save (BufLeave); a buffer auto-save does not save (unnamed, read-only, Typst/LaTeX) is not deleted: you land in the previous buffer and the unsaved one stays loaded. On the only, unnamed buffer with typed text, Vim's confirm dialog "Save changes?" appears (the unsaved buffer is shown for a moment while it asks); your answer decides whether it is closed (tested in a real terminal).
<code>\D</code>	Close all other buffers, but keep buffers with unsaved changes and terminals that are still running (one message "kept N buffer(s) (unsaved or running terminal)")
<code><Ctrl-^></code>	Switch to the alternate buffer : the buffer you were in before this one. Press it again to come back, so you can flip between two files. It is the same as <code>:b#</code> or <code>:e #</code> . On a US keyboard <code>^</code> is Shift-6, so the keys are Ctrl-Shift-6; many terminals cannot send that, which is why Neovide gets the plain <code><Ctrl-6></code> as well (see section 77)
<code>:ls</code> or <code>:buffers</code>	List all open buffers
<code>:b <name></code>	Switch to a buffer by (partial) name
<code>:b 3</code>	Switch to buffer number 3

Tabs

Command	Description
<code>:tabnew</code>	Open a new empty tab
<code>:tabe <file></code>	Open <code><file></code> in a new tab
<code>gt</code>	Go to the next tab
<code>gT</code>	Go to the previous tab
<code>:tabclose</code> or <code>\t</code>	Close the current tab
<code>:tabonly</code> or <code>\T</code>	Close all other tabs

Closing Floating Windows

Some plugins open floating windows (diagnostics, hover docs, etc.):

Keymap	Description
<code><Esc></code>	Close any floating window (custom mapping)

8. Terminal Integration

Opening a Terminal

Command / Keymap	Description
<code>:term</code> or <code>:terminal</code>	Open terminal in the current window
<code>:sp term</code>	Open terminal in a horizontal split below
<code>:vs term</code>	Open terminal in a vertical split to the right
<code><Space>rr</code>	Run code (opens a terminal in a vertical split on the left of the code window)

The terminal automatically starts in insert mode (you can type immediately) and hides line numbers. (tested in a real terminal: both start in insert mode).

Navigating In and Out of Terminal

Keymap	Context	Description
<code><Esc></code>	In terminal	Exit terminal mode and enter Normal mode. Now you can navigate away from the terminal window using <code><Ctrl-w>h/j/k/l</code> or arrow keys. Exception: in the Claude Code panel <code><Esc></code> goes to Claude; use <code><Ctrl-\><Ctrl-n></code> there (see section 9).
<code>i</code> or <code>a</code>	In terminal (Normal mode)	Re-enter terminal mode (start typing commands again)
<code><Ctrl-w>h/j/k/l</code>	In terminal (Normal mode)	Move to another window
<code><Left>/<Right>/<Up>/<Down></code>	In terminal (Normal mode)	Move to another window (arrow key shortcuts)

Workflow example: You run code with `<Space>rr`. A terminal opens showing output. To go back to your code: press `<Esc>` to exit terminal mode, then `<Ctrl-w>l` (or `<Right>`) to move to the code window (the output is on the left). To close the terminal: `<Space>q` while in the terminal window.

Closing a Terminal

Method	Description
<code><Space>q</code>	While the terminal window is focused (press <code><Esc></code> first), close the window. A program that is still running keeps running in a hidden buffer (<code>\d</code> keeps such buffers)
Type <code>exit</code>	In an interactive shell terminal (<code>:term</code>), <code>exit</code> ends the shell and the window closes. A <code><Space>rr</code> run does NOT close its window when the program ends: the output stays (tested, no exit-code line is shown) until you close it with <code><Space>q</code>
<code>\d</code>	Delete the terminal buffer If the program is still running, Vim asks "Close "term://..."? [Y]es, (N)o, (C)ancel first (tested in a real terminal).

58. Session and Productivity

Auto-Save

Plugin: **auto-save.nvim**. Files are automatically saved when you:

- Switch to another application (`FocusLost`)
- Leave the current buffer (`BufLeave`)

It never saves unnamed, read-only or special buffers (terminals, help, ...) and never saves Typst and LaTeX files (their watchers would recompile on every save). After each save the message "AutoSave: saved at HH:MM:SS" appears.

Session Management

Plugin: **persistence.nvim** saves a session for the current folder (and git branch) automatically when you quit, once a real file was opened. It is never restored by itself: restore it from the dashboard (`r` this folder, `L` last session). Windows of Claude Code, terminals, nvim-tree, the outline, help and quickfix are left out of the saved session. `<Space>sv` (restart) brings windows, tabs and files back on its own.

Plugin: **vim-obsession** (manual alternative). Save and restore your entire Neovim session (open files, window layout, etc.). It loads right after the first screen, so a session started with `nvim -S Session.vim` keeps being updated while you work (no need to type `:Obsession` again).

Command	What it does
<code>:Obsession</code>	Start recording the session (saves to <code>Session.vim</code>)
<code>:Obsession!</code>	Stop recording and delete the session file
<code>nvim -S Session.vim</code>	Restore the session from the command line

Collaborative Editing

Plugin: **instant.nvim**. Real-time collaborative editing.

- The plugin loads on its first `:Instant...` command. Host and port are arguments of `:InstantStartServer` / `:InstantStartSession` / `:InstantJoinSession` (e.g. `:InstantStartSession 127.0.0.1 8081`); the built-in server defaults to port 8080.
- Uses your system username automatically

32. Statusline (`luaLine.nvim`)

Section	Position	Contents
A	Leftmost	Filename + a lock icon (Nerd Font) when the file is read-only
B	Left	Git branch (click: pick a branch and check it out), <code>↑[n]</code> / <code>↓[n]</code> commits ahead / behind the upstream (the upstream is refreshed with a silent <code>git fetch origin</code> at most once a minute; long branch names are cut at 20 characters), diff stats (+~-), diagnostic counts (same icons as the sign column), Python virtualenv (Python buffers only)
C	Center-left	Pending command (e.g. <code>2d</code>), spell indicator (<code>[SPELL]</code>)
X	Center-right	Active LSP (gear icon; <code>(+N)</code> = other attached clients; click: popup with the attached clients), <code>[N]trailing</code> (first line with trailing whitespace), <code>MI:N</code> (mixed tabs and spaces)
Y	Right	Encoding (only when not UTF-8) and file format (only when not unix), both in red; <code>[CN]</code> input-method badge on macOS
Z	Rightmost	Progress through the file (%). The line:column position is shown only in inactive windows

33. UI Features

Feature	Description
which-key.nvim	Press <code><Space></code> and wait: a popup shows all available leader keybindings
Dashboard	Start screen for a bare <code>nvim</code> (no file, directory or stdin); menu and keys in "Dashboard (Start Screen)" below
nvim-notify	Animated notification popups (fade + slide, 1500ms)
Colorschemes	On Nix systems the base16 theme named by <code>NVIM_BASE16_THEME</code> (fallback Catppuccin Mocha); on other systems one of 19 themes chosen at random at each start. UI colours (yank flash, hop keys, notifications, float borders) follow the active theme
dropbar.nvim	Breadcrumb bar at top showing file > class > function
nvim-colorizer	Color codes (hex, rgb) are highlighted with their actual color
mini.indentscope	Visual <code> </code> guide for current indent scope (loads right after the first screen; <code>ii</code> / <code>ai</code> exist from then on)
fidget.nvim	LSP progress messages in bottom-right corner
nvim-lightbulb	Lightbulb icon when code actions are available
vim-illuminate	Highlights the other uses of the word under the cursor (<code><Alt-n></code> / <code><Alt-p></code> jump between them)
vimade	Dims inactive windows
Borders	Floating windows and the completion menu have a single-line border (exceptions: <code>:Lazy</code> and the DevDocs float use rounded corners)

Dashboard (Start Screen)

The dashboard opens for a bare `nvim` (no file, no directory, no stdin) or with `:Dashboard`. `<Enter>` runs the item under the cursor. These single-letter keys work only inside the dashboard (the items that show `[<Leader> ...]` on the right are the global keys, they work everywhere, see "Help keys" below):

Key	Item
<code>r</code>	Restore session (this folder)
<code>L</code>	Restore last session
<code>o</code>	Recent files here (only files under the current directory)
<code>d</code>	Recent directories (zoxide picker; the item is shown only when <code>zoxide</code> is installed)
<code>m</code>	Search keymaps
<code>e</code>	New file
<code>q</code>	Quit Neovim

The other items show their normal key: Find File `<Space>ff`, Recently opened files `<Space>fr`, Project grep `<Space>fg`, Open tree view `<Space>s`, Search help `<Space>fh`, Claude Code `<Space>cc`, Open Nvim config `<Space>ev`. With nothing saved, `r` / `L` show one warning ("no saved session for this folder" / "no saved session"). Sessions are never restored automatically.

Keymap	Description
<code>\h</code>	Open the dashboard in the current window (the previous buffer stays open in the background; inside the dashboard it only says "already in the dashboard")
<code>\H</code>	Close the dashboard and return to the previous buffer (the dashboard buffer is deleted so it does not pile up); outside the dashboard it only warns "not in the dashboard", with no previous buffer "no previous buffer to resume"

Help keys: the user guide and Claude (any buffer, and the dashboard)

The same two keys work in every normal buffer and as dashboard items (shown as `[<Leader> ?]` and `[<Leader> a]`), so you learn them once.

Keys	What it does
<code><Space>?</code>	Open this user guide as a PDF (<code>user-guide/neovim-user-guide.pdf</code> , built from the markdown files) in its own zathura window next to Neovim, always at page 2, the table of contents (zathura's remembered last page is ignored); close it with <code>q</code> in zathura. Pressing the key again while it is open only shows a notice. Without zathura the system viewer opens the PDF
<code><Space>a</code>	Ask Claude how to do something in Neovim: a fresh <code>claude</code> session with the <code>answering-neovim-usage-questions</code> skill loaded, started in the <code>nvim</code> config folder with permissions bypassed (<code>--dangerously-skip-permissions</code> : it reads the guide and config, and can run commands or edit files, without asking). In a normal buffer it opens in a vertical split on the right (40% of the screen); on the dashboard it opens in its own tab. Type your question; <code>/exit</code> ends the session and closes the split or tab

In the Claude split, `<Esc>` goes to Claude (it interrupts a running answer). To leave terminal mode use `<Ctrl-\><Ctrl-n>`, then move to the editor window with `<Ctrl-w>h`. To edit the guide itself open the `.md` files directly (`<A-m>` previews one in the browser).

`:Dashboard` does the same as `\h`. To close the current buffer and get the dashboard instead: `:Dashboard | bdelete #` (a buffer with unsaved changes refuses with E89). `\d` deletes the buffer but shows the previous one, not the dashboard.

Chapter 8: Code: LSP, running, debugging, folding, treesitter

13. LSP: Language Server Protocol

Plugin: `nvim-lspconfig` (default server definitions; Neovim's builtin `vim.lsp` does the rest). Provides IDE features. The keys below work per buffer according to what the attached servers support: if no attached server supports it, `K` and `gd` fall back to Vim's builtin versions and `<Space>rn` / `<Space>ca` show one warning.

Configured Language Servers

Server	Language	Program(s) that must be on PATH
<code>pyright</code> + <code>ruff</code>	Python	<code>pyright-langserver</code> , <code>ruff</code>
<code>lua_ls</code>	Lua	<code>lua-language-server</code>
<code>bashls</code>	Bash	<code>bash-language-server</code>
<code>yamlls</code>	YAML	<code>yaml-language-server</code>
<code>marksman</code>	Markdown	<code>marksman</code>
<code>nixd</code>	Nix	<code>nixd</code>
<code>jdtls</code>	Java (via <code>nvim-java</code>)	<code>java</code> (Java devShell)
<code>clangd</code>	C/C++	<code>clangd</code> (c-cpp devShell)
<code>ltex_plus</code>	Grammar and spelling (Language-Tool) for markdown, tex, plain TeX, typst, gitcommit, text	<code>ltex-ls-plus</code>
<code>typos_lsp</code>	Typos in identifiers and comments, every real file	<code>typos-lsp</code>
<code>tinymist</code>	Typst	<code>tinymist</code>
<code>texlab</code>	LaTeX	<code>texlab</code> (LaTeX devShell)
<code>rust_analyzer</code>	Rust	<code>rust-analyzer</code> AND <code>cargo</code>
<code>gopls</code>	Go	<code>gopls</code> AND <code>go</code>
<code>hls</code>	Haskell	<code>haskell-language-server-wrapper</code>
<code>sourcekit</code>	Swift, Objective-C	<code>sourcekit-lsp</code>
<code>ts_ls</code>	JavaScript / TypeScript	<code>typescript-language-server</code> (the project's <code>node_modules/.bin</code> copy is preferred)
<code>phpactor</code>	PHP	<code>phpactor</code> (php devShell)
<code>r_language_server</code>	R, Rmd, quarto	<code>R</code> plus the <code>R</code> package <code>languageserver</code> : the first R file runs one silent background check; restart nvim after installing the package

A server is enabled only when ALL its programs are on PATH; otherwise it is skipped silently (no warning when you open a file of that language outside its devShell).

The programs come from the Nix system or the language's devShell; nothing is downloaded by Neovim. `:LspStart <name>` tells you which program is missing.

LSP Keymaps

Keymap	Description
<code>gd</code>	Go to definition: jump to where the symbol is defined (several different places open the location list)
<code>K</code>	Hover: show documentation in a floating window
<code><Space>rn</code>	Rename: rename the symbol everywhere it's used
<code><Space>ca</code>	Code action: show available fixes/refactors
<code><Space>fm</code>	Format the file on demand (LSP formatter, async). Lua: stylua. Markdown: prettier. Lua, Python and JSON have <code><Space>f</code> (stylua / black / <code>:JSONFormat</code>)

Built-in Neovim LSP and Diagnostic Keys

Neovim's own LSP keys also work next to the custom ones (in every buffer with a server that supports them):

Keymap	Mode	Description
<code>grn</code>	n	Rename (same as <code><Space>rn</code>)
<code>gra</code>	n, x	Code action (same as <code><Space>ca</code>)
<code>grr</code>	n	References (quickfix list)
<code>gri</code>	n	Implementation
<code>grt</code>	n	Type definition
<code>grx</code>	n	Run the code lens of the line
<code>g0</code>	n	Document symbols (outline in the location list)
<code><Ctrl-s></code>	i, s	Signature help
<code>]d</code> / <code>[d</code>	n	Next / previous diagnostic of any severity (<code><Space>de</code> / <code><Space>dE</code> jump to errors only)
<code>]D</code> / <code>[D</code>	n	Last / first diagnostic in the buffer
<code><Ctrl-w>d</code>	n	Show the diagnostics under the cursor

LSP Commands

Command	What it does
<code>:LspInfo</code>	LSP status (same as <code>:checkhealth vim.lsp</code>)
<code>:LspAttached</code>	Small popup with the servers attached to this buffer (<code>q</code> / <code><Esc></code> closes; also opens when you click the LSP name in the statusline; with no server attached it only shows a notification)
<code>:LspLog</code>	Open the LSP log file
<code>:LspRestart [name...]</code>	Restart the servers of this buffer, or the named ones
<code>:LspStop [name...]</code>	Stop them
<code>:LspStart [name...]</code>	Start the enabled servers of this buffer that are not running, or the named ones
<code>:LspInlayHints enable / disable</code>	Switch inlay hints on / off globally (off by default)

Glance: Peek Without Jumping

Plugin: `glance.nvim`. Preview definitions/references in a popup, without leaving your current file.

Keymap	Description
<code><Space>gd</code>	Peek at definitions
<code><Space>gr</code>	Peek at all references
<code><Space>gi</code>	Peek at implementations

Diagnostics (Errors, Warnings)

Nerd Font signs in the gutter: `✖`(error), `⚠`(warning), `ℹ`(info), `💡` (hint). No underline and no inline text: the message appears in a floating window when the cursor rests on the line (it closes when you move, leave the window or enter Insert mode).

Keymap	Description
<code><Space>db</code>	Telescope picker with the diagnostics of the current file
<code><Space>dw</code>	Toggle the Trouble diagnostics list (all open buffers)
<code><Space>de</code>	Jump to next error
<code><Space>dE</code>	Jump to previous error
<code><Space>dd</code>	Show diagnostic detail in floating window
<code><Space>dt</code>	Toggle diagnostics on/off
<code><Space>qw</code>	Send the diagnostics of all open buffers to the quickfix list
<code><Space>qb</code>	Send buffer diagnostics to quickfix list

44. Language Server Protocol (LSP) In Depth

What LSP Is

LSP is a protocol that lets Neovim communicate with language-specific servers (programs that understand your code). The server analyzes your code and provides:

- **Diagnostics:** Errors and warnings shown in the gutter and floating windows
- **Go to definition:** Jump to where a function/class/variable is defined
- **Hover:** Show documentation for the symbol under cursor
- **Rename:** Rename a symbol across the entire project
- **Code actions:** Quick fixes, auto-imports, refactorings
- **Formatting:** Auto-format your code according to language standards
- **Completion:** Suggestions as you type

How LSP Is Managed

Each server is configured in `lua/config/lsp.lua` (plus `after/lsp/<name>.lua`) with Neovim's builtin `vim.lsp.config` / `vim.lsp.enable`. **nvim-lspconfig** only supplies the default server definitions. A server is enabled only when its program is on PATH: the programs come from the Nix system (`neovim.nix`) or from the language's devShell, nothing is downloaded by Neovim. Java (`jdtls`) is managed by `nvim-java`.

Configured Servers and What They Provide

Server	Language	What it provides
pyright	Python	Type checking, import resolution, diagnostics. Disables import sorting (ruff handles that).
ruff	Python	Fast linting and formatting. Complementary to pyright.
lua_ls	Lua	Full Lua analysis with vim global recognized. Its formatting is switched off: Lua is formatted by stylua (<Space>f / <Space>fm).
bashls	Bash/Shell	Shell script analysis and diagnostics.
yamlls	YAML	Schema validation and formatting for YAML files.
marksman	Markdown	Link validation, heading completion. Formatting is done by <Space>fm with Prettier (not by marksman).
nixd	Nix	Nix language analysis. Formatter: nixpkgs-fmt.
jdtls	Java	Full Java IDE features via nvim-java (see Java section). Auto-configured.
clangd	C/C++	Compilation, diagnostics, code completion for C/C++.
ltex_plus	Prose (markdown, tex, typst, gitcommit, text)	Grammar and spell checking with LanguageTool; problems are diagnostics.
typos_lsp	Every real file	Finds typos in identifiers and comments; offers fixes as code actions.
tinymist	Typst	Diagnostics, completion (PDF export is left to :TypstWatch).
texlab	LaTeX	Diagnostics, hover, symbols, rename.
rust_analyzer	Rust	Full Rust analysis (needs cargo too).
gopls	Go	Full Go analysis (needs go too).
hls	Haskell	Haskell language server.
sourcekit	Swift, Objective-C	Swift analysis (C/C++ stay with clangd).
ts_ls	JavaScript / TypeScript	TypeScript language server.
phpactor	PHP	PHP analysis and refactoring.
r_language_server	R	R analysis, after a one-time check that the R package languageserver is installed.

Each server starts only when its program is installed (see the table in section 13); `:LspAttached` (or a click on the LSP name in the statusline) shows what is attached.

LSP Keymaps (All Languages)

These keys work per buffer according to what the attached servers support: `g` and `gd` fall back to Vim's builtin versions when no attached server supports hover / definition; `<Space>rn` and `<Space>ca` show one warning instead ("rename: no attached language server supports it", or "... no language server attached to this buffer").

Keymap	What it does	When to use
<code>gd</code>	Go to definition. If there's only one definition, jumps directly. If multiple, opens a location list so you can pick. Deduplicates results.	When you want to see where a function/class/variable is defined.
<code>K</code>	Hover documentation. Shows docs in a floating window with a border (at most 100 x 40).	When you need to check what a function does, its parameters, return type, etc.
<code><Space>rn</code>	Rename symbol. Renames the symbol under cursor everywhere it appears in the project.	When refactoring: changing a function name, variable name, etc.
<code><Space>ca</code>	Code action. Shows a menu of available fixes and refactorings.	When the lightbulb icon appears, or when you want to auto-import, extract a variable, fix a lint warning, etc.
<code><Space>fm</code>	Format file. Runs the LSP formatter asynchronously (ruff, nixd, ...); in Markdown buffers Prettier, in Lua buffers stylua.	Before committing, or whenever you want clean formatting.

Peeking Without Jumping (Glance)

Plugin: **glance.nvim**. Instead of jumping away to a definition (which changes your context), you can peek at it in an inline popup:

Keymap	What it does
<code><Space>gd</code>	Peek at definitions in a popup. You see the code without leaving your current file. Press <code><Esc></code> to close.
<code><Space>gr</code>	Peek at all references. See every place in the project that uses this symbol.
<code><Space>gi</code>	Peek at implementations. See how interfaces/abstract methods are implemented.

Neovim's builtin `grn`, `gra`, `grr`, `gri`, `grt` and `g0` also work (see section 13).

When to use Glance vs `gd`: Use Glance when you want to quickly check something and come back. Use `gd` when you want to actually navigate to the definition and work there.

Diagnostics In Depth

Diagnostics are the errors, warnings, and hints that the LSP server reports about your code.

How they appear:

- Nerd Font signs in the gutter: ☹(error), ⚠(warning), ⓘ(info), ℹ (hint)
- A floating window automatically appears after ~500ms when your cursor rests on a line with diagnostics; it closes when you move the cursor or enter insert mode
- The statusline (left side) shows the diagnostic counts with the same icons, e.g.

☹ 1 ⚠ 3

Navigation:

Keymap	What it does
<code><Space>de</code>	Jump to the next error (skips warnings/hints)
<code><Space>dE</code>	Jump to the previous error
<code><Space>dd</code>	Manually open the diagnostic float for the current line
<code><Space>db</code>	Open a Telescope picker showing all diagnostics in the current file
<code><Space>dw</code>	Open Trouble showing all diagnostics across the workspace
<code><Space>dt</code>	Toggle diagnostics on/off globally (a message says which; the automatic float stays off while disabled)

Sending diagnostics to quickfix:

Keymap	What it does
<code><Space>qw</code>	Put the diagnostics of all open buffers into the quickfix list
<code><Space>qb</code>	Put current buffer diagnostics into the quickfix list (with none you only get "No diagnostics in this buffer")

Then use `:cnext / :cprev` to jump through them one by one.

The Lightbulb

Plugin: **nvim-lightbulb**. A lightbulb icon appears in the sign column whenever the LSP has code actions available for the current line. This is your cue to press `<Space>ca`.

The lightbulb filters out noisy ruff actions (`source.fixAll.ruff`, `source.organizeImports.ruff`) to avoid false positives.

18. Code Folding (nvim-ufo)

Plugin: `nvim-ufo`. Folds code blocks using the LSP folding ranges, falling back to indentation.

Keymap	Description
<code>za</code>	Toggle fold at cursor
<code>zA</code>	Toggle all folds under cursor recursively
<code>zc</code> / <code>zo</code>	Close / open fold at cursor
<code>zC</code> / <code>zO</code>	Close / open all folds recursively
<code>zR</code>	Open all folds in the file
<code>zM</code>	Close all folds in the file
<code>zr</code>	Open one more fold level (<code>z zr</code> = two levels; counted from the folds you see, so it works right after <code>zM</code> ; in a buffer without ufo folds: one warning)
<code>zm</code>	Close one more fold level (accepts a count)
<code><Space>K</code>	Preview folded lines in a popup
<code>zi</code>	Toggle folding feature on/off

47. Code Folding In Depth

What It Is

Plugin: **`nvim-ufo`** + **`promise-async`**. Code folding collapses blocks of code (functions, classes, if-blocks, etc.) into a single line to help you see the big picture.

How It Works

`nvim-ufo` uses the LSP server's folding ranges to determine what can be folded, and falls back to indentation when no server provides folds.

Folded lines show a preview: the first line of the fold + a count like `⌄ 42` showing how many lines are hidden.

Folding Keymaps

Keymap	What it does	When to use
<code>za</code>	Toggle the fold under cursor	Quick open/close of a single fold
<code>zR</code>	Open ALL folds in the file	When you want to see everything
<code>zM</code>	Close ALL folds in the file	When you want the bird's-eye view
<code>zr</code>	Open one more fold level (<code>{N}zr</code> = N levels; in a buffer without ufo folds: one warning)	Gradually reveal more detail
<code>zm</code>	Close one more fold level (<code>{N}zm</code> = N levels)	Step back to less detail
<code>zo</code> / <code>zc</code>	Open / close fold at cursor	Precise control
<code>zO</code> / <code>zC</code>	Open / close all nested folds at cursor	Deep open/close
<code><Space>K</code>	Preview folded lines in popup	See what's inside without unfolding
<code>zi</code>	Toggle folding on/off globally	Temporarily disable all folding

Workflow tip: Press `zM` to close all folds when you open a large file. This gives you an outline view. Then use `za` to open only the sections you care about. Use `<Space>K` to peek inside folds without opening them.

21. Treesitter & Text Objects

Treesitter (Plugin)

Provides tree-sitter syntax highlighting (started automatically per filetype when a parser exists). On Nix systems the parsers come from the nix store and nothing is installed by Neovim; on other systems these parsers are installed automatically: `cpp`, `diff`, `dockerfile`, `git_config`, `git_rebase`, `gitcommit`, `html`, `json`, `lua`, `python`, `toml`, `vim`.

targets.vim (Plugin)

Adds many additional text objects for quotes, brackets, arguments, separators. Works automatically with `d`, `c`, `y`, `v`. If the cursor is not inside the pair, `i(`, `i"` and friends look forward on the line.

vim-matchup (Plugin)

Enhanced `%` matching for language keywords (`if/else/end`, `do/while`, etc.). Shows offscreen match in popup. Also: `g%` (backwards `%`), `[% / ]%` (start / end of the enclosing pair), `z%` (into the next pair), text objects `i%` / `a%`. `g%` and `[% / ]%` were tested (`g%` from `if` goes backwards to `end`). `z%` was tested in a real terminal: from `if` it moves to the closing `)` of the next pair inside the block.

46. Treesitter In Depth

What Treesitter Is

Plugin: **`nvim-treesitter`**. It parses your code into a syntax tree (like an AST) and uses that for:

- **Syntax highlighting**: More accurate than regex-based highlighting. Understands the actual structure of the code.
- **Symbols**: the aerial outline (`<Space>t`) can read the symbols from the tree.

Installed Parsers

On Nix-managed systems (a folder `/etc/nixos` or `/etc/nix` exists) the parsers come from the nix store (home-manager); Neovim installs nothing. On other systems Neovim installs this fixed set at startup: `cpp`, `diff`, `dockerfile`, `git_config`, `git_rebase`, `gitcommit`, `html`, `json`, `lua`, `python`, `toml`, `vim`. Neovim itself bundles the parsers for `c`, `lua`, `vim`, `vimdoc`, `query` and `markdown`, so those highlight everywhere. Other languages get no tree-sitter highlighting there until you run `:TSInstall <lang>`. Tested on a simulated non-nix system: each of the 12 grammars is downloaded, but the install needs the `tree-sitter` command and a C compiler (`gcc/cc`); without the `tree-sitter` command every grammar fails with Error during "tree-sitter build": ... ENOENT ... 'tree-sitter' and nothing is installed (the error lines appear again at every start). So on a non-nix machine install `tree-sitter` (the CLI) and a C compiler first.

19. Code Running

Custom function in `lua/mappings.lua`. Opens the output in a vertical split terminal on the left. If the file has no name (the buffer was never saved), the filetype has no runner, or the needed program is not on PATH, you get one warning (naming the devShell to start nvim in) instead of a terminal. A named buffer with unsaved changes runs the version on disk, so save first (`:w`).

Keymap	Description
<code><Space>rr</code>	Run current file (auto-detects language)

Supported: Python, Java, C, C++, C#, JavaScript, TypeScript, Go, Rust, Bash, Lua, Ruby, PHP. Special cases: Java with jdtls attached runs `:JavaRunnerRunMain` (nvim-java's own runner split at the bottom, not the `<Space>rr` terminal on the left); Rust inside a cargo project runs `cargo run`; C# with a `.csproj` runs `dotnet run --project`; Go runs `go run .` for the whole package.

After running, the terminal output appears in a split. See [Terminal Integration](#) for how to navigate to/from it and close it.

Filetype-Specific

Keymap	Filetype	Description
<code><Space>rf</code> / <code><F9></code>	Python	Run with <code>python -u</code> via AsyncRun (<code>uv run python -u</code> inside a uv project)
<code><Space>rf</code> / <code><F9></code>	C++	Compile (clang++, else g++, C++20) and run in a split below; only mapped when a compiler is on PATH
<code><Space>rf</code> / <code><F9></code>	LaTeX	Compile with vimtex
<code><Space>rf</code> / <code><F9></code>	Lua	Run the file inside Neovim (<code>:luafile %</code>)
<code><Space>rf</code> / <code><F9></code>	Vim script	Source the file (<code>:source %</code>)

55. Code Running In Depth

The Universal Runner

The `<Space>rr` keymap detects the current filetype and runs the appropriate command in a **vertical split terminal** on the left. If the file is unsaved, the filetype has no runner, or the compiler/interpreter is not on PATH, you get ONE warning and no terminal opens (the message names the devShell to start nvim in).

How it works:

1. Detects the filetype of the current buffer
2. Builds the correct command (e.g., `python3 file.py`, `go run .`)
3. Opens a vertical split
4. Starts a terminal in that split running the command
5. Output appears in real-time

After running:

- The terminal window opens on the LEFT of your code
- Press `<Esc>` in the terminal to enter Normal mode

- Navigate back to your code with `<Ctrl-w>l` or `<Right>`
- Close the terminal window with `<Space>q`, or delete its buffer with `\d`
- Run again with `<Space>rr` (it opens a new terminal each time)

Language-Specific Details

Language	Command used	Notes
Python	<code>python3 <file></code>	
Java	<code>:JavaRunnerRunMain</code> when <code>jdtls</code> (nvim-java) is attached; otherwise <code>java <file></code>	With <code>jdtls</code> nvim-java opens its own runner split at the bottom (not the <code><Space>rr</code> terminal on the left)
C	<code>gcc -Wall -Wextra -std=c11 <file> -o <binary> && <binary></code>	Compiles and runs; the binary sits next to the source; needs gcc (c-cpp devShell)
C++	<code>g++ -Wall -Wextra -std=c++20 <file> -o <binary> && <binary></code>	Compiles and runs; needs g++ (c-cpp devShell)
C#	<code>dotnet run --project <nearest .csproj></code>	Without a project file: <code>dotnet run <file></code>
JavaScript	<code>node <file></code>	
TypeScript	<code>node <file></code>	Node runs <code>.ts</code> directly
Go	<code>go run .</code> in the file's directory	Runs the whole package
Rust	<code>cargo run</code> when a <code>Cargo.toml</code> is above the file	Otherwise <code>rustc <file></code> and runs the binary
Bash	<code>bash <file></code>	
Lua	<code>nvim -l <file></code>	Neovim's own LuaJIT
Ruby	<code>ruby <file></code>	
PHP	<code>php <file></code>	

Filetype-Specific Runners

Some filetypes have an additional `<Space>rf` runner (also `<F9>`):

Filetype	What <code><Space>rf</code> / <code><F9></code> does
Python	Runs with <code>python -u <file></code> via <code>AsyncRun</code> (unbuffered output; <code>uv run python -u</code> in a <code>uv</code> project)
C++	Compiles with <code>clang++</code> (else <code>g++</code>) <code>-Wall -Wextra -std=c++20 -O2</code> and runs it in a horizontal split; only mapped when a compiler is on <code>PATH</code>
LaTeX	<code>vimtex</code> compile; only when <code>latex</code> is on <code>PATH</code>

36. Debugging

Plugin	Keymap / Command	Description
nvim-dap	(lazy-loaded)	Debug Adapter Protocol client. Java debugging auto-configured via nvim-java.
nvim-gdb	<code><Space>dp</code>	Python buffers only: start pdb on the current file (Linux/Windows only; elsewhere one warning); during the session <code><Space>dc</code> <code>dn</code> <code>ds</code> <code>df</code> <code>dB</code> <code>du</code> <code>dv</code> step and inspect (table in section 2). <code>:GdbStart</code> (gdb) works inside the c-cpp / rust devShells

56. Debugging In Depth

Debug Adapter Protocol (DAP)

Plugin: **nvim-dap**. DAP is a standardized protocol (created by Microsoft) for communication between an editor and a debugger. It's the same protocol used by VS Code.

Java: Debugging is auto-configured via nvim-java (only inside the Java devShell, where `java` is on PATH). Open a Java file, set breakpoints, and use `<Space>jtC` (debug the current test class) or `<Space>jtM` (debug the current test method).

Python: In Python buffers `<Space>dp` starts `python -m pdb` on the current file through nvim-gdb. Only available on Linux/Windows. During the session use `<Space>dc` (continue), `dn` (next), `ds` (step), `df` (finish), `dB` (breakpoint), `du` (until) and `dv` (evaluate); the full table is in section 2 ("Python debugger keys").

GDB Integration

Plugin: **nvim-gdb**. For C/C++ debugging with GDB. Available on Linux and Windows only.

43. How the Development Toolchain Fits Together

When you open a code file in Neovim, several systems activate automatically behind the scenes:

```
You open a file
|
v
Treesitter parses the syntax tree --> accurate highlighting (plus symbols for the aerial
outline)
|
v
LSP server starts (e.g., pyright for Python) --> diagnostics, go-to-definition, hover,
rename, code actions
|
v
Completion engine (nvim-cmp) connects to LSP --> autocomplete suggestions as you type
|
v
Gitsigns reads git status --> change markers in gutter
|
v
Lightbulb watches LSP --> shows icon when code actions are available
|
```

```
v
Diagnostics config --> errors/warnings appear as Nerd Font signs, a float opens on
CursorHold
```

You don't need to start any of this manually. It all happens on file open.

53. Documentation Lookup

DevDocs (Plugin)

Plugin: **nvim-devdocs**. Browse programming documentation without leaving Neovim.

Command	What it does
<code>:DevdocsOpen</code>	Open the documentation in a normal buffer (current window)
<code>:DevdocsOpenFloat</code>	Open in floating window (25 lines tall, 100 chars wide)
<code>:DevdocsInstall</code>	Install documentation for a language (e.g., <code>:DevdocsInstall python</code>)
<code>:DevdocsUninstall</code>	Remove installed docs

Hover Documentation (LSP)

Press **K** on any symbol to see its documentation in a floating window. This pulls from:

- Function signatures and return types
 - Docstrings / JSDoc / Javadoc
 - Type information
-

59. Useful Developer Commands

Command	What it does
<code>:LspInfo</code>	LSP status (runs <code>:checkhealth vim.lsp</code>)
<code>:Lazy</code>	Open plugin manager
<code>:Lazy update</code>	Update all plugins
<code>:JSONFormat</code>	Format JSON (works on visual selection too)
<code>:ToPDF</code>	Convert markdown to PDF via pandoc
<code>:Redir <cmd></code>	Capture any Neovim command output (e.g., <code>:Redir messages</code>)
<code>:Telescope keymaps</code>	Browse all defined keymaps
<code>:checkhealth</code>	Diagnose Neovim installation issues
<code>:Flog</code>	Git log graph
<code>:DiffviewOpen</code>	Side-by-side diff view
<code>:Neogit</code>	Full git UI
<code>:DevdocsOpen</code>	Browse programming documentation
<code>:YankyRingHistory</code>	Browse yank history
<code>:LspAttached</code>	Popup with the LSP servers attached to this buffer
<code>:LspLog</code>	Open the LSP log
<code><Space>u</code>	Toggle the undo tree (the <code>:Undotree</code> command exists only after the first <code><Space>u</code>)
<code>:AerialToggle!</code>	Toggle the symbol outline (<code><Space>t</code>)
<code>:Dashboard</code>	Open the start screen
<code>:CodeDiff</code>	VSCoDe-style side-by-side diff
<code>:DBUI</code> / <code>:Dbee</code>	SQL clients

Part III: Everyday Scenarios & Recipes

Practical, step-by-step walkthroughs for common tasks.

75. Real-World Developer Workflows

Step-by-step walkthroughs of common developer tasks entirely within Neovim.

Workflow: Investigating a Bug

1. `<Space>fg` - search for the error message text across the project
2. `<Enter>` on the relevant result - jumps to the file and line
3. `gd` - go to the definition of the function that causes the error
4. `K` - read the function's documentation
5. `<Space>gr` - see everywhere this function is called (glance references)
6. `<Ctrl-o>` - jump back to where you were
7. `<Space>hb` - check git blame: who changed this and when
8. `]c` / `[c` - navigate to nearby git changes (hunks)
9. `<Space>hp` - preview what the hunk changed

10. Fix the issue, `<Space>w` to save, `<Space>rr` to run and test

Workflow: Code Review (Reviewing Your Own Changes)

1. `<Space>gs` – open git status
2. Navigate to a changed file, press `<Enter>` to open it
3. `jc` – jump to the first changed hunk
4. `<Space>hp` – preview the change
5. `jc` – next change, repeat
6. `:DiffviewOpen` – for a full side-by-side diff of all changes
7. When satisfied: `<Space>gw` to stage, `<Space>gc` to commit

Workflow: Refactoring a Function Name Across the Project

If it's a code symbol (function, class, variable):

1. Place cursor on the name
2. `<Space>rn` – LSP rename, type new name, Enter
3. Done. All references updated intelligently.

If it's arbitrary text (e.g., a string, API path, config key):

1. `<Space>fg` – search for it first, verify all the places it appears
2. `:grep "oldText"` – populate the quickfix list
3. `:copen` – review the matches
4. `:cfdo %s/oldText/newText/gc | update` – replace with confirmation (y/n each) and save all files

Workflow: Adding a Feature in a New Branch

1. `<Space>gbn` – create a new branch (type name, Enter)
2. `<Space>s` – open file tree, navigate to where you'll add files
3. `a` in the tree – create a new file
4. Write code; `<Space>fm` to format; `<Space>rr` to run/test
5. `<Space>de` – jump through any errors
6. `<Space>ca` – apply code action fixes
7. `<Space>gw` – stage the file
8. `<Space>gc` – commit
9. `<Space>gpu` – push

Workflow: Quickly Editing a Config File

1. `<Space>ff` – fuzzy find the config file by name
2. Make your changes
3. `<Space>w` – save
4. If it's the Neovim config: `<Space>sv` to restart Neovim (writes all buffers, restores windows/tabs/files; terminals such as Claude Code are not restarted)

Workflow: Working with JSON

1. Open the JSON file
2. If it's messy: `:JSONFormat` to pretty-print it
3. `<Space>fg` in another terminal to find references to JSON keys
4. `za` to fold/unfold sections for readability
5. `ci"` to change a value inside quotes
6. `<Space>w` to save

Workflow: Writing Documentation (Markdown)

1. Open the `.md` file
2. `<Alt-m>` - live preview in browser
3. Write content; wrapping is auto-enabled for markdown
4. `^^` - add a footnote
5. `+` (operator) - convert lines to a bulleted list
6. `:Tabularize /|` - align a markdown table
7. `:AddRef label url` - add a reference link
8. `<Space>cz` - toggle spell check
9. `]s / [s` - navigate misspelled words
10. `z=` - fix spelling

Workflow: Pair Programming with Split Views

1. `:vs <file>` - open another file side-by-side
 2. `<Ctrl-w>l / <Ctrl-w>h` - switch between the two files
 3. `van` in file A - select the surrounding syntax node; repeat `an` to grow the selection (for example to the whole function), `in` shrinks it again; then `y` to copy
 4. `<Ctrl-w>l` - switch to file B
 5. `p` - paste the function
 6. `<Ctrl-w>=` - equalize window sizes if they got uneven
 7. `<Ctrl-w>o` - when done, close all splits except current
-

35. Java Development (`nvim-java`)

The Java keys work only in a Java buffer with the Java language server (jdtls) attached: open nvim inside the Java devShell (`java` on PATH). Everywhere else the same keys show one warning "Java: jdtls not attached (open nvim inside the Java devShell)". which-key groups: `<Space>j` Java, `jb` build, `jr` runner, `jt` test, `je` extract.

Build & Run

Keymap	Description
<code><Space>jbb</code>	Build workspace
<code><Space>jbc</code>	Clean workspace
<code><Space>jrr</code>	Run main class
<code><Space>jrs</code>	Stop running main
<code><Space>jrl</code>	Toggle runner log window
<code><Space>jrp</code>	Profiles UI

Testing

Keymap	Description
<code><Space>jtc</code>	Run all tests in current class
<code><Space>jtC</code>	Debug all tests in current class
<code><Space>jtm</code>	Run test method under cursor
<code><Space>jtM</code>	Debug test method under cursor
<code><Space>jtr</code>	View last test report

Refactoring

Keymap	Description
<code><Space>jev</code>	Extract variable
<code><Space>jeo</code>	Extract variable (all occurrences)
<code><Space>jec</code>	Extract constant
<code><Space>jem</code>	Extract method
<code><Space>jef</code>	Extract field
<code><Space>jj</code>	Change JDK runtime
<code><Space>jd</code>	Configure debugger (DAP)

54. Java Development In Depth

Plugin: **nvim-java**.

This is the most feature-rich language setup in this config. It provides a full Java IDE experience. nvim-java loads when you open the first Java file of the session (not at startup), so opening a Java file takes a moment longer the first time; non-Java sessions do not pay for it.

How It Works

nvim-java wraps the Eclipse JDT Language Server (jdtls) and adds:

- Build system integration
- Test runner and debugger
- Spring Boot tools
- Refactoring commands
- DAP (Debug Adapter Protocol) for step-through debugging

All of this only starts inside the Java devShell (`java` on PATH). Elsewhere `.java` files open without Java tooling, and the `<Space>j` keys show one warning "Java: jdtls not attached (open nvim inside the Java devShell)".

On Nix systems the JDK comes from the Java devShell (`JAVA_HOME`) and nvim-java never downloads one. On other systems nvim-java auto-installs a JDK.

Build & Run

Keymap	Command	What it does
<code><Space>jbb</code>	<code>:JavaBuildBuildWorkspace</code>	Compile the entire workspace
<code><Space>jbc</code>	<code>:JavaBuildCleanWorkspace</code>	Clear the jdtls workspace cache (after a Yes/No question; jdtls is restarted automatically)
<code><Space>jrr</code>	<code>:JavaRunnerRunMain</code>	Run the main class
<code><Space>jrs</code>	<code>:JavaRunnerStopMain</code>	Stop the running program
<code><Space>jrl</code>	<code>:JavaRunnerToggleLogs</code>	Show/hide the runner log window (it opens as a full-width, 15-line split at the bottom)
<code><Space>jrp</code>	<code>:JavaProfile</code>	Profiles UI

Testing

Keymap	Command	What it does
<Space>jtc	:JavaTestRunCurrentClass	Run all @Test methods in the current class
<Space>jtC	:JavaTestDebugCurrentClass	Debug all tests (with breakpoints)
<Space>jtm	:JavaTestRunCurrentMethod	Run only the test method under cursor
<Space>jtM	:JavaTestDebugCurrentMethod	Debug only the test under cursor
<Space>jtr	:JavaTestViewLastReport	Show pass/fail results from the last test run

Debugging

Keymap	Command	What it does
<Space>jd	:JavaDapConfig	Configure the debug adapter (auto-runs on Java file open, but can be re-triggered)

DAP is configured automatically when jdtls starts. Debugging uses the nvim-dap commands (:DapToggleBreakpoint , :DapContinue , :DapStepOver , :DapStepInto , :DapStepOut , :DapTerminate); this config has no keys for them.

Refactoring

Keymap	Command	What it does
<Space>jev	:JavaRefactorExtractVariable	Extract the expression under the cursor (or selection) to a local variable
<Space>jeo	:JavaRefactorExtractVariableAllOccurrence	Extract and replace ALL occurrences of the expression
<Space>jec	:JavaRefactorExtractConstant	Extract a constant
<Space>jem	:JavaRefactorExtractMethod	Extract a method
<Space>jef	:JavaRefactorExtractField	Extract a field
<Space>jj	:JavaSettingsChangeRuntime	Switch the JDK version

Only the :JavaBuild* and :JavaRefactor* commands exist after jdtls has attached; the other :Java* commands exist as soon as nvim-java is loaded but do nothing useful without jdtls.

Chapter 9: Git

20. Git Integration

vim-fugitive (Plugin)

The fugitive keys (and the gitlinker keys below) exist only inside a git repository: nvim started in one, or a file of one opened. Outside a repository these keys are not mapped: `<Space>` just moves the cursor one column right and the next keys run as their normal Vim/plugin meaning (`<Space>gs` becomes `␣` plus vim-swap's `gs`). `<Space>gbl` works everywhere (fzf-lua).

Keymap	Mode	Description
<code><Space>gs</code>	n	Git status window
<code><Space>gw</code>	n	Git add current file
<code><Space>gc</code>	n	Git commit
<code><Space>gpl</code>	n	Git pull
<code><Space>gpu</code>	n	Git push (opens terminal split)
<code><Space>gb</code>	x	Git blame selected lines
<code><Space>gbn</code>	n	Create new branch (prompts for name)
<code><Space>gbd</code>	n	Puts <code>:Git branch -D</code> on the command line: type the branch name and press Enter (force delete)
<code><Space>gf</code>	n	Puts <code>:Git fetch</code> on the command line (add arguments, then Enter)
<code><Space>gbl</code>	n	Fuzzy-search git branches and check one out (fzf-lua)

Clicking the branch name in the statusline also opens a branch picker (`git checkout` of the chosen local or remote branch).

gitsigns.nvim (Plugin)

Shows `+` `~` `_` signs in the gutter for added/changed/deleted lines.

Keymap	Description
<code>]c</code>	Jump to next git change (hunk)
<code>[c</code>	Jump to previous git change
<code><Space>hp</code>	Preview the hunk in a floating window
<code><Space>hb</code>	Show git blame for current line

gitlinker.nvim (Plugin)

Keymap	Mode	Description
<code><Space>gl</code>	n, x	Copy permalink for current line(s)
<code><Space>gbr</code>	n	Open repository in browser

Other Git Tools

Plugin	Command / Trigger	Description
neogit	<code>:Neogit</code>	Full git UI (magit-like; loads on the first <code>:Neogit*</code> command, in any directory; also <code>:NeogitCommit</code> , <code>:NeogitLogCurrent</code> , <code>:NeogitResetState</code>)
diffview.nvim	<code>:DiffviewOpen</code> , <code>:DiffviewFileHistory</code> (<code>:DiffviewClose</code> once a view was opened)	Side-by-side diff viewer and 3-way merge tool; file history panel
vim-flog	<code>:Flog</code>	Visual git log graph
diffs.nvim	<code>:Diff</code> (and automatic)	Unified diff of the current file against git; also colours the diffs shown by fugitive, neogit and git-signs, and conflict markers
codediff.nvim	<code>:CodeDiff</code>	VSCoDe-style side-by-side diff (downloads a small native library on first use)

Resolving Merge Conflicts (diffview.nvim)

During a merge, `:DiffviewOpen` opens the 3-way merge tool. Inside a diffview view:

Keymap	Description
<code><Space>gCo</code> / <code><Space>gCt</code>	Choose OURS / THEIRS for the conflict
<code><Space>gCb</code> / <code><Space>gCa</code>	Choose BASE / BOTH
<code>]C</code> / <code>[C</code>	Next / previous conflict (capital C; <code>]c</code> / <code>[c</code> stay the gitsigns hunk keys)

`<Space>cb` and `<Space>ca` keep their normal meaning inside diffview. fugitive's `:Gvdiffsplit!` is the other way to resolve a conflict.

48. Git Workflow In Depth

The Git Plugin Ecosystem

This config includes several git-related plugins that each handle a different aspect:

Plugin	What it does	How to use
vim-fugitive	Run git commands from inside Neovim. The core git plugin.	<code><Space>gs</code> for status, <code><Space>gc</code> for commit, etc.
gitsigns.nvim	Shows which lines changed in the gutter. Navigate between changes.	<code>]c</code> / <code>[c</code> to jump between hunks, <code><Space>hp</code> to preview.
gitlinker.nvim	Generate shareable URLs to specific lines of code.	<code><Space>gl</code> to copy a permalink.
neogit	A full git UI inside Neovim (like Magit for Emacs).	<code>:Neogit</code> to open (loads on that command; also <code>:NeogitCommit</code> , <code>:NeogitLogCurrent</code> , <code>:NeogitResetState</code>).
diffview.nvim	Side-by-side diff viewer for comparing branches, commits, etc.	<code>:DiffviewOpen</code> to open.
vim-flog	Visual git log/graph showing branch history.	<code>:Flog</code> to open.
diffs.nvim	Syntax highlighting inside the diffs of fugitive, neogit and gitsigns; conflict markers. <code>:Diff</code> shows the file against git.	Automatic, <code>:Diff</code> .
codediff.nvim	VSCode-style side-by-side diff.	<code>:CodeDiff</code> .

Merge conflicts: `:DiffviewOpen` is the merge tool (keys in section 20, "Resolving Merge Conflicts").

Daily Git Workflow

A typical workflow entirely from within Neovim:

1. **Check status:** `<Space>gs` opens the fugitive status window
2. **Stage a file:** `<Space>gw` stages the current file (or use `s` in the status window)
3. **Review changes:** `<Space>hp` to preview hunks, or `]c`/`[c` to navigate between them
4. **Commit:** `<Space>gc` opens a commit message buffer. Write message, then `:wq`
5. **Push:** `<Space>gpu` pushes (opens a terminal split showing progress)
6. **Pull:** `<Space>gpl` pulls latest changes
7. **Blame:** Select lines in visual mode, then `<Space>gb` to see who wrote them
8. **Create branch:** `<Space>gbn` prompts for a branch name
9. **Share code:** `<Space>gl` copies a permalink to the current line

Understanding Gitsigns

The gutter signs mean:

- `+` : This line was **added** (new code)
- `~` : This line was **modified** (changed from last commit)
- `_` : A line was **deleted below** this line
- `-` : A line was **deleted above** this line
- `|` : This line has both additions and deletions (change-delete)

Hunk navigation: `]c` jumps to the next changed block (hunk), `[c` jumps to the previous. This is very useful during code review.

Chapter 10: Claude, Markdown, LaTeX/Typst, spelling, URLs

9. AI Assistant Window (Claude Code)

Claude Code

Keymap	Mode	Description
<code><Space>cc</code>	n	Toggle Claude Code terminal (opens/closes it)
<code><Space>a</code>	n	Fresh Claude session for a quick "how do I do X in Neovim" question: vertical split on the right, started in the nvim config folder with the <code>answering-neovim-usage-questions</code> skill loaded and permissions bypassed (no prompts); <code>/exit</code> closes it (on the dashboard it opens in its own tab). See section 33 "Help keys"
<code><Space>cR</code>	n	Resume/continue the last Claude conversation (<code>claude --continue</code>)
<code><Space>cV</code>	n	Start Claude in verbose mode
<code>:ClaudeCodeResume</code>		Start <code>claude --resume</code> (pick an older conversation; command only, no key)
<code><Ctrl-h/j/k/l></code>	t, n (Claude panel)	Move to the window left / below / above / right
<code><Ctrl-f> / <Ctrl-b></code>	t (Claude panel)	Scroll a page down / up

Claude Code opens as a **vertical split on the right**, 30% of the screen width. It is a terminal buffer. There is no toggle key in terminal mode. To navigate:

1. **Move to the Claude window:** `<Ctrl-w>l` or `<Right>` (it opens on the right). Focusing it puts you in terminal (insert) mode by itself.
2. **Move back to code:** `<Ctrl-h>` (works directly in the Claude terminal), or `<Ctrl-\><Ctrl-n>` and then `<Ctrl-w>h` / `<Left>`. `<Esc>` is sent to Claude (for example to interrupt it) and does **not** leave terminal mode.
3. **Close Claude:** from the code window `<Space>cc` toggles it closed. Inside the Claude terminal press `<Ctrl-\><Ctrl-n>` first, then `<Space>cc` or `<Space>q`.
4. **Type in Claude:** If in Normal mode inside the Claude terminal, press `i` to re-enter terminal mode

Claude runs in the git root of the current file. Files Claude changes are reloaded in their buffers (checked every second while the panel is open). The panel is left out of saved sessions, and `\D` never deletes it while it runs.

49. AI-Assisted Development In Depth

Claude Code

Plugin: `claude-code.nvim`

Claude Code is an AI coding assistant that runs in a terminal inside Neovim. The plugin loads right after the first screen is drawn (VeryLazy), so its keys and `:ClaudeCode*` commands exist a fraction of a second after Neovim starts.

Keymap	Mode	What it does
<code><Space>cc</code>	n	Toggle the Claude Code terminal. Opens as a vertical split on the right (30% of the width).
<code><Space>cR</code>	n	Resume/continue the last Claude conversation (<code>claude --continue</code>)
<code><Space>cV</code>	n	Start Claude in verbose mode

Using Claude Code:

1. Press `<Space>cc` to open
2. It opens as a terminal buffer in a vertical split on the right
3. Type your request and press Enter
4. Claude can read and edit your files directly
5. Leave terminal mode with `<Ctrl-\><Ctrl-n>` (a plain `<Esc>` goes to Claude, e.g. to interrupt it), then press `<Space>cc` to close the panel

Claude Code uses your project's git root as the working directory. `:ClaudeCodeResume` starts `claude --resume`. Inside the panel `<Ctrl-h/j/k/l>` move to the neighbour windows and `<Ctrl-f>` / `<Ctrl-b>` scroll.

27. Markdown Support

Preview

Keymap	Description
<code><Alt-m></code>	Toggle markdown preview in browser (markdown buffers only)

Footnotes

Keymap	Mode	Description
<code><Space>mf</code>	n	Add footnote (markdown buffers only; elsewhere one warning)
<code><Space>mr</code>	n	Return from footnote (markdown buffers only)
<code>^^</code>	n, i	Insert footnote number (markdown files only). Because of this map, a single <code>^</code> or <code>@</code> typed in insert mode appears after a short pause (500 ms), since Neovim waits for a possible second key
<code>@@</code>	n, i	Return from footnote (markdown files only; it shadows the macro replay <code>@@</code> in Markdown buffers)

Text Objects & Operators (Markdown Only)

Keymap	Description
<code>vic / vac</code>	Select inside / around code block
<code>+</code>	Convert lines to unordered list (operator: <code>+ip</code> for paragraph)
<code><Space>mb{motion}</code> (or <code><Space>mb</code> on a Visual selection)	Add a hard line break (a trailing <code>\</code>) to each line, e.g. <code><Space>mbip</code> ; blank lines and lines already ending in <code>\</code> are skipped
<code>:AddRef <label> <url></code>	Add reference link at end of buffer

Other Markdown Plugins

- **render-markdown.nvim**: In-editor rendering (pauses in insert mode). Max file: 1.5MB.
- **tabular**: Table alignment. Command: `:Tabularize`
- **ltex-ls-plus (LSP)**: grammar and spell checking (LanguageTool) for markdown, tex, typst, gitcommit and text files, when `ltex-ls-plus` is installed. Problems are diagnostics: `]d / [d`, `<Space>dd`; fixes via `<Space>ca`.
- **marksman (LSP)**: link and heading navigation, completion, symbols.

28. LaTeX and Typst Support

LaTeX (`vimtex`)

Only available if `latex` is installed (vimtex also needs `latexmk` to compile).

Keymap	Description
<code><Space>rf / <F9></code>	Start / stop continuous compilation
<code>\ll</code>	Start / stop continuous compilation (same as <code><Space>rf / <F9></code>)
<code>\lv</code>	View PDF

The texlab language server adds diagnostics, hover, symbols and rename when `texlab` is on PATH (LaTeX devShell). Auto-save never saves LaTeX files.

Typst

Keymap	Description
<code><Space>tw</code>	Typst buffers: <code>:TypstWatch</code> , recompile on save and open the PDF

Needs the `typst` program (typst devShell): without it `<Space>tw` shows one warning and the Typst plugin does not load. Errors go to the quickfix list. The PDF viewer is `$TYPST_PDF_VIEWER` if set, else `zathura` if installed. LSP: `tinymist`. Typst buffers use `textwidth=100` and wrap. Auto-save never saves Typst files.

31. Spell Checking

Languages: English, Italian, German, French.

Keymap	Description
<code><Space>cz</code>	Toggle spell checking on/off
<code>]s</code> / <code>[s</code>	Next / previous misspelled word
<code>z=</code>	Show spelling suggestions (up to 9)
<code>zg</code>	Add the word to the English word list (<code>spell/en.utf-8.add</code>)
<code>2zg</code> / <code>3zg</code> / <code>4zg</code>	Add the word to the Italian / German / French list
<code>zw</code>	Mark word as wrong (same counts)
<code>zug</code>	Undo the last <code>zg</code>
<code>:e ~/.config/nvim/spell/en.utf-8.add</code>	Show the allowed words of a language (<code>it</code> , <code>de</code> , <code>fr</code> in the other files); delete a line to remove a word

`spellfile` has one word list per language, in the order of `spelllang` (`en`, `it`, `de`, `fr`): `zg` adds to the first, a count picks another. The lists are `spell/*.utf-8.add` inside the config, which is tracked in a PUBLIC repository: `spell/README.md` says these words are public, so review new words before committing. At startup Neovim silently recompiles any list whose compiled `.add.spl` file is missing or older, so the words of the tracked lists are accepted on a fresh checkout.

Besides the builtin spell checker two language servers report problems as diagnostics: `ltx_plus` (grammar and spelling in prose files) and `typos_lsp` (typos in identifiers and comments of source files). The statusline shows `[SPELL]` while spell checking is on.

38. URL & Unicode

Keymap	Mode	Description
<code>gx</code>	n, x	Open the URL or file under the cursor (<code>gx.nvim</code>)
<code>ga</code>	n	Show Unicode info for character under cursor
<code><Ctrl-x><Ctrl-z></code>	i	Complete a Unicode character by name OR <code>U+code</code>
<code><Ctrl-x><Ctrl-g></code>	i	Complete a digraph
<code><Space>cu</code>	n	Swap <code><Ctrl-x><Ctrl-z></code> between completing the character and completing its name
<code><F4></code> + motion (needs a function key; alternatives below)	n, x	Turn 2-character digraph pairs in the text into their characters

Commands: `:UnicodeSearch {name or U+hex}`, `:UnicodeName`, `:UnicodeTable`. The plugin loads on the first `ga`, `<Space>cu` or `:UnicodeSearch`; the insert keys, `<F4>` and `:UnicodeName` / `:UnicodeTable` exist only after that. Without function keys use `<Ctrl-k>` + two letters in Insert mode (built-in Vim digraph input, outside a snippet; `<Ctrl-k>` `a` `:` gives `ä`) or `<Ctrl-x><Ctrl-g>`. Example: `<F4>$` on `a:e:o:u:` gives the umlauts (per the plugin doc).

URLs in buffers are automatically highlighted (vim-highlighturl plugin).

Chapter 11: Various: custom commands, other plug-ins, configuration, Neovide

34. Custom Commands

Command	Description
<code>:CopyPath nameonly</code>	Copy filename to clipboard
<code>:CopyPath relative</code>	Copy <code><project-root>/path/to/file</code> (root = nearest <code>.git</code> or <code>pyproject.toml</code> ; a warning when there is none)
<code>:CopyPath absolute</code>	Copy absolute path
<code>:JSONFormat</code>	Format JSON (whole file or visual range)
<code>:Redir <cmd></code>	Run the command and show its output in a new tab (scratch buffer, wiped when closed)
<code>:Edit <pattern>...</code>	Open every file matching the glob patterns (<code>:Edit src/*.lua</code>); <code>:edit</code> typed as the first word expands to <code>:Edit</code>
<code>:Datetime [format]</code>	Show date and time (optional format argument)
<code>:ToPDF</code>	Convert markdown to PDF (requires pandoc)
<code>:Z {keywords}</code>	zoxide jump (see Working with Directories)
<code>:TermHL</code>	Show the current buffer (e.g. a log with ANSI colour codes) rendered with its colours in a read-only terminal buffer
<code>:LogAutocmds</code>	Toggle logging of all autocommand events to <code>~/.local/state/nvim/log-autocmds.log</code> (the file is emptied each time logging starts)
<code>:StripTrailingWhitespace</code>	Remove trailing whitespace (same as <code><Space><Space></code>)
<code>:Notifications</code>	Show the notification history (nvim-notify)
<code>:Inspect</code> / <code>:InspectTree</code>	Show the highlight groups / the Treesitter tree at the cursor

Plugin Manager Shortcuts

Type these in command mode, then press space (or Enter) to expand:

Shortcut	Expands to
<code>pi</code>	<code>:Lazy install</code>
<code>pu</code>	<code>:Lazy update</code>
<code>pc</code>	<code>:Lazy clean</code>
<code>ps</code>	<code>:Lazy sync</code>

39. Other Plugins

Plugin	Trigger	Description
<code>auto-save.nvim</code>	Automatic (active from right after the first screen)	Saves on <code>FocusLost</code> / <code>BufLeave</code> (message "AutoSave: saved at HH:MM:SS"); never saves unnamed, read-only or special buffers, nor Typst and LaTeX files
<code>better-escape.vim</code>	<code>jk</code> (insert)	Fast escape from insert mode (200ms window)
<code>vim-repeat</code>	<code>.</code>	Makes plugin actions repeatable with <code>.</code>
<code>vim-swap</code>	<code>gs</code> (n, x)	Interactively swap function arguments / list items
<code>vim-eunuch</code>	<code>:Rename</code> , <code>:Delete</code>	Unix file operations
<code>vim-obsession</code>	<code>:Obsession</code>	Session save/restore
<code>instant.nvim</code>	<code>:InstantStartServer</code> , <code>:InstantStartSession {host} {port}</code> , <code>:InstantJoinSession {host} {port}</code>	Collaborative editing (loads on its first <code>:Instant...</code> command)
<code>firenvim</code>	Browser	Neovim in browser text areas
<code>aerial.nvim</code>	<code><Space>t</code>	Symbol outline (section 37)
<code>treesj</code>	<code>gS</code>	Split / join code blocks
<code>vim-illuminate</code>	<code><Alt-n></code> , <code><Alt-p></code> , <code><Alt-i></code>	Word references: <code><Alt-n></code> / <code><Alt-p></code> jump to the next / previous one, <code><Alt-i></code> selects the reference (Visual and operator-pending mode)
<code>vimade</code>	Automatic	Dims inactive windows
<code>persistence.nvim</code>	Dashboard <code>r</code> / <code>L</code>	Saves a session per folder when you quit (see Session Management)
<code>snacks.nvim</code>	Automatic	Nicer input / select popups; light mode for big files
<code>colorful-menu.nvim</code>	Automatic	Completion labels coloured like code
<code>live-command.nvim</code>	<code>:norm</code>	Live preview of <code>:norm</code> while you type
<code>lazydev.nvim</code>	Lua files	Neovim API completion when editing the config
<code>vim-oscyank</code>	<code>:OSCYank</code>	Copy to the system clipboard through the terminal (works over SSH; Linux)
<code>vim-scriptease</code>	<code>:Messages</code> , <code>:Scriptnames</code>	Vim-script debugging helpers
<code>nvim-dbee</code> , <code>vim-dadbod-ui</code>	<code><Space>D...</code>	SQL clients (see below)

SQL Databases (nvim-dbee, vim-dadbod-ui)

Two independent SQL clients. Neither has connections by default: this repo is public, so connections are never stored in the config.

Keymap	Mode	Description
<Space>Dt	n	Dbee: toggle the UI
<Space>Do	n	Dbee: open the UI
<Space>Dc	n	Dbee: close the UI
<Space>Du	n	Dadbod: toggle the UI (connections, saved queries, results)
<Space>Da	n	Dadbod: add a connection
<Space>Df	n	Dadbod: find the DB buffer

Commands: `:Dbee`, `:DB <url> <query>` (`:%DB <url>` runs the whole buffer as a query), `:DBUI`, `:DBUIToggle`, `:DBUIAddConnection`, `:DBUIFindBuffer`.

Connections come from the environment (set them in an untracked shell file, `direnv` or `sops`, never in the repo):

- `nvim-dbee`: `$DBEE_CONNECTIONS`, a JSON array such as `[{"name": "local", "type": "postgres", "url": "postgres://user:pw@localhost:5432/db"}]`. Connections added inside the UI are not saved.
- `vim-dadbod-ui`: `$DADBOD_CONNECTIONS`, a JSON object `{"name": "url"}`. Saved queries go to `~/.local/share/nvim/db_ui`.

`:checkhealth` shows a known, accepted `vim.validate{}` deprecation warning from `nvim-dbee`. Connections come from the environment, never from the config: `$DBEE_CONNECTIONS` (JSON list, `dbee`) and `$DADBOD_CONNECTIONS` (JSON object `name -> URL`, `dadbod`), for example `export DBEE_CONNECTIONS='[{"name": "test", "type": "sqlite", "url": "/path/test.sqlite"}]'`. Tested keys:

Where	Keys	What it does
dbee (<code><Space>Do</code>)	opens four windows: drawer, SQL editor, result, call log	<code><Space>Dt</code> toggles, <code><Space>Dc</code> closes
dbee drawer	<code><CR></code> select / expand, <code>o</code> toggle, <code>r</code> refresh, <code>cw</code> rename, <code>dd</code> delete	the drawer's own help is listed inside it
dbee editor	<code>BB</code> run the whole file (Visual: the selection), <code><CR></code> run the statement under the cursor	the rows appear in the result window
dbee result	<code>L</code> / <code>H</code> next / previous page, <code>E</code> / <code>F</code> last / first page, <code>yaj</code> / <code>yac</code> yank the row as JSON / CSV, <code><C-c></code> cancel	
dadbod (<code><Space>Du</code>)	drawer: <code>o</code> open/toggle, <code>s</code> open in a vertical split, <code>R</code> redraw, <code>A</code> add a connection, <code>H</code> toggle details, <code>d</code> delete, <code>r</code> rename, <code>q</code> close, <code>?</code> help	the drawer tree: connection, New query, Saved queries, Tables
dadbod SQL buffer	<code><Space>S</code> run, <code><Space>W</code> save, <code><Space>E</code> edit bind parameters	<code>:DB sqlite:/path select ...</code> runs one query and shows the rows (tested)

In an automated test the dadbod result window opened after `<Space>S` but stayed empty, so check that step in a real terminal.

40. Configuration Management

Keymap / Command	Description
<code><Space>ev</code>	Open <code>init.lua</code> in a new tab
<code><Space>sv</code>	Write all buffers and restart Neovim (windows, tabs and files are restored; terminals such as Claude Code are not restarted). Builtin <code>zR</code> restarts without writing
<code>:Lazy</code>	Open plugin manager UI
<code>:Lazy update</code>	Update all plugins

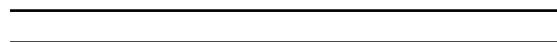
41. Filetype-Specific Settings

Filetype	Settings
Python	4-space indent, <code><Space>rf</code> (or <code><F9></code>) to run, <code><Space>f</code> to format with Black (needs <code>black</code> , <code>python devShell</code> ; one warning otherwise; in a <code>uv</code> project both use <code>uv run</code>)
Lua	<code><Space>rf</code> (or <code><F9></code>) to execute (<code>:luafile %</code>), <code><Space>f</code> and <code><Space>fm</code> format with Stylua
C++	<code><Space>rf</code> (or <code><F9></code>) to compile and run (only when a C++ compiler is on PATH, e.g. the <code>c-cpp devShell</code>)
Markdown	Word wrap enabled, syntax highlighting continues up to column 3000 on long lines; <code><Space>fm</code> formats with Prettier (one warning if <code>prettier</code> is missing); <code><Space><Space></code> does not strip trailing spaces
JSON	<code><Space>f</code> runs <code>:JSONFormat</code> on the buffer (in Visual mode on the selection)
Typst	<code><Space>tw</code> <code>TypstWatch</code> , <code>textwidth=100</code> , <code>wrap</code>
Vim script	<code><Space>rf</code> (or <code><F9></code>) sources the file
Line-length marker	The coloured column marker (<code>colorcolumn</code>) sits at 100 by default and, per language, exactly at that language's convention: 80 for C, C++, shell, YAML, Vim script, Haskell, R, JavaScript and TypeScript (also <code>jsx/tsx</code>); 88 for Python (black); 100 for Java, Rust, Swift, Nix, Typst; 120 for Lua, PHP, TeX; plain <code>.txt</code> files show none. A line touching the marker is over that language's limit. Nothing wraps or reflows.

42. Automatic Behaviors

These happen without any keypress:

Behavior	Description
Auto-create directories	Missing parent directories are created on save
Auto-resize windows	Windows resize equally when terminal is resized (the Claude Code panel goes back to 30% width)
Relative line numbers	Relative in the focused window in normal mode; absolute in insert mode and in other windows; none in terminals
Non-UTF-8 warning	Warns if file encoding is not UTF-8
Yank highlight	Yanked text highlighted for 300ms
Cursor restore	Cursor returns to original position after yank
Auto-quit	When only utility windows remain in a tab (quickfix, aerial outline, nvim-tree), the tab closes; on the last tab Neovim quits
Diagnostic float	Diagnostics auto-show when cursor rests on a line
Smart case	Case-insensitive search unless uppercase is used (only <code>/</code> and <code>?</code> ; <code>:s</code> and <code>:g</code> always ignore case)
Colorscheme	Fixed base16 theme on Nix systems (<code>NVIM_BASE16_THEME</code> , fallback Catppuccin Mocha); random on other systems
Auto-save	Files save automatically on focus lost / buffer leave (not unnamed, read-only or special buffers, not Typst/LaTeX)
File changed on disk	Checked when Neovim gets focus and when idle. An unmodified buffer is reloaded ("File changed on disk. Buffer reloaded!"); if the buffer was changed too it is kept ("File changed on disk and in the buffer (buffer kept)"); a deleted file keeps its buffer (one warning)
Format check after save	After saving a Python or Lua file, <code>black --check</code> / <code>stylua --check</code> run in the background; an unformatted file gives the warning <code><file>: file is not formatted (black) ((stylua) for Lua)</code> ; a file the tool cannot check (syntax error) gives <code><file>: <tool> could not check the file (syntax error?)</code> plus the first error line. Nothing is changed
<code>nvim <directory></code>	The directory becomes the working directory and the file tree opens there
Git plugins	fugitive and gitlinker load when the working directory or an opened file is inside a git repository; neogit loads on its first <code>:Neogit*</code> command (with diffview and fzf-lua)
Big files	Files over about 1.5 MB (or with very long lines) open in a light mode: no Treesitter, no completion, the language server starts a little later. <code>:set ft=<language></code> gives the full mode back
Help window	On a screen of at least 200 columns <code>:help</code> opens as a full-height split on the far left



Part II: Developer Guide

Everything below is aimed at developers. It explains the plugins and tools in this config that make Neovim a full development environment, what they do under the hood, why they matter, and how to use them effectively.

66. Shell Commands from Inside Neovim

Running a Shell Command

Command	What it does
<code>:!ls</code>	Run <code>ls</code> and show the output (press Enter to return)
<code>:!python %</code>	Run the current file with python (<code>%</code> is the current filename)
<code>:!git diff</code>	Run git diff without leaving Neovim
<code>:!mkdir -p src/utls</code>	Create directories

Inserting Command Output into the Buffer

Command	What it does
<code>:read !date</code>	Insert the output of <code>date</code> below the cursor
<code>:read !ls</code>	Insert directory listing into the buffer
<code>:read !curl -s <url></code>	Insert the contents of a URL
<code>:%!sort</code>	Replace the entire buffer with its sorted version
<code>:%!python -m json.tool</code>	Format the entire buffer as JSON with 4-space indent (<code>:JSONFormat</code> does the same with 2 spaces and leaves invalid JSON untouched)

Filtering a Selection Through a Command

1. Select lines with `v`
2. Type `:!sort` – the selected lines are replaced with the sorted result
3. Or `:!awk '{print $2}'` – replace with second column only

The AsyncRun Plugin

Plugin: **asyncrun.vim**. Runs commands asynchronously (non-blocking) and sends output to the quickfix list.

Command	What it does
<code>:AsyncRun make</code>	Run make in the background, results go to quickfix
<code>:AsyncRun python %</code>	Run current file, output in quickfix

The quickfix window auto-opens (6 lines tall) when AsyncRun starts.

77. Neovide (Graphical Neovim)

Neovide is a graphical program that runs Neovim in its own desktop window, instead of inside a terminal such as kitty and tmux. It is the same Neovim and the same configuration (`~/.config/nvim`); only the window around it changes. Nothing in this guide depends on it: everything works in the terminal.

Starting it

Command	What it does
<code>neovide</code>	Open an empty Neovim window
<code>neovide file.txt</code>	Open a file in a Neovide window

You start it from a terminal (or your application launcher). A Neovide window has no tmux and no terminal around it, so tmux keys do nothing there; the editor itself, `:terminal` and the Claude panel work as usual.

What this configuration sets for Neovide

The settings live in the Neovide block of `ginit.vim` in the nvim config (they apply only when Neovide runs):

Setting	Value	Effect
<code>guifont</code>	JetBrainsMono Nerd Font, size 10	The font of the window (the same font as the kitty terminal; change the name or <code>:h10</code> in the Neovide block of <code>ginit.vim</code>)
<code>neovide_transparency</code>	1.0	Opaque window (no see-through background)
<code>neovide_cursor_animation_length</code>	0.1	The cursor glides to its new place in 0.1 seconds
<code>neovide_cursor_trail_size</code>	0.3	A short trail behind the moving cursor
<code>neovide_cursor_vfx_mode</code>	empty	No particle effects around the cursor

Change a value by editing that block; the `neovide_...` names are the standard Neovide options.

Keys that only exist in a GUI

A terminal swallows some key combinations, so `ginit.vim` adds them for GUIs (`:imap`, `:cmap` and `:nmap` show them with a description ending in "(GUI)"):

Keys	Mode	What it does
<code><Shift-Insert></code>	Insert, command line	Paste the system clipboard
<code><Ctrl-6></code>	Normal	Jump to the alternate (previously open) buffer, like <code><Ctrl-^></code>

`ginit.vim` also has small blocks for other graphical frontends (nvim-qt and fvim); they do nothing in Neovide.

Chapter 12: Java

78. Java (nvim-java, jdtls, tests, debugging)

This section is one complete walk through everything that is specific to Java in this config: the tools, why each one is there, how it works, how to use it, and what to do when it does not work. It covers only Java-only things (nvim-java and its parts, the language servers, the test runner, the debugger, the Java keys, commands and snippets). Global tools with global keys (`gd`, `K`, `<Space>rn`, `git`, `fzf`, ...) get only a pointer to their own section. Sections 35 and 54 hold the short key lists and section 56 the debugging basics; this section puts everything in one order.

Everything marked "tested" below was tried with real keys in a Maven test project (JDK 25 devShell, `Main`, `Calc`, `CalcTest`).

1. What You Get And Why

Part	Why it is in the setup	What it does	Where it comes from
nvim-java	One plugin that wires all the Java pieces together	Starts jdtls, finds the debugger and test extensions, adds the <code>:Java*</code> commands	lazy plugin <code>nvim-java/nvim-java</code> , loaded on <code>ft = "java"</code>
jdtls	Java has no useful editing without a real compiler front end	Completion, diagnostics (compile errors while you type), go to definition, rename, code actions, formatting, refactoring	nvim-java's own jdtls 1.54.0, in <code>~/.local/share/nvim/nvim-java/packages/jdtls</code>
spring-boot	Understands Spring annotations and <code>application.properties</code>	A second language server that attaches next to jdtls	nvim-java, only started when <code>java</code> is on PATH
java-debug (DAP adapter)	Without it the debugger cannot talk to the JVM	Lets nvim-dap debug Java	VS Code extension <code>vscjava.vscode-java-debug</code> , linked by the devShell
java-test	Runs JUnit tests and reports per-test results	<code><Space>jt...</code> keys	VS Code extension <code>vscjava.vscode-java-test</code> , linked by the devShell
nvim-dap	The general debugger client (breakpoints, stepping)	<code>:Dap*</code> commands	plugin <code>mfussenegger/nvim-dap</code> , a dependency of nvim-java
Lombok	Many Java projects use it; jdtls must load it as an agent	Annotations like <code>@Getter</code> compile	<code>lombok</code> package of nvim-java, and <code>JAVA_TOOL_OPTIONS</code> of the devShell
typos_lsp	Spell checker for code (global tool, not Java-only)	Underlines misspelled words in identifiers and comments, also in Java	<code>typos-lsp</code> from the Nix system
Snippets	Typing the same loops and switches over and over	See part 9 below	<code>my_snippets/java.snippets</code>

Tools you get only through the devShell: the JDK (25), `mvn`, `gradle`, `lombok`, `jdtls` wrapper, and the two VS Code extensions. The Neovim config itself never installs them.

How nvim-java is loaded

The real spec (`lua/plugin_specs.lua`):

```

{
  "nvim-java/nvim-java",
  ft = "java",
  dependencies = { "MunifTanjim/nui.nvim", "mfussenegger/nvim-dap" },
  config = function()
    local is_nix_managed = vim.uv.fs_stat("/etc/nixos") or vim.uv.fs_stat("/etc/nix")
    local has_java = vim.fn.executable("java") == 1
    require("java").setup({
      jdk = { auto_install = not is_nix_managed },
      java_test = { enable = true },
      java_debug_adapter = { enable = true },
      spring_boot_tools = { enable = has_java },
    })
    vim.lsp.config("jdtls", { settings = { java = { home = vim.env.JAVA_HOME } } })
    if has_java then vim.lsp.enable("jdtls") end
  end,
}

```

In plain words:

- `ft = "java"`: nvim-java loads when you open the first Java file of the session, not at startup. Non-Java sessions pay nothing. The very first Java file of a session opens a little slower.
- On a Nix system `jdk.auto_install` is false: nvim-java never downloads a JDK. The JDK comes from the devShell (`JAVA_HOME`).
- `jdtls` and `spring-boot` start only if `java` is on `PATH`. That is how the config stays silent outside the devShell.
- `java.home = $JAVA_HOME` tells `jdtls` which JDK to compile with.
- `jdtls` itself is NOT taken from the devShell (the devShell `jdtls` is only a wrapper script); nvim-java uses its own `jdtls` 1.54.0.

The Java devShell (`use_dev_env java`)

A Java project has a `.envrc` with one line:

```
use_dev_env java
```

`direnv` loads the devShell when you `cd` into the folder. The devShell (template `dev-environments/language-specific/java/flake.nix`) gives you `JAVA_HOME` (JDK 25), `mvn`, `gradle`, Lombok (`JAVA_TOOL_OPTIONS` has a `-javaagent:` for it) and, in its shell hook, links the extensions for Neovim:

```

~/local/share/nvim/nvim-java/packages/java-debug-adapter/extension -> vscode-java-debug
~/local/share/nvim/nvim-java/packages/java-test/extension           -> vscode-java-test
./direnv/tools/jdtls                                               -> jdt-language-server

```

`.direnv/` is ignored by `git`, so nothing shows up in `git status`. First time in a new folder: `direnv allow`.

Outside the devShell

`java` is not on `PATH`, so `jdtls` and `spring-boot` do not start. `.java` files still open with syntax highlighting, snippets and `typos_lsp`, but without Java language features. Every `<Space>j...` key shows ONE warning, `Java: jdtls not attached (open nvim inside the Java devShell)`. These global keys are fall-

backs: without them `<Space>jrr` would fall through to plain Vim keys (`<Space>`, `j`, `rr`). The universal run key `<Space>rr` (section 19) then runs plain `java <file>` if `java` exists, otherwise gives one warning.

2. Quick Start

1. In a terminal: `cd` into the project folder (the one with `pom.xml`). `direnv` loads the `devShell` and prints `Java Environment Active (JDK 25)`. First time: `direnv allow`.
2. `nvim src/main/java/demo/Main.java`
3. Wait about 10 seconds. The statusline shows `jdtls (+2)` (`jdtls`, `spring-boot`, `typos_lsp`). Tested: all three attach: `jdtls,spring-boot,typos_lsp`. Before that the `<Space>j` keys only show the warning. A big project's first start is slower.
4. Run: `<Space>jrr`. A full-width, 15-line terminal split opens at the bottom with the program output.
5. Test: open `CalcTest.java` and press `<Space>jtc`, then `<Space>jtr` for the result tree.
6. Debug: `:DapToggleBreakpoint` on a line, then `<Space>jtC`. The window stops at the line with a `-` sign.

3. Project Layout Rules

The root folder

`jdtls` needs one root folder per project. `nvim-java` decides it with these markers (`lua/java-core/ls/servers/jdtls/root.lua`), two groups, the first group that matches wins:

Group	Markers	Meaning
1	<code>mvnw</code> , <code>gradlew</code> , <code>settings.gradle</code> , <code>settings.gradle.kts</code> , <code>.git</code>	multi-module projects; the git root is the last resort on purpose
2	<code>build.xml</code> , <code>pom.xml</code> , <code>build.gradle</code> , <code>build.gradle.kts</code>	single-module projects

Consequences:

- The git root wins over a `pom.xml` below it, by design: in a multi-module Maven project you cannot tell the parent from a submodule, and `jdtls` does not break when the root is higher than the real parent `pom.xml`.
- A Maven project nested in a bigger git repository (`repo/java/pom.xml`) still works: the root is the git root and `jdtls` finds the `pom.xml` below it. `<Space>jrr` works in this layout (tested by the user's real layout).
- No git folder: the folder with `pom.xml` is the root. A loose `.java` file with neither: `jdtls` has no project (create a `pom.xml` or run `git init`).
- Maven and Gradle projects are both imported by `jdtls`.
- Expected layout: code in `src/main/java/...`, tests in `src/test/java/...`.

First start and where files appear

- The first start of a project is slower: `jdtls` imports the project, runs Maven or Gradle and builds an index. The statusline and notifications show progress.
- Compiled classes go to `target/` (Maven) next to `pom.xml`. `jdtls` also writes Eclipse files into the project (`.project`, `.classpath`, `.settings/`). In the test project `.gitignore` contains only `target/` and `.direnv/`, so `.project`, `.classpath` and `.settings/` show up in `git status`; add them to the project's `.gitignore` if you do not want them in git.

- `jdtls` workspace data goes to `~/.cache/nvim/jdtls/` (`config_<hash>` and `workspace/proj_<hash>`), never into the project. The workspace folder name is a hash of the folder where Neovim was STARTED (`vim.fn.getcwd()` at startup), not of the `jdtls` root. Always start Neovim from the same project folder, otherwise `jdtls` builds a second workspace.
- Saving: the auto-save plugin saves a buffer when you leave it or when Neovim loses focus. Refactor results and edits are therefore written to disk without `:w` once you switch windows (tested: a refactor was saved this way).

4. Keys And Commands

All `<Space>j` keys are normal mode only. The real maps are created per buffer when `jdtls` attaches and removed when it detaches (`LspAttach` / `LspDetach` in `lua/mappings.lua`); before that, the global fallback shows the warning. which-key shows the groups `j` Java, `jb` build, `jr` runner, `jt` test, `je` extract.

Build (`<Space>jb`)

Key	Command	What it does	Mode
<code><Space>jbb</code>	<code>:JavaBuildBuildWorkspace</code>	Compile the whole workspace; a "Background task" notification shows while it runs (tested)	n
<code><Space>jbc</code>	<code>:JavaBuildCleanWorkspace</code>	Delete the <code>jdtls</code> workspace cache of the project after a Yes/No question, then restart <code>jdtls</code> (see below)	n

Runner (`<Space>jr`)

Key	Command	What it does	Mode
<code><Space>jrr</code>	<code>:JavaRunnerRunMain</code>	Run the main class	n
<code><Space>jrs</code>	<code>:JavaRunnerStopMain</code>	Stop the running program (tested: the java process is gone and the runner shows <code>Process finished with exit code::129</code>); with nothing running it does nothing visible	n
<code><Space>jrl</code>	<code>:JavaRunnerToggleLogs</code>	Show or hide the runner log window (tested: toggles between 1 and 2 windows)	n
<code><Space>jrp</code>	<code>:JavaProfile</code>	Run profiles window (main class and arguments), see below	n

Test (`<Space>jt`)

Key	Command	What it does	Mode
<code><Space>jtc</code>	<code>:JavaTestRunCurrentClass</code>	Run all tests of the class in the buffer	n
<code><Space>jtm</code>	<code>:JavaTestRunCurrentMethod</code>	Run only the test method under the cursor	n
<code><Space>jtr</code>	<code>:JavaTestViewLastReport</code>	Show the result tree of the last run	n
<code><Space>jtC</code>	<code>:JavaTestDebugCurrentClass</code>	Debug the class (breakpoints stop)	n
<code><Space>jtM</code>	<code>:JavaTestDebugCurrentMethod</code>	Debug the test method under the cursor	n

Refactor / extract (<Space>je)

Key	Command	What it does	Mode
<Space>jev	:JavaRefactorExtractVariable	Extract the expression to a local variable	n
<Space>jeo	:JavaRefactorExtractVariableAllOccurrences	Same, and replace ALL occurrences	n
<Space>jec	:JavaRefactorExtractConstant	Extract a constant	n
<Space>jem	:JavaRefactorExtractMethod	Extract a method	n
<Space>jef	:JavaRefactorExtractField	Extract a field	n

Settings and debugger setup

Key	Command	What it does	Mode
<Space>jd	:JavaDapConfig	Configure the debug adapter again (it is done automatically when jdtls starts)	n
<Space>jj	:JavaSettingsChangeRuntime	Pick another JDK runtime for jdtls; only works if runtimes are configured, see below	n

The :Java* commands

Tested with :command Java after jdtls attached:

- Existing at that point: JavaBuildBuildWorkspace, JavaBuildCleanWorkspace, JavaRefactorExtract... (five), JavaRunnerRunMain, JavaRunnerStopMain, JavaRunnerToggleLogs, JavaRunnerSwitchLogs, JavaTestRunCurrentClass, JavaTestRunCurrentMethod, JavaTestRunAllTests, JavaTestDebugCurrentClass, JavaTestDebugCurrentMethod, JavaTestDebugAllTests, JavaTestViewLastReport, JavaDapConfig, JavaProfile, JavaSettingsChangeRuntime.
- Three have no key: :JavaRunnerSwitchLogs, :JavaTestRunAllTests (all tests of the project) and :JavaTestDebugAllTests.
- Only :JavaBuild* and :JavaRefactor* appear after jdtls attached; the other :Java* commands exist as soon as nvim-java is loaded but do nothing useful without jdtls.
- :JavaRefactor... commands accept a range, so :<,>JavaRefactorExtractMethod is valid.

The runner window

<Space>jrr opens the runner as a full-width terminal split at the bottom (tested: 14 text rows plus the status line, 200 columns wide at 200 columns). It prints the real command, then the program output, then the exit code. Tested output for Main:

```
/nix/store/...-openjdk-25.0.2+10/lib/openjdk/bin/java -cp ../target/classes demo.Main
Picked up JAVA_TOOL_OPTIONS: -javaagent:../lombok.jar
result: 20
Process finished with exit code::0
```

(The Picked up JAVA_TOOL_OPTIONS line comes from the devShell's Lombok setting; it is harmless.) <Space>jrl hides and shows this window again. To go back to the code: <Esc>, then <Ctrl-w>k or <Up>.

Profiles window (<Space>jrp)

A profile stores VM arguments and program arguments for the runner of one main class. Tested flow:

1. <Space>jrp opens a window Profiles with the entry New Profile and the hint [a]ctivate [d]elete [b]ack [q]uit.
2. <Enter> on New Profile opens a form with three boxes: Name, VM arguments, Program arguments, and the hint [s]ave [b]ack [q]uit.
3. In the form (normal mode): i types in the box, <Esc> leaves it, <Tab> or j goes to the next box, k to the previous one, s saves.
4. Saved example: name prof1, program arguments hello. The profile is stored in ~/.local/share/nvim/nvim-java-profiles.json, keyed by project folder and main class, and marked active:

```
{"/path/to/project": {"nvim-java-test -> demo.Main": [{"name": "prof1", "is_active": true, "prog_args": "hello", "vm_args": ""}]}}
```

5. The next <Space>jrr uses it. Tested: the runner printed ... demo.Main hello.
6. To switch or remove profiles: <Space>jrp, choose the profile, a activates it, d deletes it, q closes.

Clean workspace (<Space>jbc)

1. <Space>jbc opens a Yes/No list with the question Do you want to delete ".../.cache/nvim/jdtls/workspace/proj_<hash>".
2. <Enter> on 1. Yes deletes that folder and restarts jdtls (tested: the folder was recreated fresh a moment later with a new timestamp, jdtls came back). No does nothing.
3. Your source files and target/ are not touched.
4. The folder named in the question is computed from jdtls's root folder, while jdtls really uses a folder named from the folder where Neovim was started. They are the same when you start Neovim in the project root (the case of a project that is its own git root). If your git root is above the folder where you start Neovim, the question may name a folder that does not exist; then delete the real one by hand: rm -r ~/.cache/nvim/jdtls/workspace (from the nvim-java source, not tested in a nested layout).

Change runtime (<Space>jj)

Tested: in the test project it only shows a notification: No configured runtimes available plus a link to the nvim-java README. The list of JDKs comes from jdtls setting java.configuration.runtimes, which this config does not set (only java.home = \$JAVA_HOME). So <Space>jj is a no-op here; the JDK in use is the devShell JDK.

5. Language Server Features In A Java Buffer

All need jdtls attached. The general keys are described elsewhere (section 13 for LSP); here is what they do in Java.

Key / command	What it does in Java
<code>gd</code>	Go to definition; several results open the location list
<code>K</code>	Hover: type and Javadoc. Tested: a hover float shows a "java" progress bar while loading
<code><Space>rn</code>	Rename a class, method or variable in all files
<code><Space>ca</code>	Code actions: quick fix, organize imports, generate getters, constructors, <code>toString</code> ; also the way to extract with a selection (see Refactoring)
<code><Space>fm</code>	Format the file with the jdtls formatter (Eclipse style). Tested: <code>calc.add(2,3)*4</code> became <code>calc.add(2, 3) * 4</code> . Formatting is never automatic
<code>:LspInlayHints enable / disable</code>	Inlay hints (parameter names, types); off by default (tested: enable turns hints on)
<code><Space>t</code>	Symbol outline (aerial); tested in Java: shows <code>Main</code> and <code>main</code>
diagnostics	Compile errors and warnings appear while you type, without running anything

Other notes:

- **illuminate**: other uses of the word under the cursor are highlighted (Java is in its filetype list).
- **typos_lsp**: attaches to Java buffers too (root markers include `.gitignore`), also outside the devShell. If `<Space>rn` or `<Space>ca` only warn "no attached language server supports it", only `typos_lsp` is attached and `jdtls` is missing.
- **Completion** and **snippets** work as in other languages (sections 14 and 15).
- The coloured line marker (`colorcolumn`) is at 100 for Java.
- spring-boot attaches next to `jdtls` in every Java buffer, even in a project without Spring; it is harmless.

6. Tests (JUnit)

Why and how

java-test knows JUnit 4 and 5 and TestNG. nvim-java asks `jdtls` for the test classes and methods, then starts the tests through the debug adapter machinery. That is why the test terminal is called `[dap-terminal]` and why debugging a test works with the same keys.

Using it

Example test (`src/test/java/demo/CalcTest.java`):

```
class CalcTest {
    @Test
    void addWorks() {
        assertEquals(5, new Calc().add(2, 3));
    }

    @Test
    void multiplyWorks() {
        assertEquals(6, new Calc().multiply(2, 3));
    }
}
```

```
}  
}
```

1. `<Space>jtc` anywhere in the class runs both tests; with the cursor inside one method, `<Space>jtm` runs only that one.
2. A terminal window named `[dap-terminal] Launch All Java Tests` opens at the bottom (the tab line also shows it). Tested content: only the start lines (`Picked up JAVA_TOOL_OPTIONS...`) and `[Process exited 0]`. The terminal does NOT print pass or fail text.
3. The results are in the report: `<Space>jtr` opens a floating window with a tree. Tested, all passing:

```
demo.CalcTest  
  addWorks(demo.CalcTest)  
  multiplyWorks(demo.CalcTest)
```

Each line has a status icon in front (a different icon for pass and fail). Tested with a wrong expected value (`assertEquals(7, ...)`): the failing method gets the failure message and stack below it:

```
multiplyWorks(demo.CalcTest)  
org.opentest4j.AssertionFailedError: expected: <7> but was: <6>  
at org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:531)  
at demo.CalcTest.multiplyWorks(CalcTest.java:15)
```

4. Close the report with `<Esc>` or `q` (both tested). `<Space>jtr` again after a new run shows the new result. The exit code in the terminal is 0 even when tests fail; trust the report.

Needs

- A JUnit dependency in `pom.xml` (the test project: `junit-jupiter` 5.10.2 and `maven-surefire-plugin` 3.2.5, release 21).
- `jdtls` attached and the `java-test` extension linked by the `devShell` (see section 1).
- Network the first time Maven has to download the JUnit jars into `~/.m2`.

7. Debugging

Why and how

`nvim-dap` is the debugger client; `java-debug` is the adapter that talks to the JVM; `nvim-java` registers the Java configurations for you. This config has NO dap keymaps and NO dap-ui panel (no variables or stack window). You work with `:Dap*` commands, the sign column and small floating windows. Tested end to end:

Step	How	Tested result
Breakpoint on / off	<code>:DapToggleBreakpoint</code> on the line (put it on a line with code, e.g. the <code>assertEquals</code> line, not on the <code>void addWorks() {</code> line)	A <code>B</code> sign in the sign column
Debug a test class	<code>:DapToggleBreakpoint</code> , then <code><Space>jtC</code>	After a few seconds the window shows a <code>→</code> sign on the breakpoint line; the program is paused
Debug one test method	<code><Space>jtM</code> with the cursor in the method	same, only that method
Debug the program (<code>main</code>)	<code>:DapToggleBreakpoint</code> in <code>Main.java</code> , then <code>:DapContinue</code>	A picker "Configuration" opens (tested: 5 identical entries <code>nvim-java-test -> demo.Main</code>); press <code><Enter></code> on the first: the program stops at the breakpoint with <code>→</code>
Step over	<code>:DapStepOver</code>	<code>→</code> moves to the next line
Step into	<code>:DapStepInto</code>	On a call to your own method it opens that file (tested: <code>calc.add(2, 3)</code> opens <code>Calc.java</code> at <code>int sum = a + b;</code>). On a line without a call, or into library code, it opens an empty <code>unknown</code> buffer and a DAP warning "Adapter reported frame ... Invalid cursor line"
Step out	<code>:DapStepOut</code>	Back to the caller
Look at a value	put the cursor on the variable, <code>:lua require("dap.ui.widgets").hover</code>	A small float with the value (tested: <code>sum</code> shows <code>5</code>). <code><Esc></code> closes it
Continue	<code>:DapContinue</code> while paused	Runs to the next breakpoint or to the end (tested: the test terminal shows <code>[Process exited 0]</code>)
Debug console	<code>:DapToggleRepl</code>	A <code>[dap-repl-N]</code> window; run again to hide
Stop	<code>:DapTerminate</code>	The session ends (tested: no session afterwards)

Notes:

- Without a breakpoint, `<Space>jtC` simply runs the tests to the end.
- The debugged program's output goes to the `[dap-terminal]` window.
- `<Space>jrr` runs without a debugger; to debug `main` use `:DapContinue` as above.

- If a debug window stays open after `:DapTerminate`, close it with `<Space>q` in that window.
- See section 36 and section 56 for the general DAP notes.

8. Refactoring (Extract)

How it works: each `<Space>je...` key runs `vim.lsp.buf.code_action` filtered to one kind (`refactor.extract.variable`, `.constant`, `.function`, `.field`). So `<Space>je...` and `<Space>ca` use the same jdtls actions; the keys just pick the right one for you.

The cursor or selection decides what is extracted. After the action a small `New Name` window opens with a proposed name (tested: `i`, `_4`, `extracted`). The change is ALREADY applied; the window renames it:

- `<Enter>` accepts the name in the box (to rename, delete the text with `<BS>` first, then type the new name; it must be a valid Java identifier or you get "is not a valid Java identifier").
- `<Esc>` closes the window and keeps the proposed name.

Undo needs `u` twice (once for the rename step, once for the extraction; tested).

Try-it code (`Main.java`), tested results with the cursor on the shown part:

```
Calc calc = new Calc();
int total = calc.add(2, 3) * 4;
String label = "result";
System.out.println(label + ": " + total);
```

Key	Cursor	Tested result
<code><Space>jev</code>	on <code>calc.add(2, 3)</code>	<code>int i = calc.add(2, 3);</code> and <code>int total = i * 4;</code> (the <code>New Name</code> box proposes <code>i</code>)
<code><Space>jeo</code>	on an expression that appears several times	same, but every identical occurrence uses the new variable
<code><Space>jec</code>	on the <code>4</code>	<code>private static final int _4 = 4;</code> above the method and <code>... * _4;</code> (rename to <code>FACTOR</code> gives <code>private static final int FACTOR = 4;</code>)
<code><Space>jem</code>	on the identifier <code>calc</code> (no selection)	extracts only that identifier: <code>extracted(calc).add(2, 3)</code> and <code>private static Calc extracted(Calc calc) { return calc; }</code>
<code><Space>jem</code>	selection, see below	extracts the selected statements into a method
<code><Space>jef</code>	on an expression	extracts to a field (same flow, not tested separately)

Extracting a whole statement or block into a method needs a selection, and the key is normal mode only:

1. Select with `v` (characterwise), for example `v$` on the `System.out.println(...)` line (tested; `v` linewise gave a worse result: only `System.out` was extracted).
2. Press `<Esc>` (the selection marks stay).

3. <Space>jem

Tested result:

```
        extracted(total, label);
    }

    private static void extracted(int total, String label) {
        System.out.println(label + ": " + total);
    }
}
```

Pressing <Space>jem while the selection is still active does NOT work (visual mode has no such key; <Space>ca there would even run c, change, on the selection).

9. Snippets

Source: my_snippets/java.snippets. Type the trigger in insert mode in a Java buffer and expand it with <Ctrl-j> (section 15); <Ctrl-j> / <Ctrl-k> jump to the next / previous placeholder.

Trigger	Result
fdijscanner	package fundamentidiinformatica.X; , the Scanner import and a class with main reading one line from System.in
jarr	int[] arrayName = new int[size];
jarrlit	int[] arrayName = {value1, value2, value3};
jdict	HashMap<String, Integer> mapName = new HashMap<>();
jdictfull	The same with import java.util.HashMap; and a put(key, value) line
jfor	for (int i = 0; i < length; i++) { }
jforeach	for (String item : collection) { }
jwhile	while (condition) { }
jdowhile	do { } while (condition);
jif	if (condition) { }
jifelse	if ... else ...
jifelif	if ... else if ... else ...
jswitchtraditional	Classic switch with case, break, default
jswitchmulti	Classic switch, several case labels per result (months to seasons)
jswitcharrow	switch with case x -> result;
jswitcharrowmulti	Arrow switch, several values per case
jswitchyield	Switch expression assigned to a variable
jswitchyieldblock	Switch expression with a default -> { ...; yield ...; } block
jtrycatch	try { } catch (Exception e) { }
jtryfinally	try / catch / finally
jwhilescannerbreak	Scanner loop that reads values until a stop value, then closes the scanner

10. Windows While Running, Testing And Debugging

Keys	What it does
<code><Ctrl-w>h j k l</code>	Move between windows: code, the runner or test terminal at the bottom, the report
<code><Left> <Down> <Up> <Right></code>	The same with arrow keys (normal mode)
<code><Esc></code> first	In a terminal window press <code><Esc></code> to get to normal mode, then use a window key. <code>i</code> types in the terminal again
<code><Space>q</code>	Save if modified and close the current window (use it in a runner, test or debug terminal)
<code><Ctrl-w>o</code>	Keep only the current window
<code><Esc></code> in a float	Closes the report float, the Profiles window, <code>New Name</code> box and hover floats (section 7 lists <code><Esc></code> closing floats)

See section 7 (windows) and the cheat sheet in section 2 for the same rows.

11. Troubleshooting

Symptom	Likely cause	Fix
<code><Space>j...</code> shows Java: jdtls not attached (open nvim inside the Java devShell)	java is not on PATH (not in the devShell) or jdtls has not finished starting	Quit, <code>cd</code> into the project, check <code>direnv allow</code> and which java, start nvim again; inside the devShell wait about 10 s and check <code>:LspAttached</code>
jdtls never attaches	Neovim started before direnv loaded; jdtls crashed	<code>:LspLog</code> , <code>:checkhealth vim.lsp</code> , <code>:edit</code> the file; check which java
Tests or debugger do not start, adapter not found	The devShell was not entered since the extensions were linked, or the links broke	Enter the project with direnv once (the shell hook relinks <code>~/.local/share/nvim/nvim-java/packages/java-debug-adapter/extension</code> and <code>java-test/extension</code>), restart nvim
<code><Space>jrr</code> prints nothing	jdtls not finished importing, or the file is not under <code>src/main/java</code> , or no <code>main</code> method	Wait for the first import, <code><Space>jbb</code> , then <code><Space>jrr</code> again; a project that is its own git root and one nested in a bigger git repo both work
Old errors or deleted classes still shown	Stale jdtls workspace	<code><Space>jbc</code> then Yes (jdtls restarts); if it stays wrong: <code>rm -r ~/.cache/nvim/jdtls/workspace</code>
A different, empty workspace appears after starting from another folder	The workspace folder name is a hash of the START folder of Neovim	Always start nvim in the same project folder
Test terminal shows no pass/fail	By design: results are in the report	<code><Space>jtr</code>
First test run fails with a download error	Maven offline: JUnit jars not in <code>~/.m2</code> . Tested in a shell with an empty repo: <code>mvn -o test</code> stops with Cannot access central (https://repo.maven.apache.org/maven2) in offline mode and the artifact org.apache.maven.plugins:maven-resources-plugin:jar:3.3.1 has not been downloaded from it before	Connect once, run <code><Space>jtc</code> again (or <code>mvn test</code> in a shell)
Java version mismatch: JDtls ... requires Java ...	The JDK on PATH is too old or too new for jdtls 1.54.0	Use the devShell JDK (25)
"release version N not supported" when compiling	<code>maven.compiler.release</code> in <code>pom.xml</code> is higher than the JDK	Lower it (the test project uses 21 on JDK 25)
<code>.java</code> file in a scratch folder: no Java features	No project root (no <code>pom.xml</code> , <code>.git</code>)	Create <code>pom.xml</code> or <code>git init</code>
Debugger does not stop <code><Space>jj</code> does not stop, says, No configured runtimes available server supports it"	No breakpoint, or the breakpoint is on a line without code Neovim User Guide · page 148	<code>:DapToggleBreakpoint</code> on a line with code, then <code><space>jtc</code> fix <code>ignore {tend-words}</code> in the project root

Related Sections

Section 35 (Java keys), 54 (Java in depth), 36 and 56 (debugging), 19 and 55 (running code, `<Space>rr`), 13 (LSP), 14 (completion), 15 (snippets), 7 (windows), 8 (terminal), 20 (git; the git root decides the jdtls root), 2 (quick reference and cheat sheet).

Chapter 13: Python

79. Python (pyright, ruff, black, uv, running and debugging)

This section is one walk-through for everything Python in your config: what starts by itself, how to run, format and debug a file, how virtual environments are detected, and what to check when something does not work. Deeper background on the shared parts lives in the sections named in "Related sections" at the end.

What You Get

Feature	What it does	Needs
pyright (language server)	Type checking, import resolution, hover (<code>K</code>), go to definition (<code>gd</code>), rename (<code><Space>rn</code>), references. Mode is <code>standard</code> , whole workspace is checked. Its own import sorting is turned off (ruff does that)	<code>pyright-langserver</code> on PATH (installed globally by <code>neovim.nix</code> , also in the <code>python devShell</code>)
ruff (language server, <code>ruff server</code>)	Fast linting (diagnostics), quick fixes and "organise imports" through <code><Space>ca</code> . It is the server that offers formatting for <code><Space>fm</code>	<code>ruff</code> on PATH (python and jupyter devShells; NOT global)
black (formatter)	<code><Space>f</code> formats the file; after every save a background check warns when the file is not formatted	<code>black</code> on PATH (python devShell), or a uv project that has black (see Running)
typos_lsp	Spell checker for identifiers and comments, attaches to every file type (also Python)	<code>typos-lsp</code> on PATH (installed globally by <code>neovim.nix</code>)
tree-sitter	Syntax highlighting (the <code>python</code> parser)	Nothing on Nix systems (parsers come from the nix store)
Snippets	<code>print</code> , <code>impa</code> , <code>main</code> , <code>sol</code> (see Snippets below)	Nothing
uv awareness	In a project with <code>uv.lock</code> and no active virtual environment, <code><Space>rf</code> / <code><F9></code> and <code><Space>f</code> go through <code>uv run</code>	<code>uv</code> on PATH (it is: <code>/run/current-system/sw/bin/uv</code> , tested)
Statusline label	Shows the active environment as <code>name (venv)</code> or <code>name (conda)</code> in Python buffers	An activated environment
Format check	After saving, <code>black --check</code> runs in the background and warns <code><file>: file is not formatted (black)</code>	<code>black</code> on PATH, otherwise silent
Run	<code><Space>rf</code> / <code><F9></code> (output in the quickfix window) and <code><Space>rr</code> (output in a terminal split)	<code>python</code> / <code>python3</code> on PATH
Debug	<code><Space>dp</code> starts <code>python -m pdb</code> inside nvim-gdb	Linux or Windows
vim-illuminate	Other uses of the word under the cursor are highlighted (Python is in its file)	Nothing
highlight	Synonym for <code>vim-illuminate</code> ; of classes, <code>colorcolumn</code> under methods (<code><Space>t</code>)	Nothing

Quick Start

1. Start nvim **inside the Python devShell** (a project folder with `.envrc` containing `use_dev_env python`, see "Devshell and Tools"). Outside it, only `pyright` and `typos_lsp` exist.
2. Open a file: `nvim hello.py`. Wait a second or two. `:LspAttached` lists the running servers; tested in the python devShell: `pyright`, `ruff`, `typos_lsp`. The statusline shows `pyright (+2)` (the first server and two more).
3. Type a small program, save with `:w`, then run it with `<Space>rf` (or `<F9>`). A 6-line quickfix window opens at the bottom and shows the output.
4. Run it again as a full terminal with `<Space>rr`. A terminal split opens on the LEFT of the code.
5. Format with `<Space>f` (black). The file on disk is rewritten and the buffer reloads (`File changed on disk. Buffer reloaded!`).
6. Debug with `<Space>dp`. The code gets a  marker and a `pdb` pane opens below it. Step with `<Space>dn` (or `<F10>`), quit with `:GdbDebugStop`. (`<F9>` and `<Space>rf` keep working afterwards; tested.)

Why Each Tool Exists

Tool	Why it is in your setup	How it works
pyright	Python does not check types by itself. Pyright finds wrong argument types, missing imports and typos in attribute names before you run the file	A language server (<code>pyright-langserver --stdio</code>). Neovim starts it when you open a <code>.py</code> file and it answers questions: errors, hover, definition, rename. Installed globally, so it also works outside a devShell
ruff	Fast linter (unused imports, undefined names, style problems) with automatic fixes and import sorting	<code>ruff server</code> is a second language server in the same buffer. Its diagnostics appear next to pyright's, and <code><Space>ca</code> lists its fixes
black	One fixed code style, so you never decide about spaces or quotes	A command-line formatter. <code><Space>f</code> runs it on the file, and a background check runs after each save
uv	Fast Python project and package manager. Projects managed by uv have a <code>uv.lock</code> file	The config checks for <code>uv.lock</code> and then runs <code><Space>rf / <F9></code> and <code><Space>f</code> through <code>uv run</code> , so they use the project's own environment
venv / conda label	Tells you which environment nvim (and so pyright) was started with	Reads <code>\$VIRTUAL_ENV</code> / <code>\$CONDA_DEFAULT_ENV</code>
direnv + devShell	Gives each project its own tools without installing them globally	<code>.envrc</code> with <code>use_dev_env python</code> loads the nix devShell; start nvim from that folder
nvim-gdb	Step through a script with pdb (Python's built-in debugger) with the current line marked in your code	Starts <code>python -m pdb file.py</code> in a terminal pane and talks to it
AsyncRun	Run a script without leaving the editor and read the output in the quickfix window	<code><Space>rf / <F9></code> runs a shell command as a background job and streams its output into quickfix
typos_lsp	Catches misspelt words inside names and comments	A language server for every file type
tree-sitter, aerial, illuminate, treesj, ufo	General tools that also work for Python (highlighting, outline, word highlight, split/join, folding)	See the sections in "Related Sections"

Python-specific configuration lives in only a few places:

File	What it holds
<code>after/ftplugin/python.lua</code>	indentation, no wrapping, <code><Space>rf / <F9></code> , <code><Space>f</code> , <code><Space>dp</code>
<code>after/lsp/pyright.lua</code> , <code>after/lsp/ruff.lua</code>	server settings
<code>lua/config/lsp.lua</code>	which servers exist and when they are enabled
<code>my_snippets/python.snippets</code>	snippets
<code>lua/custom-autocmd.lua</code>	the after-save format check (<code>black --check</code>)
<code>lua/options.lua</code>	<code>colorcolumn 88</code> for Python

Devshell and Tools

Python tools are not installed globally except two. Open nvim in a devShell to get the rest.

Tool	Global (<code>neovim.nix</code>)	python devShell	jupyter devShell
<code>pyright</code>	yes	yes	no (uses the global one)
<code>typos-lsp</code>	yes	no (uses the global one)	no (uses the global one)
<code>ruff</code>	no	yes	yes
<code>black</code>	no	yes	no
<code>uv</code>	yes, system-wide (which <code>uv</code> shows <code>/run/current-system/sw/bin/uv</code> ; <code>python3</code> is also on the system PATH)	not added	not added
<code>python / python3</code>	system <code>python3</code> exists	yes (version depends on the variant)	python 3.13

Templates live in `~/nix/templates/krit/dev-environments/language-specific/`:

Template	What it gives you
<code>python</code>	Python, black, flake8, isort, pip, matplotlib, pylint, setuptools, numpy, pyright, ruff, and <code>venvShellHook</code> which creates <code>.venv</code> in the project folder (<code>venvDir = ".venv"</code>). It warns when <code>.venv</code> was built with a different Python version than the shell (delete <code>.venv</code> and reload)
<code>jupyter</code>	poetry, Python 3.13, ruff, ipykernel, pip, <code>venvShellHook</code> (also <code>.venv</code>). No black, no pyright of its own

Variants of the `python` template (flake outputs): `default` = Python 3.15, `py-stable` = 3.13, `py-lts` = 3.12, `py311` = 3.11.

How `direnv` picks one: the `direnv.nix` helper defines `use_dev_env() { use flake <dev-environments>/language-specific/$1 }`. So the `.envrc` line decides the variant:

<code>.envrc</code> content	Result
<code>use_dev_env python</code>	<code>python</code> template, default output (latest Python)
<code>use_dev_env "python#py-lts"</code>	same template, output named after <code>#</code>
<code>use_dev_env "python#py-stable"</code>	same template, output <code>py-stable</code>

Your folders in `~/github-repos/personal/developing-projects/python-projects/`: `python-latest` (`use_dev_env python`), `python-lts` (`use_dev_env "python#py-lts"`), `python-stable` (`use_dev_env "python#py-stable"`). After a new or changed `.envrc` run `direnv allow` once. Then start nvim from that folder so nvim inherits the environment.

When a tool is missing the config does not complain:

Tool missing	What happens
<code>pyright</code> or <code>ruff</code>	That server is silently not enabled (no warning when opening the file). <code>:LspStart pyright</code> or <code>:LspStart ruff</code> names the missing program
<code>black</code>	<code><Space>f</code> shows ONE warning: Python: black not found on PATH (open nvim inside the python devShell). The after-save format check stays silent
<code>uv</code> (in a uv project)	Not a real case here (uv is installed system-wide). If it were missing, <code><Space>f</code> would show the same black warning and <code><Space>rf</code> / <code><F9></code> would fail inside the quickfix window
<code>python</code> / <code>python3</code>	<code><Space>rf</code> / <code><F9></code> shows the shell error in the quickfix window. <code><Space>rr</code> has no PATH check for Python: tested with an empty PATH, the terminal shows <code>bash: line 1: python3: command not found</code> and no nvim warning appears
<code>typos-lsp</code>	Silent

Running Code

Two ways, for different purposes.

Key	Command it runs	Where output goes
<code><Space>rf</code> / <code><F9></code>	<code>python -u "<file>"</code> , or <code>uv run python -u "<file>"</code> in a uv project (see below)	Quickfix window, 6 lines tall at the bottom (AsyncRun opens it by itself). The last line says <code>[Finished in N seconds with code C]</code>
<code><Space>rr</code>	<code>python3 <file></code> (shell-escaped)	A new terminal in a vertical split on the LEFT of your code, titled like <code>term://...:python3 'file.py'</code> ; Claude's panel stays on the right. When the program ends it shows <code>[Process exited 0]</code>

Facts that differ between the two (all tested):

- `<Space>rf` and `<F9>` are **buffer-local** and the same command: it exists only in Python buffers. `-u` means unbuffered, so `print` output appears while the program runs.

- `<Space>rr` is global and detects the file type. For Python it is always plain `python3 <file>`: **it never uses `uv run`, and has no `-u`**. In a `uv` project without an active environment use `<Space>rf`.
- Both run the file as saved on disk. Save first (`:w`).
- `<Space>rr` on an unnamed buffer shows one warning (`save the file first`).
- Paths with spaces or special characters are safe in `<Space>rr` (the name is shell-escaped). `<Space>rf` / `<F9>` wraps the name in double quotes, so a double quote or `$` inside a file name would break it.

Example to try both. Save as `hello.py`:

```
import sys, os

def add(a: int, b: int) -> int:
    return a + b

print(add(1, 2))
print(sys.version_info[:2], os.environ.get("VIRTUAL_ENV"), sys.executable)
```

Press `<Space>rf` (or `<F9>`). Expect in the quickfix window: `3`, then the Python version, the venv path (or `None`) and the interpreter path, then `[Finished in 0 seconds with code 0]`. A program that crashes ends with `code 1` and the traceback, as in the tested example `add(1, "two")`:

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
[Finished in 0 seconds with code 1]
```

When `<Space>rf` / `<F9>` uses `uv`

The choice is made when the Python file is opened (`after/ftplugin/python.lua`), from the project root (nearest `.git` or `pyproject.toml`):

```
local py_env = utils.get_py_env()
local py_cmd = (py_env == "uv") and "uv run python" or "python"
vim.keymap.set("n", "<F9>", string.format('<C-U>AsyncRun %s -u "%%"<CR>', py_cmd), ...)
```

Situation	<code><Space>rf</code> / <code><F9></code> runs	<code><Space>f</code> runs
No project root (no <code>.git</code> , no <code>pyproject.toml</code>)	<code>python -u</code>	<code>black</code>
An environment is active (<code>\$VIRTUAL_ENV</code> or <code>\$CONDA_DEFAULT_ENV</code> set)	<code>python -u</code>	<code>black</code>
<code>uv.lock</code> in the project root and no active environment	<code>uv run python -u</code> (tested: quickfix title : <code>AsyncRun uv run python -u "hello.py"</code>)	<code>uv run black</code>
Project root without <code>uv.lock</code> , no active environment	<code>python -u</code>	<code>black</code>

Tested: in a folder with `pyproject.toml` + `uv.lock` and a plain shell, the quickfix title showed `uv run python -u`; inside the python devShell (where `$VIRTUAL_ENV` is the project's `.venv`) it showed plain `python -u`.

Because the check runs at file open, activate the environment (or create `uv.lock`) before opening the file, or reload with `:e!`.

Create a uv test project like this:

```
mkdir uvp && cd uvp
git init
uv init --bare      # creates pyproject.toml
uv lock             # creates uv.lock
nvim hello.py
```

Stopping a running program

Started with	How to stop
<code><Space>rf / <F9></code>	<code>:AsyncStop</code> (tested with a 60-second <code>time.sleep: ps</code> showed <code>python -u slow.py</code> before and nothing after; stronger kill: <code>:AsyncStop!</code>)
<code><Space>rr</code>	In the terminal press <code><Ctrl-C></code> , or delete the terminal buffer with <code>:bd!</code> (tested: <code>ps</code> showed <code>python3 slow.py</code> before <code>:bd!</code> and nothing after)

Warning, tested earlier: `<Space>q` on a terminal window only **closes the window**; the running program keeps running hidden. Stop it first. See section 55 and section 8 for terminal navigation.

Formatting and Linting

Key / command	What it does
<code><Space>f</code> (Python buffers)	Runs <code>black</code> on the file on disk (<code>:silent !black %</code>); in a uv project <code>uv run black %</code> . Nvim then reloads the buffer and shows <code>File changed on disk. Buffer reloaded!</code> (tested)
<code><Space>fm</code>	LSP format, async, in the buffer (not saved). In Python the formatter is ruff's server, so it is <code>ruff format</code> , not <code>black</code> (tested: the buffer became modified, <code>•</code> in the tab, and <code>{"a":1,\n "b":2}</code> became <code>{"a": 1, "b": 2}</code>)

Try it. Save as `messy.py`:

```
import os, sys
def f( a,b ):
    return a+b
print(f(1,2))
```

Press `:w`. Expect the warning `messy.py: file is not formatted (black)`. Press `<Space>f`. Expect the file to become:

```
import os, sys

def f(a, b):
    return a + b

print(f(1, 2))
```

Save again: no warning. (`os` and `sys` stay; ruff still shows `E401` and `F401` hints. See "Code actions from ruff".)

Key points:

- Both formatters give almost the same result on normal code, but they are different programs. `<Space>f` (black) matches the after-save check; `<Space>fm` does not need black installed.
- Nothing formats automatically on save.
- **Format check after save:** every `:w` of a Python file runs `black --check --quiet <file>` in the background. If the file would change you get `<file>: file is not formatted (black)`. If black fails (for example a syntax error) you get `<file>: black could not check the file (syntax error?)` plus the first error line. Nothing is changed. If `black` is not on PATH the check is silent. It uses plain `black`, even in uv projects.
- **Without black:** tested outside the devShell, `<Space>f` shows ONE warning `Python: black not found on PATH (open nvim inside the python devShell)`, and saving shows nothing. In a uv project (uv is on PATH) the key stays silent and changes nothing when the project has no black: the command fails inside `:silent`. When the project has black (tested with `uv add --dev black` in a scratch project), `<Space>f` runs `uv run black` and reformats the file, with the same `Buffer reloaded!` message.
- **Line length:** black's default is 88, and the `colorcolumn` marker for Python is also 88 (default for other files: 100). A line touching the marker is too long for black. Black does not wrap long strings or comments.
- **Indentation:** `tabstop`, `softtabstop`, `shiftwidth` = 4, `expandtab` on. No wrapping (`wrap` off, `sidescroll` 5, `sidescrolloff` 2).

Code actions from ruff

Put the cursor on the first line of `messy.py` and press `<Space>ca`. Tested result, a list of four ruff actions:

```
1. Ruff (E401): Split imports [ruff]
2. Ruff (E401): Disable for this line [ruff]
3. Ruff: Fix all auto-fixable problems [ruff]
4. Ruff: Organize imports [ruff]
```

Pick a number with `<CR>`. "Organize imports" sorts and groups the imports (pyright's own version is disabled in `after/lsp/pyright.lua` so there is no conflict). "Fix all" applies every automatic fix, such as removing unused imports.

Linting from the command line (`:compiler ruff`)

The ftplugin sets `vim.g.ruff_makeprg_params = ""` so that Neovim's built-in ruff compiler plugin works without `--preview`. Tested:

```
:compiler ruff
:make %
```

runs `ruff check --output-format=concise nav.py` and puts every finding in the quickfix list (then `:copen`, `jq`, `[q`; see section 26). Needs `ruff` on PATH (devShell).

LSP Keys

Active in a Python buffer once pyright (and ruff) have attached. Maps are set per buffer and show ONE warning instead of falling through when no server supports the action.

Key	What it does	Server
<code>gd</code>	Go to definition (one result jumps, several open the location list)	pyright
<code>K</code>	Hover: type and docstring in a floating window (plain text format)	pyright
<code><Space>rn</code>	Rename symbol everywhere	pyright
<code><Space>ca</code>	Code action: quick fixes, organise imports, fix all	ruff (and pyright)
<code><Space>gd</code> / <code><Space>gr</code> / <code><Space>gi</code>	Glance popups: definitions / references / implementations	pyright
<code><Space>de</code> / <code><Space>dE</code>	Next / previous error	diagnostics from pyright and ruff
<code><Space>dd</code>	Show diagnostic detail under the cursor	any
<code>]d</code> / <code>[d</code>	Next / previous diagnostic (<code>]D</code> / <code>[D</code> last / first)	any
<code><Space>db</code> / <code><Space>dw</code>	Buffer / whole workspace diagnostics list	any
<code><Space>dt</code>	Toggle diagnostics on and off	any
<code><Space>qb</code> / <code><Space>qw</code>	Put buffer / all diagnostics in the quickfix list	any
<code><Ctrl-w>d</code>	Diagnostics under the cursor in a float	any
<code><Space>fm</code>	Format the buffer (see above)	ruff

Commands:

Command	Use
<code>:LspAttached</code>	Popup with the servers attached to this buffer
<code>:LspInfo</code>	<code>checkhealth vim.lsp</code> (configs, enabled servers, problems)
<code>:LspRestart</code>	Restart the servers of this buffer (needed after changing a venv or installing a package in some cases)
<code>:LspStop</code> / <code>:LspStart [name]</code>	Stop / start; <code>:LspStart</code> explains a missing program
<code>:LspLog</code>	Open the LSP log file
<code>:LspInlayHints enable</code> / <code>disable</code>	Switch inlay hints globally (off by default)

Who does what:

Task	Server
Type errors, wrong arguments, unknown imports, hover, definitions, rename	pyright (<code>standard mode</code> ; <code>deprecateTypingAliases hint off</code> ; <code>useLibraryCodeForTypes on</code>)
Style and lint diagnostics, unused imports, fixes	ruff
Import sorting	ruff (<code>organizeImports = true</code>); pyright's is disabled
Spelling mistakes in names and comments	typos_lsp

Tested: after `:LspInlayHints enable` no inline hints appeared in a small Python file (pyright sends none for this code), so expect little or nothing in Python.

Ruff reads its rules from `pyproject.toml` / `ruff.toml` in the project. Pyright reads `pyrightconfig.json` OR `[tool.pyright]` in `pyproject.toml`. See section 13 and section 44 for the LSP basics.

Debugging with pdb

`<Space>dp` (Python buffers only) runs `:GdbStartPDB python -m pdb %` through the nvim-gdb plugin. Linux and Windows only; on macOS you get one warning. In any other file type `<Space>dp` shows one warning (`only in python buffers`). It always uses plain `python -m pdb`, never `uv run`.

Why pdb and not a "real" DAP debugger: nvim-dap is installed only for Java. For Python you use pdb (in the standard library, nothing to install) and nvim-gdb adds the visual part: it marks the current line in your code and gives you function keys and leader keys.

Example to try. Save as `dbg.py`:

```
def square(n):
    r = n * n
    return r

total = 0
for i in range(3):
    total += square(i)
print("total", total)
```

Press `<Space>dp` in this file. Tested result: the source window shows the file with a ► mark in the sign column on line 1, and below it a terminal pane shows

```
> ../dbg.py(1)<module>()
-> def square(n):
(Pdb)
```

Press `<Space>dn` (or `<F10>`) twice. The ► moves to line 6, then line 7, and the terminal shows `n` typed for you each time (tested). `<Space>dc` (or `<F5>`) continues; this small program finished and printed `total 5`, then `pdb` says `The program finished and will be restarted` and starts again at line 1.

Keys during the session. The `<Space>d` keys work in Python buffers without function keys; the F-keys are nvim-gdb's own. Outside a debug session the `<Space>d` keys show one warning `pdb: no debug session here (start one with <Space>dp)`. All tested except `<F4>`; run them from the code window (go there with `<Ctrl-\><Ctrl-n>` then `<Ctrl-w>k` if you are in the `pdb` pane):

Key	Action	Command
<code><Space>dB / <F8></code>	Toggle breakpoint on the current line (a ● appears in the sign column)	<code>:GdbBreakpointToggle</code>
<code><Space>dc / <F5></code>	Continue (stops at the next breakpoint: ► lands on the ● line)	<code>:GdbContinue</code>
<code><Space>dn / <F10></code>	Next (step over)	<code>:GdbNext</code>
<code><Space>ds / <F11></code>	Step (into a call): from <code>total += square(i)</code> the ► jumped to <code>def square(n):</code>	<code>:GdbStep</code>
<code><Space>df / <F12></code>	Finish (run until the function returns): the ► stopped on <code>return r</code>	<code>:GdbFinish</code>
<code><Space>du / <F4></code>	Until (pdb: continue until the next line greater than the current one)	<code>:GdbUntil</code>
<code><Ctrl-p> / <Ctrl-n></code>	Frame up / down (<code><Ctrl-p></code> typed up in <code>pdb</code> and the ► went to the caller line)	<code>:GdbFrameUp / :GdbFrameDown</code>
<code><Space>dv</code>	Evaluate the word under the cursor (visual mode: the selection); with the cursor on <code>square</code> <code>pdb</code> printed <code><function square at 0x...></code>	<code>:GdbEvalWord / :GdbEvalRange</code>

Other commands: `:GdbBreakpointClearAll`, `:GdbFrame` (jump to the current line), `:GdbInterrupt`, `:GdbLopenBacktrace`, `:GdbLopenBreakpoints`, `:GdbCreateWatch`.

You can also type plain `pdb` commands in the terminal pane (`n`, `s`, `c`, `p var`, `l`, `bt`, `q`): go there with `<Ctrl-w>j`, press `i`, type, press `<Enter>`. Leave terminal mode with `<Ctrl-\><Ctrl-n>`.

Quit: `:GdbDebugStop`. The debug layout was in the same tab here (tested: one tab, two windows during the session, one after). Closing the debug windows also ends it.

History (fixed, tested): `nvim-gdb` used to bind `<F9>` itself (evaluate) and removed it from the code buffer after a session, which also removed your `<F9>` run key until

:e! . Its evaluate key is now `<Space>dv`, so `<F9>`, `<Space>rf`, `<Space>f` and `<Space>dp` work before, during and after a debug session.

Notes:

- nvim-gdb's own evaluate key was `<F9>`; it is moved to `<Space>dv` (only while a debug session is active), and `<F9>` is no longer the eval key. `<F4>` `<F5>` `<F8>` `<F10>` `<F11>` `<F12>` still work.
- nvim-gdb loads the first time you use a `:GdbStart*` command. Its default start keys `<Space>dd/dl/dp/db/dr` are turned off (`vim.g.nvimgdb_disable_start_keymaps = true`) so they do not replace your own `<Space>dd` and `<Space>db`; `<Space>dp` is yours, set in the ftplugin.
- A quick alternative without any plugin: add `breakpoint()` in the code and run it with `<Space>rr`; the terminal stops there with a `(Pdb)` prompt.
- Section 56 has the wider debugging picture.

Virtual Environments

Statusline label. In Python buffers only, the statusline shows the environment (section B of the lualine bar, section 32):

Environment variable set	Label
<code>\$VIRTUAL_ENV</code> (any <code>venv</code> , <code>.venv</code> , <code>devShell's venvShellHook</code> , <code>UV source .venv/bin/activate</code>)	<code><folder name> (venv)</code> , for example <code>.venv (venv)</code>
only <code>\$CONDA_DEFAULT_ENV</code>	<code><env name> (conda)</code>
neither	nothing shown

`$VIRTUAL_ENV` wins when both are set. In a `uv` project with `uv.lock` and no active environment the label stays empty (tested), even though `uv run` will use the project's `.venv`. Tested in the python devShell: the label read `.venv (venv)` (the folder name of `$VIRTUAL_ENV`, with its dot).

How to activate an environment.

Goal	Do this
Use the devShell tools and the project's <code>.venv</code>	Enter the project folder (direnv loads the devShell and the <code>venvShellHook .venv</code>), then start nvim there
Plain venv	<code>python -m venv .venv</code> , <code>source .venv/bin/activate</code> (fish: <code>source .venv/bin/activate.fish</code>), then start nvim in that shell
uv project	Either <code>source .venv/bin/activate</code> before starting nvim, or leave it inactive and use <code><Space>rf</code> / <code><F9></code> / <code><Space>f</code> , which run through <code>uv run</code>
Switch environment	Close nvim, change environment, start nvim again (the shell environment is inherited at start)

How pyright finds packages (tested). In a `uv` project (`.venv` with packages) opened from the python devShell (whose own `python` lacks them), `import pytokens`

gave the error `Import "pytokens" could not be resolved`. After adding a `pyrightconfig.json` with `{"venvPath": ".", "venv": ".venv"}` the error was gone. So the `.venv` folder alone is not enough: pyright uses the `python` on PATH, unless `venvPath` / `venv` say otherwise. Background: Pyright runs as a child of nvim and uses the `python` on PATH. If `VIRTUAL_ENV` was set when nvim started, it resolves imports from that environment. With `autoSearchPaths` it also searches `src/`. To point pyright explicitly, put `venvPath` and `venv` in `pyrightconfig.json` OR `[tool.pyright]` in `pyproject.toml`, then `:LspRestart`.

Testing

There is **no test runner support** for Python in this config: no neotest, no vim-test, no pytest or unittest keys. What you can do:

Goal	How
Run all tests	In the terminal split: <code>:terminal pytest</code> , or run <code>pytest</code> in a shell next to nvim (pytest comes from the project environment; it is not in the python devShell list)
Run a test file	<code><Space>rf</code> (or <code><F9></code>) or <code><Space>rr</code> on a file that ends with <code>unittest.main()</code> , OR <code>:!pytest %</code>
Debug a failing test	Put <code>breakpoint()</code> in the test and run <code><Space>rr</code> , or <code>pytest --pdb</code> in a terminal
See failures in quickfix	<code>:AsyncRun pytest -q</code> (output opens in the 6-line quickfix window, section 26)

The only plugin that knows about tests is nvim-java (Java only, `<Space>jt...`).

Navigation and Text Objects

All keys below exist in the keymap dump for Python buffers (or globally).

Key	Mode	What it does
<code>]] / [[</code>	n, x, o	Next / previous top-level <code>class</code> or <code>def</code> (Neovim's runtime Python ftplugin; not mapped by your config)
<code>]] / [[</code>	n, x, o	End of the next / previous top-level function or class
<code>] m / [m</code>	n, x, o	Next / previous <code>def</code> or <code>class</code> , any nesting (method start)
<code>] M / [M</code>	n, x, o	Next / previous end of a method
<code><Space>t</code>	n	Toggle the aerial outline (classes, functions, methods)
<code>] t / [t</code>	n	Next / previous symbol (aerial)
<code>gS</code>	n	treesj: toggle split / join of the construct under the cursor (a long call or list to one item per line, and back)
<code>ii / ai</code>	x, o	Indent scope inside / with its border lines (mini.indentscope): <code>dii</code> deletes the body of the block you are in, <code>vai</code> selects a whole <code>if</code> or <code>def</code>
<code>za, zc, zo, zR, zM, zr, zm</code>	n	Folds (nvim-ufo: LSP folding ranges, else indent); <code><Space>K</code> previews a closed fold
<code>gd</code> then <code><Ctrl-o></code>	n	Jump to a definition and back
<code><Space>gr</code>	n	References in a Glance popup

Operators combine with the motions: `d]]` deletes up to the next top-level definition, `v]m` selects up to the next method. `ii` depends on indentation, so it works well in Python. Section 21, 37, 47 and 50 have more.

Snippets

Snippets come from `my_snippets/python.snippets` (UltiSnips) and the shared vim-snippets collection. Type the trigger and press `<Ctrl-j>` to expand; `<Ctrl-j>` / `<Ctrl-k>` jump to the next / previous placeholder.

Trigger	Expands to	Only at line start
<code>print</code>	<code>print("\$1".format(\$2))</code>	no
<code>impa</code>	<code>import F00 as BAR</code> (two placeholders)	yes
<code>main</code>	<code>def main():</code> <code>with</code> <code>an</code> <code>empty</code> <code>body</code> <code>and</code> <code>if __name__ == "__main__": main()</code>	yes
<code>sol</code>	<code>solution = Solution()</code> (coding-challenge helper)	yes

The snippet menu may also offer vim-snippets entries (`def`, `class`, `ifmain`, ...). Section 15 and section 52 explain the engine.

Troubleshooting

Symptom	Likely cause	Fix
Nothing highlights errors, <code>:LspAttached</code> shows no <code>pyright</code>	<code>nvim</code> was not started in a shell where <code>pyright-langserver</code> is on PATH	Check <code>which pyright-langserver</code> ; start <code>nvim</code> in the python devShell or after <code>direnv allow</code> ; <code>:LspStart pyright</code> names the missing program
No <code>ruff</code> diagnostics	<code>ruff</code> is not global	Start <code>nvim</code> inside the python or jupyter devShell
Import "xyz" could not be resolved	Package is not in the Python that <code>pyright</code> sees (no active <code>venv</code> when <code>nvim</code> started, or packages not installed)	Activate the environment BEFORE starting <code>nvim</code> , install the package, then <code>:LspRestart</code> ; or set <code>venvPath</code> / <code>venv</code> in <code>pyrightconfig.json</code>
Statusline shows no environment	No <code>\$VIRTUAL_ENV</code> or <code>\$CONDA_DEFAULT_ENV</code> , or it is a <code>uv</code> project that is not activated	Activate the environment before starting <code>nvim</code> ; label only appears in Python buffers
<code><Space>f</code> says <code>black not found on PATH</code>	Not in the python devShell, or <code>uv</code> missing in a <code>uv</code> project	Start <code>nvim</code> in the python devShell; for <code>uv</code> projects make sure <code>uv</code> is on PATH (and <code>uv add --dev black</code>)
No "not formatted" warning after save, although the file is untidy	<code>black</code> is not on PATH (check is silent), or the file type is not detected as <code>python</code>	<code>:!which black; :set ft?</code>
<code><Space>rf</code> / <code><F9></code> prints <code>ModuleNotFoundError</code>	Wrong interpreter: plain <code>python</code> was used (environment not active when the file was opened)	Activate the environment, reopen the file with <code>:e</code> so the <code>uv/venv</code> choice is re-evaluated
<code><Space>rf</code> / <code><F9></code> does nothing visible	Quickfix window closed or scrolled, or <code>AsyncRun</code> not loaded	<code>:copen</code> ; check <code>:AsyncRun echo hi</code> works
<code><Space>rr</code> opens nothing	Unnamed buffer (one warning: save the file first) or an unsupported file type	<code>:w file.py</code> ; check <code>:set ft?</code>
<code><Space>rr</code> terminal flashes and closes, or shows <code>command not found</code>	<code>python3</code> is not on PATH	Start <code>nvim</code> in the devShell
<code><Space>rr</code> output is late	Python buffers stdout when not attached to a tty	Use <code><Space>rf</code> (it uses <code>-u</code>) or <code>print(..., flush=True)</code>
<code><Space>dp</code> shows one warning	Not a Python buffer, or macOS	Open a <code>.py</code> file; on macOS <code>pdb</code> is not available through this key
Pyright hover looks plain	Hover is set to plain text on purpose	None needed
Program keeps running after closing its terminal Spelling squiggles on names	<code><Space>q</code> only closes the window <code>typos_lsp</code>	Reopen the buffer with <code>:ls</code> , <code>:b <n></code> , <code><Ctrl-C></code> , or <code>:bd!</code> Add the word to a <code>typos.toml</code> or <code>_typos.toml</code> at the project root

Related Sections

Section 8 (terminal integration), section 13 (LSP), section 15 (snippets), section 18 (folding), section 19 (code running), section 21 (text objects), section 26 (quickfix), section 32 (statusline), section 36 (debugging), section 37 (aerial), section 41 (file-type settings), section 42 (automatic behaviours), section 43 (toolchain), section 44 (LSP in depth), section 52 (snippets for developers), section 55 (running in depth), section 56 (debugging in depth).

Chapter 14: LaTeX

80. LaTeX (vimtex, texlab, Itex, PDF viewer)

This section covers everything that is specific to LaTeX in this config: what each tool is for, how it works, the keys, and what to do when something fails. Global things (LSP keys, completion menu, windows, spell checking) are only mentioned with a pointer to their own section. Typst and Markdown have their own sections.

Everything marked "(tested)" was run in a real Neovim inside the LaTeX devShell with a scratch project (October 2026).

The big picture

Writing LaTeX means: a `.tex` source file, a compiler (`latexmk` runs `pdflatex` as many times as needed and runs BibTeX/biber), and a PDF viewer. Each tool in the config covers one part:

Tool	Why it is in the setup	What you use it for
vimtex (plugin)	Knows LaTeX syntax and the compile chain	Compile (<code><Space>rf</code>), view PDF, jump between sections/environments, text objects, change/delete/toggle environments and commands, table of contents, error list
texlab (language server)	Understands the whole project (labels, citations, includes)	Completion of <code>\ref{}</code> / <code>\cite{}</code> / commands, diagnostics, hover (<code>κ</code>), symbols, rename, <code>:LspTexlabBuild</code>
Itex_plus (language server)	Grammar and spelling for prose	Underlines mistakes in the text (not in commands) as diagnostics
typos_lsp (language server)	Catches common typos in any file	Same diagnostics channel; it attaches to every file type
UltiSnips snippets	Typing shortcuts	<code>use</code> , <code>eqa</code> (file <code>my_snippets/tex.snippets</code>)
zathura (PDF viewer)	Light viewer that reloads when the PDF changes and supports forward/inverse search	Shows the result
LaTeX devShell	Provides all the programs	<code>latex</code> , <code>latexmk</code> , <code>texlab</code> , <code>zathura</code> , <code>pandoc</code>

In a `.tex` buffer the config also sets `textwidth = 120`, `wrap` on and a column marker at 120 (`after/ftplugin/tex.lua`, `lua/options.lua`). The statusline shows `texlab (+2)` when `texlab`, `Itex_plus` and `typos_lsp` are attached (tested).

What it needs (the devShell)

`vimtex` is **enabled only when** `latex` **is on PATH**:

```
-- lua/plugin_specs.lua
"lervag/vimtex",
enabled = function() return utils.executable("latex") end,
ft = { "tex" },
```

Outside a LaTeX environment the plugin does not load: no `<Space>rf`, no `<F9>`, no `\ll`, no `:Vimtex*` commands. The environment comes from the flake `~/nix/templates/krit/dev-environments/language-specific/latex/flake.nix`. Its packages: `texlive.combined.scheme-full` (gives `latex`, `latexmk`, all packages), `texlab`, `zathura`, `pandoc`, `tectonic`, `latex2html`, `latex2mathml`.

How you enter it	Details
direnv (normal)	The project folder has an <code>.envrc</code> with one line: <code>use_dev_env latex</code> (see <code>~/github-repos/personal/developing-projects/latex-projects/.envrc</code>). Run <code>direnv allow</code> once. Then <code>cd</code> in and start <code>nvim</code> there.
By hand	<code>nix develop <path-to-flake></code> and then <code>nvim</code>

Check inside the folder: `which latex latexmk texlab zathura` must print four paths.

Start Neovim from inside the devShell. The tools are looked up when Neovim starts; entering the shell afterwards does not enable vimtex in the already running Neovim.

Outside the devShell a `.tex` file still gets syntax colours, `ltex_plus`, `typos_lsp`, snippets and buffer/path completion. It gets no vimtex, no texlab, no PDF.

Quick start

1. `cd` into the project folder (direnv loads the devShell).
2. Create and open `main.tex`:

```
\documentclass{article}
\begin{document}
\section{Test}
Hello \LaTeX.
\end{document}
```

3. Wait until the statusline shows `texlab (+2)`. `:LspAttached` lists the clients.
4. `<Space>rf` (or `<F9>`) starts continuous compiling. After the first successful compile **zathura opens by itself** with the PDF (see "The viewer").
5. Edit, then `:w`. The compiler notices the saved file, recompiles, zathura reloads. (Auto-save never saves `.tex`; save yourself.)
6. `\lv` shows or re-opens the PDF at the cursor position.
7. `<Space>rf` / `<F9>` again stops compiling; `\lc` removes the auxiliary files.

Compiling

`<Space>rf` and `<F9>` (buffer-local, set by an autocommand for `tex` files; description "LaTeX: start/stop compiling (vimtex)") and `\ll` are the same command, `<Plug>(vimtex-compile)` (tested). It starts `latexmk` in the background in **continuous mode** (recompiles whenever a source file changes); pressing it again stops it. `\` is the local leader: the config sets only `mapleader = <Space>`, so vimtex keeps the backslash.

Where the output goes. Tested: `main.pdf`, `main.aux`, `main.log`, `main.bbl` and the other files are written **next to** `main.tex`, not into a `build/` folder. The config contains

```
let g:vimtex_compiler_latexmk = { 'build_dir' : 'build' }
```

but the installed vimtex has no `build_dir` option (its documentation does not mention it; the current name is `out_dir`), so this line has no effect. `:LspTexlabBuild` (texlab's own one-shot build) also writes `main.pdf` next to the source (tested). If you want a `build/` folder, the option to set is `out_dir`.

Command / key	What it does
<code><Space>rf</code> , <code><F9></code> or <code>\ll</code> (<code>:VimtexCompile</code>)	Start or stop continuous compiling (tested)
<code>\lk</code> (<code>:VimtexStop</code>)	Stop. Message "VimTeX: Compiler stopped (name.tex)" (tested)
<code>\lc</code> (<code>:VimtexClean</code>)	Remove auxiliary files; keeps <code>main.pdf</code> , <code>main.bbl</code> , <code>main.synctex.gz</code> . Message "VimTeX: Compiler clean finished" (tested)
<code>\lC</code>	Clean everything including the PDF
<code>\lo</code>	Show the compiler output
<code>\lg</code> / <code>\lG</code>	Status of this / all compilers
<code>\ll</code>	Compile only the selected lines (visual mode too)
<code>\lq</code>	Show the vimtex log
<code>\li</code> / <code>\lI</code>	Info about the vimtex state of this document (short / full)
<code>\lx</code> / <code>\lX</code>	Reload vimtex / reload its state (use after changing the document structure)
<code>\ls</code>	Toggle the main file (<code>:VimtexToggleMain</code>)
<code>\la</code>	Context menu for the item under the cursor
<code>:LspTexlabBuild</code>	texlab's own build, once (tested: produces <code>main.pdf</code>). For daily work use <code><Space>rf</code> : it keeps recompiling.

All `\l...` keys above were checked in the live buffer; each points at the `<Plug>(vimtex-...)` map named in the vimtex documentation (tested).

Errors

When a compile fails vimtex opens the **quickfix list** by itself, titled "VimTeX errors (LaTeX logfile)", and shows "VimTeX: Compilation failed!" (tested with a file containing `\foobar`; the entry was `Undefined control sequence. \foobar`, with the line number). texlab also puts the problem in the text as a diagnostic (`]d` / `[d`, `<Space>dd`; see the LSP section).

Key / command	What it does
<code>\le</code> (<code>:VimtexErrors</code>)	Open the error list again
<code>:cnext</code> / <code>:cprev</code>	Next / previous error; <code>:cclose</code> closes the list
<code>\lo</code>	Read the raw compiler output

If an error stops the compile, fix it, `:w`: the continuous compiler retries by itself (it does not have to be restarted).

Common messages: "Undefined control sequence" (a command is misspelled or its `\usepackage` is missing), "File x.sty' not found" (a package is not in texlive; in the full scheme this should not happen), "Missing \$ inserted" (maths outside ...), "Runaway argument" (a `{` without `}`).

Several files (a project with chapters)

Compile always starts from the **main file** (the one with `\documentclass`). When you open a chapter that is loaded with `\input` or `\include`, vimtex has to know the main file. The reliable way is a magic comment on the first lines of the chapter:

```
%! TEX root = ../main.tex
Chapter text.
```

Tested: opening `chap/cl.tex` with this comment gives `b:vimtex.tex` = the full path of `main.tex`, and `<Space>rf` in the chapter built `main.pdf`. Without the comment vimtex tries to find the main file itself, but in my test it picked a different `main.tex` from a neighbouring folder, so always add the comment in chapters. `\ls` toggles between the file and the main file.

The viewer

`vim.g.vimtex_view_method` is `zathura` when `zathura` is on PATH, otherwise `general` (the system default PDF program). Windows uses SumatraPDF and macOS Skim (config lines for those platforms exist in the same spec).

- **A zathura window opens by itself after the first successful compile.** This is vimtex's default (`g:vimtex_view_automatic = 1`), not a bug. Later compiles only refresh the open window. If you do not want it: `:let g:vimtex_view_automatic = 0` (until you quit); you then open the PDF yourself with `\lv`. For a permanent change the line has to go into the config. (Tested both ways: without the option a zathura window titled with the full path of `main.pdf` opened about 14 seconds after `<Space>rf` in a fresh Neovim; with `view_automatic=0` nothing opened until `\lv`.)
- `\lv` (`:VimtexView`) opens the viewer or, if open, jumps to the place of the cursor (forward search). Tested from a chapter file with `%! TEX root:` zathura opened the PDF of the **main** file (`main.pdf`), and the process got `--synctex-forward 1:1:<path>/chap/cl.tex`, i.e. the cursor position of the chapter.
- **Inverse search** (Ctrl+click in the PDF jumps back to the source): the config writes the address of the running Neovim into `/tmp/vimtexserver.txt` every time a `tex` file is opened (`v:servername`). It is only a helper file; never edit it. With two Neovims open, the last one wins (the file is shared in `/tmp`). Tested prerequisites: the file exists and holds the same address as the running Neovim (`:echo v:servername`);

the zathura process was started by vimtex with the inverse-search callback `-x "nvim --headless -c \"VimtexInverseSearch %\{line\}:\{column\} '\{input\}'\""`. The jump itself is tested by running that same callback in a second Neovim: the first Neovim jumped to the clicked line and column. This only works because vimtex is loaded at startup (not only for `tex` files): the callback Neovim has no `tex` file open and needs the `:VimtexInverseSearch` command (found and fixed 2026-10-03; before, the click did nothing and showed no error).

Table of contents

`\lt` toggles, `\lt` opens the vimtex table of contents (`:VimtexTocToggle`, `:VimtexTocOpen`) (tested). It is a 30-column window on the left called TOC; the help lines are shown on top (config: `split_width = 30`, `show_help = 1`, layers content, todo, include). It lists sections, labelled equations, `\input` files and `TODO/FIXME` comments of the whole project.

Key in the TOC	Action
<code><Space></code>	Jump to the entry, keep the TOC open
<code><Enter></code>	Jump and close the TOC
<code><Esc></code> / <code>q</code>	Close
<code>r</code>	Refresh
<code>h</code>	Toggle the help text
<code>t</code>	Toggle sorted TODO list
<code>s</code>	Hide / show numbers
<code>-</code> / <code>+</code>	Show fewer / more section levels
<code>f</code> / <code>F</code>	Apply / clear a filter
<code>L</code> , <code>I</code> , ...	Switch layers (label, include, ...)

Move between the TOC and the text with `<Ctrl-w>h` / `<Ctrl-w>l` or `<Left>` / `<Right>`; the same keys move to the quickfix window (`<Ctrl-w>j` / `<Ctrl-w>k`, `<Down>` / `<Up>`). `<Ctrl-w>c` closes the window you are in.

Moving and editing (vimtex)

Example text for the tests below (all keys tested in this exact file):

```
\section{One}
Text with  $a+b$  and \textbf{bold} here. See \cite{knuth}.
\begin{equation}
x = 1
\end{equation}
\section{Two}
```

Motions

Key	Action
<code>]]</code> / <code>[[</code>	Next / previous section start (<code>\section</code> , <code>\begin{document}</code> , ...). From line 1 <code>]]</code> went to <code>\begin{document}</code> ; from the text of One it went to <code>\section{Two}</code>
<code>][</code> / <code>[]</code>	Next / previous section end
<code>]m</code> / <code>[m</code>	Next / previous <code>\begin</code> of an environment
<code>]M</code> / <code>[M</code>	Next / previous <code>\end</code> of an environment
<code>]n</code> / <code>[n</code> , <code>]N</code> / <code>[N</code>	Next / previous start / end of a maths zone
<code>]r</code> / <code>[r</code> , <code>]R</code> / <code>[R</code>	Frames (beamer)
<code>]/</code> / <code>[/</code> , <code>]*</code> / <code>[*</code>	Comment blocks
<code>K</code>	In this config LSP hover (texlab), not vimtex's package-documentation lookup
<code>%</code>	Jumps between <code>\begin</code> and <code>\end</code> and brackets, but here the map belongs to matchup (<code><Plug>(matchup-%)</code>), not vimtex (tested: from <code>\begin{equation}</code> it went to the <code>\end</code>)

Text objects (use after `d` , `y` , `c` , `v`)

Keys	Object	Result in the example
<code>ie</code> / <code>ae</code>	Environment: inside / with <code>\begin...\end</code>	cursor on <code>x = 1</code> : <code>yie</code> = <code>x = 1</code> , <code>yae</code> = whole equation (tested)
<code>ic</code> / <code>ac</code>	Command: name only / whole command	cursor on <code>textbf</code> : <code>yic</code> = <code>textbf</code> , <code>yac</code> = <code>\textbf{bold}</code> (tested)
<code>id</code> / <code>ad</code>	Delimiter pair: inside / with delimiters	cursor in <code>{bold}</code> : <code>yid</code> = <code>bold</code> , <code>yad</code> = <code>{bold}</code> (tested)
<code>i\$</code> / <code>a\$</code>	Maths	unreliable : on display maths it gives <code>x = 1</code> (<code>yi\$</code> inside the equation), but on inline <code>\$a+b\$</code> it selected the display equation instead (tested; tree-sitter highlighting replaces vim's syntax for maths), see the note below
<code>iP</code> / <code>aP</code>	Section	<code>yaP</code> on the heading line = from <code>\section{One}</code> up to before <code>\section{Two}</code> (tested)
<code>im</code> / <code>am</code>	List item	needs an <code>itemize/enumerate</code> item

`ic` / `ac` also exist in Markdown buffers with a different meaning (a config map for Markdown only); in `.tex` they are vimtex's.

Note on maths detection. Tree-sitter colours `.tex` files here, so Vim's own syntax is off, and vimtex shows a hint at start ("For more info, see :help vimtex-faq-treesitter"). Features that need the syntax groups to know "this is maths" are therefore unreliable for inline maths: `i$` / `a$` , `]n` , and the math insert maps. Environments, commands, delimiters and sections work (tested).

Change, delete, toggle

Keys	Action	Tested result
dse	Delete the surrounding environment	the <code>\begin..\\end</code> lines vanish, the body stays
cse	Change the environment name (prompt "Change surrounding environment: equation", type the new name, Enter)	<code>equation</code> became <code>align</code> in both places
tss	Toggle star of the environment	<code>equation</code> became <code>equation*</code>
tse	Toggle between two environments; default pair only <code>itemize</code> / <code>enumerate</code> (<code>g:vimtex_env_toggle_map</code>)	no change on <code>equation</code>
dsc / csc	Delete / change the surrounding command	<code>\textbf{bold}</code> became <code>bold</code>
tsc	Toggle star of a command	<code>\section{One}</code> became <code>\section*{One}</code>
ds\$ / cs\$ / ts\$	Delete / change / toggle maths delimiters (<code>\$. \$</code> , <code>\[. .\]</code> , <code>equation</code>)	not tested
tsd / tsD	Toggle <code>\left..\\right</code> modifiers	not tested
tsf	Toggle fraction <code>a/b</code> and <code>\frac{a}{b}</code>	not tested
<F6>	Surround the line (or visual selection) with an environment (vimtex default; no leader alternative is set)	map exists
<F7>	Create a command from the word (insert and normal mode; vimtex default, no leader alternative)	map exists
<F8>	Add <code>\left/\right</code> to delimiters (vimtex default, no leader alternative)	map exists
] in insert mode	Close the open environment/delimiter	map exists
` + letter in insert mode	Maths shortcuts (<code>`a</code> = <code>\alpha</code>), made by <code>vimtex#imaps#wrap_math</code> . <code>\lm</code> (<code>:VimtexImapsList</code>) opens a "VimTeX imaps" window that lists all of them; close it with <code>:close</code>	The maps exist but did not expand in the test, even inside <code>equation</code> : they only fire when <code>vimtex#syntax#in_mathzone()</code> is true, and it returned 0 there (tree-sitter note below). <code>\lm</code> itself was tested

These do not clash with the vim-sandwich keys of the config (`sa`, `sd`, `sr`).

Language server (texlab)

texlab attaches to `tex` files when `texlab` is on PATH. It reads the whole project, so it also knows labels and citations of other files.

- **Completion:** commands, environments, `\ref{}` labels, `\cite{}` keys. The menu opens while you type; `<Ctrl-n>` opens it manually, `<Tab>` / `<Ctrl-n>` move down, `<CR>` confirms only an item you picked, `<Ctrl-e>` or `<Esc>` closes it (see section 14).
- **Diagnostics** (including errors from the build log), hover (`⌘`), symbols, rename, code actions (`<Space>ca`): the global LSP keys, see the LSP section.
- `:LspTexlabBuild`: one build (tested).

Sources for `tex` files (`lua/config/nvim-cmp.lua`):

```
cmp.setup.filetype("tex", { sources = {
  { name = "omni" },      -- vimtex
  { name = "nvim_lsp" },  -- texlab
  { name = "ultisnips" },
  { name = "buffer", keyword_length = 2 },
  { name = "path" },
}})
```

Duplicates are normal: typing `\sec` and `<Ctrl-n>` shows entries from vimtex (omni) and texlab (nvim_lsp), 26 entries with several "section" (tested). Either inserts the same text.

Citations (bibliography)

Put references in a `.bib` file and load it in the document:

```
See \cite{knuth}.
\bibliographystyle{plain}
\bibliography{refs}      % refs.bib next to main.tex
```

with `refs.bib`:

```
@book{knuth, author={Knuth}, title={TAOCP}, year={1968}, publisher={AW}}
```

Typing `\cite{kn` and `<Ctrl-n>` shows `knuth [book] Knuth (1968), "TAOCP"` in the menu (tested). `latexmk` runs BibTeX itself during compile; the first compile after adding a `\cite` may show `[?]` until the next automatic run (tested: `main.bbl` was created by the first run). With biblatex use `\usepackage{biblatex}` and `\addbibresource{refs.bib}`; `latexmk` then runs biber (not tested here).

Grammar and spelling (ltex_plus)

LanguageTool as a language server. Config (`after/lsp/ltex_plus.lua`):

```
filetypes = { "markdown", "tex", "plaintex", "typst", "gitcommit", "text" },
settings = { ltex = { language = "en-US", ... } },
```

- **One language per document:** `en-US`, although `spelllang` is `en,it,de,fr`. LaTeX commands are skipped; only the prose is checked.

- Problems are diagnostics. In my test a misspelled word got **two** underlines, one from `typos` and one from `LTex`; this is normal.
- **Fix:** `<Space>ca` on the word opens the list (tested):

Entry	Result
Use 'sentence'	Replaces the word. Works (tested)
Use 'sen tense'	Other suggestion
Add 'sentence' to dictionary	Had no effect in my test: the <code>LTex</code> diagnostic stayed (the config has no handler for <code>ltex</code> 's client commands). Do not rely on it
Hide false positive, Disable rule	No effect, same as Add to dictionary (tested, see section 81)
sentence, Ignore ... in the project (<code>typos_lsp</code>)	<code>typos</code> fix; "Ignore" is a <code>typos</code> command

- **What works for false positives:** `ltex` magic comments in the file (tested).
`% LTex: enabled=false` on a line of its own switches `ltex` off for the file (the `LTex` underlines disappeared, `typos` stayed). `% LTex: language=de-DE` makes the file checked as German (tested: German messages). Remove the line to undo.
- **Built-in spell checker** (different thing): `<Space>cz` toggles, `]s` / `[s`, `z=`, `zg` (adds to `spell/en.utf-8.add`, a file in the public repo). More in section 31.

Snippets

UltiSnips, file `my_snippets/tex.snippets`. Both are start-of-line snippets (`b`): type the trigger at the beginning of a line, `<Ctrl-j>` expands and jumps forward, `<Ctrl-k>` jumps back. They also appear in the completion menu (tested: `use` + `<Ctrl-j>` gave `\usepackage{}` with the cursor inside; `eqa` + `<Ctrl-j>` gave the equation environment with the cursor in `\label{}`).

Trigger	Result
<code>use</code>	<code>\usepackage{package}</code> , name selected
<code>eqa</code>	<code>\begin{equation}\label{}</code> / body / <code>\end{equation}</code> ; first stop in the label, second in the body

Troubleshooting

Problem	Cause and fix
<code><Space>rf</code> , <code><F9></code> and <code>\ll</code> do nothing, <code>:VimtexCompile</code> unknown	vimtex not loaded: <code>latex</code> not on PATH. Start Neovim inside the devShell (which <code>latex</code> , <code>direnv</code> allow). Check <code>:set ft? is tex</code>
Compile fails, quickfix opened	Read the first entry (<code>\le</code>), fix that line, <code>:w</code> ; the compiler retries. Raw output: <code>\lo</code>
No PDF	Look next to <code>main.tex</code> (not in <code>build/</code>), see "Where the output goes". A fatal error stops the PDF
Viewer did not open	<code>zathura</code> not on PATH, or the compile failed, or <code>g:vimtex_view_automatic</code> is 0. Try <code>\lv</code>
A viewer opened that I did not expect	Normal after the first successful compile; <code>:let g:vimtex_view_automatic = 0</code>
"Undefined control sequence"	Misspelled command or missing package: add <code>\usepackage{...}</code> (snippet <code>use</code>)
Chapter file compiles the wrong document or errors	Add <code>%! TEX root = ../main.tex</code> at the top of the chapter
Citation shows <code>[?]</code>	<code>.bib</code> file not found or first run: compile again; check <code>\bibliography{refs}</code>
texlab not in the statusline	<code>texlab</code> not on PATH, or still starting. <code>:LspAttached</code> , <code>:checkhealth vim.lsp</code>
Same completion twice	Normal (vimtex omni and texlab)
<code>i\$ /]n / `</code> shortcuts do not find inline maths	Tree-sitter highlighting replaces vim syntax, see the note above
Real word marked wrong by ltx	Magic comment, or fix the spelling; "Add to dictionary" does not work here
German/other language text full of errors	<code>% LTeX: language=de-DE</code> on its own line
Edits not in the PDF	Auto-save does not save <code>.tex</code> : <code>:w</code> . Check the compiler is running (<code>\lg</code>)
First compile is slow	<code>latexmk</code> builds everything once (bibliography, references); later runs are fast. Stop with <code><Space>rf</code> when not needed
Inverse search from zathura jumps nowhere	Quit and restart Neovim (an old session may still have vimtex lazy-loaded); <code>/tmp/vimtexserver.txt</code> has the address of the last Neovim that opened a <code>tex</code> file, so re-open the file in the Neovim you want

Related sections

Typst (`<Space>tw`), Markdown, 28 (LaTeX and Typst, older summary), 31 (spell checking), the LSP and completion sections (diagnostic and completion keys), snippets (UltiSnips), windows (`<Ctrl-w>` moves).

Chapter 15: Markdown

81. Markdown (writing, preview, footnotes, PDF)

This section covers everything your config does that is specific to `.md` files: what each tool is for, how it works, the exact keys, and what to do when it fails. Global things (diagnostics keys, code actions, the spell keys, `gc` comments) are only mentioned briefly with a pointer. Section 27 is the short key list; this one is the full story.

Most results below were checked in a real Neovim session today (marked "tested"). The few things that could not be tested say so where they appear.

The tools and why they exist

Tool	Why it is in your setup	What it gives you
marksman (LSP)	Markdown has links and headings that point to each other	Link and heading completion, jump to a heading, symbols. Only for plain <code>markdown</code> files
ltex_plus (LSP, Language-Tool)	Prose needs grammar checking, not only spelling	Grammar and spelling problems as diagnostics. Language fixed to <code>en-US</code>
typos_lsp	Catches common typos in every file type	Diagnostics like <code>recieve</code> to <code>receive</code>
render-markdown.nvim	Raw Markdown is hard to read	Headings, lists, code blocks, tables, checkboxes drawn nicely inside the buffer
markdown-preview.nvim	See the final look in a browser	Live preview, <code><Alt-m></code> toggles it
vim-markdownfootnotes	Footnotes need numbering and a jump back	<code>^^ / <Space>mf</code> create a footnote, <code>@@ / <Space>mr</code> return
<code>:AddRef</code> (your own command)	Reference-style links keep the text readable	Adds <code>[label]: url</code> at the end of the file
prettier (via <code><Space>fm</code>)	marksman cannot format	Tidies lists, tables, spacing
tabular	Align table columns	<code>:Tabularize / </code>
<code>ic / ac</code> (your own text objects)	Work on fenced code blocks quickly	<code>dic</code> , <code>yac</code> , <code>vic</code> ...
<code>+</code> (your own operator)	Make a list from plain lines	<code>+ip</code>
<code><Space>mb</code> (your own operator)	Hard line breaks in Markdown	Adds a trailing <code>\</code>
<code>:ToPDF</code> (your own command)	Share a document as PDF	<code>pandoc</code> plus <code>xelatex</code> , PDF next to the file
smart_comment	Comments in Markdown are HTML comments	<code>gc</code> writes <code><!-- ... --></code> (the filetype's comment string); <code>gcs / gcr</code> (smart comment) use the language of the fence inside a fenced block

Settings for the Markdown filetype (`after/ftplugin/markdown.vim`):

```
setlocal concealcursor=c
setlocal synmaxcol=3000 " For long Chinese paragraphs
setlocal wrap
```

Tested in a Markdown buffer: `wrap` is on (the global default is off), `textwidth` is 0, `colorcolumn` is 100, `synmaxcol` is 3000, `conceallevel` is 3. `concealcursor` shows as empty: `render-markdown.nvim` sets its own conceal options on the window after the `ftplugin`, so the `concealcursor=c` line in the file has no visible effect. Facts that follow from this:

- Long lines wrap at word boundaries (`linebreak` is on globally). Neovim never hard-wraps while you type (no `textwidth`), and prettier keeps your line breaks.
- Syntax colours stop after column 3000 of a very long line.

Quick start

1. Open a file, for example `nvim "my notes.md"`. In normal mode headings, lists, code blocks and tables are drawn nicely. While you type in insert mode the drawing pauses (`render_modes`), and returns after about half a second in normal mode.
2. Press `<Alt-m>`. The browser opens with a live preview. Press `<Alt-m>` again to stop it.
3. Press `<Space>fm`. Prettier tidies the file as one undo step (`u` undoes it). The file is not saved to disk.
4. Footnote: put the cursor on the last letter of a sentence, press `<Space>mf`. A `[^1]` appears and the cursor jumps to the new `[^1]:` line at the end of the file (in normal mode: press `A` to type the note). Press `@@` to jump back.
5. PDF: save (`:w`) and run `:ToPDF` from a Neovim that was started inside the latex dev shell. Wait 30 seconds or more the first time. `my notes.pdf` appears next to the file.

Requirements

Tool	Where it comes from	Needed for
marksman, prettier, ltex-ls-plus, typos-lsp	<code>~/nix/users/krit/common/programs/cli-programs/neovim.nix</code> Global nix profile (	LSP features, <code><Space>fm</code> , grammar, typos
node and npm	Needed once by lazy.nvim to build the preview server (<code>cd app && npm install</code>)	<code><Alt-m></code>
A web browser	Your system default	<code><Alt-m></code>
pandoc, xelatex	Only in the latex dev shell (<code>~/nix/ates/krit/dev-environments/language-specific/latex/flake.nix</code>) and <code>pandoc</code> and <code>texlive.combined.scheme-full</code>)	<code>:ToPDF</code>

Tested: inside the latex dev shell `prettier`, `pandoc`, `xelatex` and `marksman` are all found by Neovim, and the attached LSP clients on a `.md` file are `ltex_plus`, `marksman`, `typos_lsp`. Note that `pandoc` is also found on this system outside the dev shell (it is on the system PATH), but `xelatex` and `prettier` are not found by a plain shell: Neovim must be started from a shell where they are on PATH (the dev shell, or the nix profile that provides them).

If a tool is missing, nothing falls back to a plain Vim key:

- No prettier: `<Space>fm` shows one warning `Markdown: prettier not found on PATH`.
- No pandoc: `:ToPDF` shows the error `pandoc not found`.
- No xelatex: pandoc runs and fails, one warning `ToPDF: pandoc failed (exit N)`.
- No marksman or ltex-ls-plus: that LSP simply does not start (check with `:checkhealth vim.lsp`).

Try the keys: an example file

Create a scratch file and paste this text. All the examples below refer to it.

```
# Title

Some text here.

```python
x = 1
y = 2
```

* item a
* item b

plain one
plain two

a	b
longer cell	x

A sentence.
```

Keys

All keys work only in Markdown buffers unless stated. `<Space>` is the leader key.

| Keymap | Mode | What it does |
|--|------|--|
| <code><Alt-m></code> | n | Toggle the browser preview. In another file type: one warning <code>Markdown preview: only in markdown buffers</code> |
| <code><Space>mf</code> | n | Add a footnote after the character under the cursor (see Footnotes). Elsewhere: one warning |
| <code><Space>mr</code> | n | Return from the footnote to the text. Elsewhere: one warning |
| <code>^^</code> | n, i | Add a footnote before the character under the cursor (see the off-by-one note below) |
| <code>@@</code> | n, i | Return from the footnote (replaces the macro replay <code>@@</code> in Markdown) |
| <code><Space>fm</code> | n | Format the file with prettier |
| <code><Space>mb + motion</code> | n | Add a trailing <code>\</code> (hard line break) to the lines of the motion, e.g. <code><Space>mbip</code> |
| <code><Space>mb</code> | x | Same on a Visual selection |
| <code>+ + motion</code> | n | Put <code>+</code> in front of the lines of the motion: <code>+ip</code> |
| <code>+</code> | x | Same on a Visual selection |
| <code>ic, ac</code> | o, x | Fenced code block, without or with the fence lines: <code>vic</code> , <code>dic</code> , <code>yac</code> , <code>cic</code> |
| <code>]] / [[</code> | n, x | Next / previous heading (levels 1 to 5) |
| <code>gO</code> | n | Outline: opens a location list window with one line per heading, indented by level (tested: <code>Heading One</code> , <code>Heading Two</code> , <code>Heading Three</code>). <code><CR></code> jumps to the heading, <code>:lclose</code> closes it |
| <code><Space>t</code> | n | Aerial symbol outline panel; <code>]t</code> / <code>[t</code> next / previous symbol |
| <code><Space>cz</code> | n | Toggle spell checking (global key, see Writing quality) |
| <code><Space><Space></code> | n | Trailing-space remover, but in Markdown it only warns <code>markdown: trailing spaces are hard line breaks, not stripped</code> (tested) |
| <code>:AddRef <label> <url></code> | cmd | Add a reference link at the end of the file |
| <code>:Tabularize / </code> | cmd | Align table columns |
| <code>:ToPDF</code> | cmd | Export a PDF |
| <code>:RenderMarkdown toggle</code> | cmd | Turn the rendering off / on |

Global keys that also work here: `<Space>ca` (code action, for example a `ltex_plus` fix), `<Space>dd`, `]d`, `[d` (diagnostics), `<Space>rn` renames through marksman (headings and links; tested: marksman advertises rename).

Accepted effects of the footnote maps:

- A single `^` or `@` typed in insert mode appears after 500 ms, because Neovim waits for a possible second key. `^` followed by any other key comes out at once.
- In normal mode `@@` is not "repeat last macro" in Markdown (tested: it is `<Plug>ReturnFromFootnote`). Use `@a` with the register name instead.

Text objects and operators

These live in `after/ftplugin/markdown.vim`. Tested on the example file:

| You do (cursor on a line inside the code block) | Result |
|---|---|
| <code>dic</code> | The two code lines are deleted, the lines with the fences stay (an empty block) |
| <code>dac</code> | The whole block with both fences is deleted |
| <code>yac then p</code> | The block, fences included, is copied and pasted |
| <code>dic then u</code> | Everything is back (one undo step) |

| You do | Result |
|---|---|
| <code>+ip on plain one</code> | Both lines become <code>+ plain one</code> and <code>+ plain two</code> |
| <code>V, j, +</code> | The same on the Visual selection |
| <code><Space>mbip on plain one</code> | The lines become <code>plain one\</code> and <code>plain two\</code> |

Notes: `+` keeps existing indentation (`+ text`). `<Space>mb` skips blank lines and lines that already end in `\`. `+` is also a normal Vim key (next line start) that is replaced here in Markdown buffers.

The `+` map is built so a count works (`3+j`): the file explains that a plain `:set` style map would turn the count into a range and give error E481.

Footnotes

Why: you write the sentence, press a key, write the note, jump back, without scrolling to the end and counting numbers.

How it works: vim-markdownfootnotes inserts `[^N]` in the text and a line `[^N]:` at the end of the file, numbering them in order. Your config adds the keys `^^` and `@@` and removes the plugin's own `<Space>f` and `<Space>r` maps in Markdown, because with Space as leader they swallowed `<Space>f...` typed quickly.

Tested flow (normal mode):

1. Cursor on the last letter of `Some text here.` (`$` puts it on the period).
2. `<Space>mf`. The line becomes `Some text here.[^1]` (the mark is inserted after the character under the cursor) and the cursor sits on the new last line `[^1]:` in normal mode.
3. Press `A`, type `My note`, press `<Esc>`.
4. Press `@@` (or `<Space>mr`). The cursor is back at the `[^1]` in the sentence.

Tested flow (insert mode, the `^^` variant):

```
Mid word done.      (cursor before "word", in insert mode, type ^^)
```

The result is `Mid [^1]word done.` and you are still in insert mode on the note line: type the note, then `<Esc>`, then `@@` to go back. A second footnote becomes `[^2]` and its note is added under `[^1]: ...`.

The off-by-one at the end of a line (tested): in insert mode with the cursor at the very end of a line, `^^` puts the mark one character too early (`Neovim is fas[^1]t`). The reason is that the `<C-O>` command inside the map first moves the cursor onto the last character, and `^^` inserts before that character. Typing `^^` in the middle of a line is exact. At the end of a line, end the sentence in normal mode instead: `<Esc>`, `<Space>mf` (inserts after the last character). In normal mode `^^` inserts before the character under the cursor, `<Space>mf` after it.

Reference links and tables

```
:AddRef docs https://neovim.io/doc
```

Tested result at the end of the buffer (a blank line, a comment line once, then the definition):

```
<!-- Reference links -->
[docs]: https://neovim.io/doc
```

A second `:AddRef two https://a.b` adds only `[two]: https://a.b` under it (the comment line is not repeated). In the text you write `[the docs][docs]`. The first argument completes from labels already used as `[text][label]` in the file. Label and URL cannot contain spaces; there is no title argument.

Tables (tested): select the table lines and run `:'<,'>Tabularize /|`:

```
a	b
longer cell	x
```

Tabularize pads the separator row with spaces. `<Space>fm` is the better table aligner: prettier makes the separator row `| ----- | --- |` (tested).

Checkboxes (`- [ ]`): render-markdown draws them; no key toggles them.

Preview (markdown-preview.nvim)

Why: render-markdown shows structure in the editor, the browser preview shows the final look (fonts, images, math, diagrams).

How: the plugin starts a small node server and opens your default browser. The server follows the text and the cursor while you edit.

Config (from `lua/plugin_specs.lua`):

```
{
  "iamcco/markdown-preview.nvim",
  build = "cd app && npm install && git restore .",
  ft = { "markdown" },
  init = function()
    -- Do not close the preview tab when switching to other buffers (all platforms)
    vim.g.mkdp_auto_close = 0
  end,
},
```

| Command | What it does |
|-------------------------------------|--|
| <code>:MarkdownPreview</code> | Start the preview and open the browser |
| <code>:MarkdownPreviewStop</code> | Stop it |
| <code>:MarkdownPreviewToggle</code> | Start or stop (what <code><Alt-m></code> runs) |

- Tested: the three commands exist in a Markdown buffer and `vim.g.mkdp_auto_close` is `0`.
- Tested server life cycle (browser launch replaced by a no-op function, port fixed to 18765): before `<Alt-m>` nothing listens; after `<Alt-m>` a server listens on `127.0.0.1:18765` and the plugin reports `Preview page: http://localhost:18765/page/1`; after the second `<Alt-m>` the port is closed. By default the port is random and only the local machine can connect (`mkdp_open_to_the_world = 0`).
- To try this yourself without a browser tab: `:let g:mkdp_browserfunc = 'NoBrowser'` after defining `function! NoBrowser(url)` that does nothing, and `:let g:mkdp_echo_preview_url = 1` to see the address.
- `mkdp_auto_close = 0`: the browser tab stays open when you switch to another buffer. It ends when you press `<Alt-m>` again or leave Neovim.
- No browser, port or theme is set, so the plugin defaults and your system default browser are used.
- Outside Markdown buffers `<Alt-m>` shows one warning (the real map is buffer-local in `after/ftplugin/markdown.lua`).
- After a fresh install the server may not be built: `:Lazy build markdown-preview.nvim` (needs `npm`).

Rendering inside the buffer (render-markdown.nvim)

Why: you read Markdown all day; rendered headings and tables are easier on the eyes, and the raw text is one `<Esc>`-then-`i` away.

Config (from `lua/plugin_specs.lua`):

```
opts = {
  debounce = 500,
  render_modes = { "n", "c" },
  max_file_size = 1.5,
  anti_conceal = { enabled = true },
},
```

| Setting | Meaning |
|----------------------------------|--|
| <code>render_modes = n, c</code> | Rendered only in normal and command mode; insert mode shows the raw text |
| <code>debounce = 500</code> | Waits 0.5 s after typing stops before redrawing |
| <code>max_file_size = 1.5</code> | Files over 1.5 MB are not rendered (and the big-file mode takes over, see below) |
| <code>anti_conceal</code> | The line under the cursor shows raw text so you can edit it |

Commands (all tested to run without error; the global state changed `true`, `false`, `true` with two `toggle` calls):

| Command | Effect |
|---|-------------------------------------|
| <code>:RenderMarkdown toggle</code> | Rendering off / on in all buffers |
| <code>:RenderMarkdown buf_toggle</code> | Same, this buffer only |
| <code>:RenderMarkdown enable / disable</code> | Explicit on / off |
| <code>:RenderMarkdown buf_enable / buf_disable</code> | Explicit on / off, this buffer only |

An unknown name (for example `:RenderMarkdown bogus`) gives the plugin's error `invalid command - bogus`. `:checkhealth render-markdown` shows the setup.

Big files: above 1.5 MB (or lines averaging over 5000 characters) the file gets the filetype `bigfile`: no tree-sitter, no ftplugin keys (so no `<Alt-m>`, `<Space>fm`, `^^`), and `ltex_plus` and `typos_lsp` are not attached. `:set ft=markdown` brings the full mode back (can be slow on huge files).

Formatting with prettier

Why: marksman has no formatting. Without this key `<Space>fm` (the global format key) would do nothing in Markdown.

How: the key runs `prettier --parser markdown --stdin-filepath <file>` on the buffer text (so a project `.prettierrc` is honoured) and writes back only the changed hunks. Tested result on the example file:

```
* item a      -> - item a
* item b      -> - item b
(two blank lines) -> (one blank line)
| a | b |      -> | a          | b |
|---|---|      -> | ----- | --- |
```

The code block and the `plain one / plain two` lines (two lines, no blank between) stayed as they were. Also `*emphasis*` becomes `_emphasis_` and a final newline is added (prettier defaults, not shown in the test).

Why not `:%!prettier --parser markdown`? It would replace the whole buffer: one giant change, all marks lost, and the cursor jumps. The key changes only the differing lines, in one undo step (tested: `u` restores everything), keeps marks and puts the cursor back on the same text. Nothing is written to disk. If you type while prettier runs, the result is discarded with a warning (`prettier: buffer changed while formatting, result discarded`): press the key again. A prettier error appears as `prettier failed: ...`.

Trailing spaces: two spaces at the end of a line are a Markdown hard line break, so they are never stripped here. The whitespace plugin excludes `markdown` (`trailing_whitespace_exclude_filetypes`), and `<Space><Space>` only warns. The other hard-break form is a trailing backslash; `<Space>mb` adds it. For rewrapping long paragraphs use `gq` after `:set textwidth=80` yourself.

PDF export (`:ToPDF`)

Why: a Markdown note you can send to someone.

How: `plugin/command.vim` starts pandoc as a background job:

```
pandoc --pdf-engine=xelatex --highlight-style=zenburn --table-of-content
--include-in-header=<config>/resources/head.tex -V fontsize=10pt -V colorlinks
... -s "<file>.md" -o "<file>.pdf"
```

- The output is `<same name>.pdf` next to the file; an old PDF is overwritten.
- You get a table of contents, coloured links, zenburn code colours, and your LaTeX header `resources/head.tex`.
- Spaces and special characters in the file name are safe (an argument list, no shell). Tested on `my notes.md`: a valid PDF, no error.
- Save first. An unnamed buffer gives `ToPDF: save the buffer to a file first`, and unsaved edits are not in the PDF.
- The first run took more than 25 seconds. Neovim stays usable. There is no message when it finishes.
- Success is silent and on Linux no viewer opens (a viewer is started only on macOS and Windows): open the PDF yourself.
- Failure: one warning `ToPDF: pandoc failed (exit N)`. The usual reason is that xelatex is missing because Neovim was not started in the latex dev shell. To see the real LaTeX error, run the pandoc command in a terminal.

To use it: `cd` into a folder with the latex shell (direnv), run `nvim "my notes.md"`, then `:ToPDF`.

Writing quality

ltex_plus (grammar, LanguageTool)

Why: spell checkers do not see "their" vs "there". ltex_plus checks grammar in prose files: `markdown`, `tex`, `plaintex`, `typst`, `gitcommit`, `text`.

How: it is a Java language server that sends problems as diagnostics.

Config (`after/lsp/ltex_plus.lua`):

```
settings = { ltex = {
  language = "en-US",
  enabled = { "markdown", "latex", "tex", "plaintex", "typst", "gitcommit", "git-commit",
    "plaintext", "text" },
  ["ltex-ls"] = { logLevel = "warning" },
} },
```

- The language is fixed to `en-US`: LanguageTool checks one language per file. Italian, German or French text is flagged; change `language` in that file (`"auto"` detects the language, less reliable on short texts) or run `:lsp stop` for the buffer.
- Tested: the status line shows `Completed Checking document` and `ltex_plus` after opening a file. The first start is slow.
- Use the diagnostics: `]d / [d`, `<Space>dd` (message), `<Space>ca` (fixes).
- No personal dictionary or disabled-rule list is configured in the repo.
- Tested code actions on an unknown word (`Zorblat`, `<Space>ca`): `Use 'Format'`, `Use 'Combat'`, `Use 'Orbit'`, `Use 'Cobalt'`, `Use 'Oblast'`, `Add 'Zorblat' to dictionary`, `Hide false positive`, `Disable rule`, and `Create a Table of Contents` (from `marksman`). Choosing a replacement works: the word changes and `u` undoes it.

- `Add ... to dictionary`, `Hide false positive` and `Disable rule` do NOT work in this setup (tested). They are client-side commands (`_ltex.addToDictionary ...`) that Neovim must implement, and your config has no handler; the server answers `Unknown command '_ltex.addToDictionary'`, ignoring, nothing is saved, the warning stays. To silence a word use Vim's `zg` (see below), which only affects the Vim spell checker, not `ltex_plus`; to silence `ltex_plus` for a word, add it to the `ltex.dictionary` setting in `after/lsp/ltex_plus.lua` yourself.

Vim spell checking

| Key | What it does |
|-----------------------------------|----------------------------------|
| <code><Space>cz</code> | Toggle <code>spell</code> |
| <code>]s</code> / <code>[s</code> | Next / previous misspelled word |
| <code>z=</code> | Up to 9 suggestions |
| <code>zg</code> | Add the word to the allowed list |
| <code>zw</code> | Mark the word as wrong |

`spelllang` is `en,it,de,fr` together, so a word correct in any of them is fine. `zg` writes to the first list `spell/en.utf-8.add`; `2zg` writes to the second (`it`), `3zg` to `de`, `4zg` to `fr`. The `spell/` folder is in a public repo: do not add private words (see `spell/README.md`).

typos_lsp

Checks common typos in every normal buffer (not help, terminal, quickfix, or start screens). Diagnostics and fixes work as for `ltex_plus`.

Troubleshooting

| Problem | Cause and fix |
|---|---|
| No rendering, raw <code>#</code> and <code>**</code> visible | Insert mode (press <code><Esc></code>), file over 1.5 MB, or rendering toggled off: <code>:RenderMarkdown enable</code> . Check <code>:set ft?</code> is <code>markdown</code> |
| <code><Alt-m></code> says "only in markdown buffers" | The buffer is not <code>markdown</code> : <code>:set ft=markdown</code> |
| Preview does not open | Server not built or no browser: <code>:Lazy build markdown-preview.nvim</code> (needs <code>node</code> and <code>npm</code>) |
| Preview tab stays open after I left the file | Intended (<code>mkdp_auto_close = 0</code>). Stop it with <code><Alt-m></code> in the Markdown buffer |
| <code><Space>fm</code> warns "prettier not found on PATH" | Start Neovim from a shell that has prettier (nix profile or dev shell) |
| <code><Space>fm</code> : "buffer changed while formatting" | You typed during the run. Press the key again |
| <code>:ToPDF</code> : "pandoc not found" / "pandoc failed (exit N)" | Start Neovim inside the latex dev shell. Run the pandoc command in a terminal to see the error |
| <code>:ToPDF</code> does nothing visible | Normal: silent, 30 s or more the first time, no viewer on Linux. Look for the PDF next to the file |
| <code>@@</code> does not replay my macro | In Markdown it means "return from footnote". Use <code>@a</code> (register name) |
| Typing <code>^^</code> or <code>@</code> has a delay | The <code>^^</code> / <code>@@</code> insert maps wait 500 ms for a second key. Accepted |
| <code>^^</code> puts the mark one letter early | At the end of a line in insert mode (see Footnotes). Use <code><Space>mf</code> in normal mode |
| <code><Space><Space></code> does not remove trailing spaces | Intended in Markdown (hard line breaks) |
| Grammar warnings on Italian or German text | <code>ltx_plus</code> is <code>en-US</code> only |
| <code>^^</code> does nothing | The footnote plugin is not loaded: <code>:echo exists(':FootnoteNumber')</code> must give <code>2</code> (tested) |
| Maps do nothing in a huge file | Big-file mode: <code>:set ft=markdown</code> |

Related sections

Section 16 (Code Commenting: `gc` in Markdown writes `<!-- -->`; `gcs` / `gcr` use the comment style of the fence language inside a fenced block), section 27 (short Markdown key list), section 28 (LaTeX and Typst: same latex dev shell and `ltx_plus`), and the sections on spelling, LSP diagnostics and big-file mode.

Chapter 16: Typst

82. Typst (typst.vim, tinymist, watch and preview)

Typst is a modern markup language that compiles to PDF (like LaTeX, but much faster and simpler). This section covers everything that is specific to Typst files (`*.typ`): the plugin, the language server, the compile-and-preview workflow, prose checking and the files that make it work. Global tools (completion, folding, quickfix, spell keys) are only mentioned with a pointer to their own section.

What you get

| Piece | What it does | Where it comes from |
|---|--|---|
| <code>kaarmu/typst.vim</code> | <code>:TypstWatch</code> , <code>:Toc / :Toch / :Tocv / :Toct</code> , <code>:make</code> (compiles once), indentation, <code>//</code> comments, Typst syntax colours | plugin, loaded only when <code>typst</code> is on PATH |
| <code><Space>tw</code> | Starts <code>typst watch</code> for the current file (recompile on every save, open the PDF) | <code>after/ftplugin/typst.lua</code> |
| <code>tinymist</code> | Language server: diagnostics, completion, hover, go to definition, rename, symbols, formatter | <code>typst devShell</code> |
| <code>typstyle</code> | The formatter that <code>tinymist</code> uses (<code><Space>fm</code>) | <code>typst devShell</code> |
| <code>zathura</code> | PDF viewer that reloads by itself when the PDF changes | <code>typst devShell</code> |
| <code>ltex_plus</code> and <code>typos_lsp</code> | Grammar, spelling and typo diagnostics in the prose | global (see "Prose checking") |
| Buffer settings | <code>textwidth=100</code> , wrap on, colour marker at column 100, indent of 2 spaces | <code>after/ftplugin/typst.lua</code> , <code>lua/options.lua</code> , <code>typst.vim</code> |

There are no Typst snippets (`my_snippets/` has no `typst.snippets`) and no Typst-specific completion source: completion comes from the language server (section 14).

Requirements: the Typst devShell

The programs `typst`, `tinymist`, `typstyle` and `zathura` come from a Nix devShell, not from the system. The template is `~/nix/templates/krit/dev-environments/language-specific/typst/flake.nix`; its package list is:

```
packages = with pkgs; [ typst typstyle typstwriter tinymist prettypst utpm zathura ]
```

A project enters it through direnv. The `.envrc` of a Typst project contains one line:

```
use_dev_env typst
```

Then `cd` into the project (run `direnv allow` once per project). Start Neovim from that shell, so it sees the programs. Check:

```
which typst tinymist typstyle zathura
```

Outside the devShell

| What | Result |
|-----------------------------------|---|
| <code>typst</code> not on PATH | The plugin <code>typst.vim</code> does not load: no <code>:TypstWatch</code> , no <code>:Toc</code> . Tested: <code>exists(':TypstWatch')</code> is 0 |
| <code><Space>tw</code> | Still mapped (buffer-local); it shows ONE warning: "Typst: typst not found on PATH (open nvim inside the typst devShell)". Nothing starts |
| <code>tinymist</code> not on PATH | No Typst language server. Tested: only <code>ltex_plus</code> and <code>typos_lsp</code> attach |
| Filetype | <code>*.typ</code> is still recognised as <code>typst</code> ; prose checking (<code>ltex_plus</code> , <code>typos_lsp</code>) still works |

Quick start

1. In a terminal, enter the project folder (the devShell loads through direnv).
2. Open a file: `nvim main.typ`.
3. Wait a few seconds until the statusline on the right shows `tinymist` (it is the main server; `(+2)` means two more servers: `ltex_plus` and `typos_lsp`).
4. Press `<Space>tw`. The command line shows `Starting: typst watch --diagnostic-format short 'main.typ' --open zathura` and the PDF opens in zathura.
5. Edit the text and save with `:w` (Typst files are never saved automatically, see "Auto-save" below). The PDF is rebuilt and zathura reloads it.
6. If the document has an error, a quickfix window opens at the bottom with the message. Fix it and save again: the window closes by itself.
7. To stop: quit Neovim (`:qa`). The watcher is a child job and ends with Neovim (tested: no `typst watch` process was left). Closing the zathura window alone does not stop the watcher.

Keys and commands

| Key / command | Where | Description |
|--|--------------------|---|
| <code><Space>tw</code> | Typst buffers only | Runs <code>:TypstWatch :</code>
<code>typst watch --diagnostic-format short <file> --open <viewer></code> as a background job. Pressing it again stops the old job and starts a new one (tested: new process id) |
| <code>:TypstWatch {args}</code> | needs typst.vim | Same, with extra <code>typst</code> options, e.g. <code>:TypstWatch --root ..</code> |
| <code>:make</code> | needs typst.vim | Compiles the file once (<code>typst compile --diagnostic-format short %</code>); errors go to the quickfix list (section 26) |
| <code>:Toc / :Tocv</code> | needs typst.vim | Table of contents of the = headings in a vertical location list on the right; <code>:Toch</code> horizontal, <code>:Toct</code> in a new tab. Press <code>Enter</code> on a line to jump |
| <code><Space>fm</code> | global | Format the whole file with the language server (typstyle). Tested: <code>#greet("x") #let y=3</code> became <code>#greet("x") #let y = 3</code> |
| <code>gq</code> | global | Reformats prose lines to <code>textwidth</code> (100); not the Typst formatter |
| <code>K</code> | LSP | Hover: signature and parameters of a function (tested on <code>greet</code> and <code>text</code>) |
| <code>gd</code> | LSP | Go to definition: from a function call to its <code>#let</code> , from <code>@intro</code> to the <code><intro></code> label (tested) |
| <code><Space>rn</code> | LSP | Rename symbol (<code>tinymist</code> supports rename) |
| <code><Space>ca</code> | LSP | Code action menu at the cursor |
| <code>]d / [d , <Space>dd</code> | global | Next / previous diagnostic; <code><Space>dd</code> shows a float with the diagnostic of the current line (section 13) |
| <code>gcc</code> | global | Comment line with <code>//</code> (typst.vim sets <code>commentstring</code> to <code>// %s</code>) |
| <code><Space>t</code> | global | Symbol outline (aerial, section 37); in Typst buffers see the note below |
| <code>[t /]t</code> | aerial | Previous / next symbol (heading or <code>#let</code>) |

Note on `<Space>t`: in Typst buffers `<Space>tw` exists next to the global `<Space>t` (outline). When you press `<Space>t` alone, Neovim waits `timeoutlen` (500 ms) for a possible `w` before it opens the outline. This is expected; press `<Space>t` and wait half a second.

The `tinymist` server also advertises extra commands (`tinymist.exportPdf`, `tinymist.startDefaultPreview`, `tinymist.exportSvg`, `tinymist.pinMain`, ...). No key is bound to them; this setup uses `typst watch` instead (see next part). The setting `exportPdf = "never"` stops `tinymist` from writing PDFs on its own.

The language server: tinymist

Why: it understands Typst while you type, without compiling to a PDF. Config (`lua/config/lsp.lua`):

```

tinymist = {
  cmd = { "tinymist" },
  filetypes = { "typst" },
  root_dir = function(bufnr, on_dir)
    local fname = vim.api.nvim_buf_get_name(bufnr)
    local root = vim.fs.dirname(vim.fs.find({ "typst.toml", ".git" }, { path = fname, upward
= true }))[1])
    or vim.fs.dirname(fname)
    on_dir(root)
  end,
  settings = {
    exportPdf = "never",
    outputPath = "$root/out/$name",
    formatterMode = "typstyle",
  },
},

```

| Setting | Meaning |
|---|--|
| <code>root_dir</code> | The project root is the nearest folder above the file with <code>typst.toml</code> (a Typst package) or <code>.git</code> ; with neither, the folder of the file. Tested: a file in a folder without <code>.git</code> got that folder as root |
| <code>exportPdf = "never"</code> | The server does not export PDFs; only <code>typst watch</code> does. So there is one PDF writer, not two |
| <code>outputPath</code> | Only used when the server exports (it does not here); it is a leftover default |
| <code>formatterMode = "typstyle"</code> | <code><Space>fm</code> formats with typstyle |

What it provides, as tested in a real session:

| Feature | How to see it |
|-------------|--|
| Diagnostics | A broken line (<code>#greet(1 , 2)</code> with too many arguments) showed <code>typst:18:unexpected argument in vim.diagnostic.get()</code> and the error sign in the gutter. Source name: <code>typst</code> |
| Completion | 38 items for <code>#set te...</code> (array literal, <code>cite</code> , <code>context expression</code> , ...) in the completion menu (section 14) |
| Hover | <code>K</code> on a function shows its signature, e.g. <code>let greet(name: str) = str</code> |
| Definition | <code>gd</code> on a call or on a <code>@label</code> |
| Symbols | The outline (<code><Space>t</code>) lists headings and <code>#let</code> definitions |
| Formatter | <code><Space>fm</code> (typstyle) |

Check which servers are attached: `:LspAttached` (popup) or the statusline (right side, first name is the main server `tinymist`, (+2) the others). `:LspInfo` shows details; `:LspLog` opens the log (section 13).

Watch, compile and the PDF

`<Space>tw` is defined like this (`after/ftplugin/typst.lua`):

```

if vim.fn.executable("typst") == 1 then
  vim.keymap.set("n", "<leader>tw", "<cmd>TypstWatch<cr>",

```

```

    { buffer = true, desc = "Typst: watch & recompile" })
else
    vim.keymap.set("n", "<leader>tw", function()
        vim.notify("Typst: typst not found on PATH (open nvim inside the typst devShell)",
            vim.log.levels.WARN)
    end, { buffer = true, desc = "Typst: watch & recompile (needs typst)" })
end

```

The plugin settings (`lua/plugin_specs.lua`):

```

vim.g.typst_auto_open_quickfix = 1
vim.g.typst_pdf_viewer = vim.env.TYPST_PDF_VIEWER or (utils.executable("zathura") and
    "zathura") or ""

```

| Question | Answer |
|--------------------------------------|---|
| What runs? | <code>typst watch --diagnostic-format short '<file>' --open <viewer></code>
in a background job, started from the current folder of Neovim |
| Which viewer? | <code>\$TYPST_PDF_VIEWER</code> if set, else <code>zathura</code> if installed, else (empty value) <code>--open</code> alone, i.e. the system default PDF program |
| Where is the PDF? | Next to the <code>.typ</code> file with the same name (<code>main.typ</code> gives <code>main.pdf</code>). Tested. (<code>g:typst_output_to_tmp</code> is not set, so <code>/tmp/typst_out</code> is not used) |
| When does it recompile? | Every time the <code>.typ</code> file (or a file it imports) is saved |
| Why is auto-save off for Typst? | A recompile on every focus change would restart the watcher over and over; save with <code>:w</code> (section 42) |
| Where are compile errors? | In the quickfix list (opens at the bottom, cursor stays in your window). Lines look like <code>main.typ 19 col 13 error: unexpected argument .</code>
jq-style quickfix keys are in section 26 |
| What happens to the PDF on an error? | The old PDF stays (tested: file time unchanged) until the error is fixed |
| Does a fixed error clear the window? | Yes: the next successful compile empties the list and closes the window (tested) |
| How to stop? | Quit Neovim. A second <code><Space>tw</code> replaces the running watcher. There is no stop command |
| One watcher only | The plugin keeps one watcher job; watching another file with <code><Space>tw</code> stops the first |

To test the viewer by hand without Neovim: `zathura main.pdf` (reloads on change).

Prose checking in Typst

| Tool | Result in a <code>.typ</code> file |
|---|---|
| <code>ltex_plus</code> | LanguageTool
(English, <code>en-US</code>) on the text: <code>'smal': Possible spelling mistake found., Don't put a space before the full stop.. Source name LTeX. Needs ltex-ls-plus installed globally</code> |
| <code>typos_lsp</code> | Common typos: <code>'smal' should be 'small'</code> . Source name <code>typos</code> |
| <code><Space>cz</code> | Toggles Vim's own spell checker (section 31); <code>zg</code> adds a word to your English list, <code>2zg / 3zg / 4zg</code> Italian/German/French |
| <code><Space>ca</code> on a warning | Offers the fix (replace the word) |

All three appear as diagnostics together with the `tinymist` ones (use `jd`, `<Space>dd`). Typst code is not skipped: LanguageTool may complain about markup such as `#link(...)`. Both tools never attach to files that are too big (section 42).

Filetype settings

| Setting | Value | From |
|--|--|---|
| <code>textwidth</code> | 100 | <code>after/ftplugin/typst.lua</code> |
| <code>wrap</code> | on | <code>after/ftplugin/typst.lua</code> |
| <code>colorcolumn</code> | 100 | <code>lua/options.lua</code> (per-language table) |
| <code>expandtab</code> , <code>shiftwidth</code> | spaces, 2 | <code>typst.vim</code> |
| <code>commentstring</code> | <code>// %s</code> | <code>typst.vim</code> |
| <code>iskeyword</code> | letters, digits, <code>_</code> , <code>-</code> and accents | <code>typst.vim</code> : <code>ciw</code> on <code>my-label</code> takes the whole word |
| Folding | off (<code>typst_folding = 0</code> ; <code>foldmethod=manual</code>); folds in general come from <code>nvim-ufo</code> (section 18) | <code>plugin_specs.lua</code> |
| Concealing | off (<code>typst_conceal*</code> all 0): you see the raw source | <code>plugin_specs.lua</code> |
| Auto-save | never for Typst files | <code>plugin_specs.lua</code> (auto-save condition) |

Example document

Save as `main.typ` in a folder with the devShell:

```
#set page(paper: "a5")
#set text(size: 11pt)
#set heading(numbering: "1.")

= Introduction <intro>

This is a smal test document with a typo. See @intro and #link("https://typst.app")[Typst].

#let greet(name) = [Hello, #name!]
```

```
#greet("World")

== Math

$ E = m c^2 $

- first item
- second item
```

What to try

| Do | Expected result |
|--|--|
| Open the file, wait 5 seconds | Statusline shows <code>tinymist (+2)</code> ; signs for line 7 (<code>smal</code>) |
| <code><Space>tw</code> | Mes-
sage Starting: typst watch ... --open zathura;
<code>main.pdf</code> appears; zathura opens it (tested) |
| Change <code>a5</code> to <code>a4</code> , <code>:w</code> | The PDF is recompiled and zathura shows the bigger page |
| Add a line <code>#greet(1 , 2)</code> at the end, <code>:w</code> | Quickfix window with <code>error: unexpected argument</code> ; the error sign on the line; PDF unchanged |
| Delete that line, <code>:w</code> | Quickfix window closes; PDF updated |
| <code>gd on @intro</code> | Cursor jumps to the heading line
<code>= Introduction <intro></code> |
| <code>gd on greet in #greet("World")</code> | Cursor jumps to <code>#let greet(name) = ...</code> |
| <code>K on greet</code> | Floating window with <code>let greet(name: str) = str</code> |
| Type <code>#set te</code> and wait | Completion menu with <code>text</code> and others |
| Type <code>#greet("x") #let y=3</code> then <code><Space>fm</code> | <code>#greet("x") #let y = 3</code> |
| <code>:Tocv</code> | Location list at the right with <code>Introduction</code> and <code>Math</code> |
| <code><Space>t</code> and wait | Symbol outline (aerial): <code>Introduction</code> , <code>Math</code> , <code>greet</code> |
| <code>:make</code> | Compiles once; empty quickfix when everything is fine |

Troubleshooting

| Symptom | Cause | Fix |
|---|--|---|
| <code><Space>tw</code> shows "Typst: typst not found on PATH" | Neovim was not started inside the devShell | <code>cd</code> into the project (direnv), check <code>which typst</code> , restart Neovim |
| E492: Not an editor command: TypstWatch | Same: <code>typst.vim</code> did not load without <code>typst</code> | Same fix |
| No <code>tinymist</code> in the statusline | <code>tinymist</code> not on PATH, or file is not <code>typst</code> filetype | <code>which tinymist</code> ; <code>:set ft?</code> must say <code>typst</code> ; <code>:LspAttached</code> |
| <code>direnv: errorenvrc is blocked</code> | The project's <code>.envrc</code> is not approved | <code>direnv allow</code> once in the project |
| Watch starts but the PDF does not change | The file has a compile error (old PDF stays), or you did not save | Look at the quickfix window or <code>:copen</code> ; save with <code>:w</code> |
| Watch compiles the wrong file or cannot find imports | Watcher started from another current folder than the file's | <code>:cd %:p:h</code> , then <code><Space>tw</code> again |
| Viewer does not open | <code>zathura</code> missing, or <code>\$TYPST_PDF_VIEWER</code> points to a missing program | <code>which zathura</code> ; <code>unset</code> or fix <code>TYPST_PDF_VIEWER</code> ; open <code>main.pdf</code> by hand |
| Two PDFs or a PDF in <code>out/</code> | A server export was enabled by hand | Keep <code>exportPdf = "never"</code> in <code>lua/config/lsp.lua</code> |
| Quickfix window did not open | The error has no <code>file:line:col</code> (e.g. a missing font warning) | Run <code>:!typst compile --diagnostic-format short %</code> and read the output |
| <code><Space>t</code> feels slow in Typst files | It waits for a possible <code>w</code> (<code><Space>tw</code>) | Expected; wait 500 ms, or use <code>:AerialToggle</code> |
| Warnings about prose inside markup | LanguageTool reads Typst source as text | Ignore, or <code><Space>cz/zg</code> for words; see section 31 |
| File not saved automatically | By design, Typst files are excluded from auto-save | <code>:w</code> |

Related sections

Section 13 (LSP), 14 (autocompletion), 18 (folding), 26 (quickfix and location list), 27 (Markdown), 28 (LaTeX and Typst overview), 31 (spell checking), 32 (statusline), 37 (symbol outline), 41 (filetype settings), 42 (automatic behaviours), 44 (LSP in depth).